

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO
CHUYÊN ĐỀ CÔNG NGHỆ PHẦN MỀM
CHỦ ĐỀ: CÁC THUẬT TOÁN ĐỐI SÁNH MẪU

Sinh viên thực hiện : Vũ Trọng Khôi
Mã sinh viên: B22DCCN468
Lớp: D22CNPM06
Giảng viên hướng dẫn: TS Nguyễn Duy Phương

Hà Nội, 05/2026

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN THUẬT TOÁN ĐỐI SÁNH XÂU	1
1.1. Bài toán đối sánh xâu	1
1.2. Các khái niệm cơ bản	1
1.3. Phân loại thuật toán phổ biến	2
CHƯƠNG 2. THUẬT TOÁN TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI	3
2.1. Thuật toán Knuth-Morris-Partt	3
2.2. Thuật toán Karp-Rabin.....	6
2.3. Thuật toán Shift Or.....	9
2.4. Thuật toán Not So Naïve.....	12
2.5. Thuật toán Morris-Pratt.....	14
2.6. Thuật toán Automat hữu hạn.....	17
2.7. Thuật toán Simon	21
CHƯƠNG 3. THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI	26
3.1. Thuật toán Horspool.....	26
3.2. Thuật toán Quick search	29
3.3. Thuật toán Smith	32
3.4. Thuật toán Raita	36
3.5. Thuật toán Zhu-Takaoka	40
3.6. Thuật toán Turbo BM	44
3.7. Thuật toán Berry-Ravindran	49
CHƯƠNG 4. THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH.....	54
4.1. Thuật toán Galil-Seiferas	54
4.2. Thuật toán Two way.....	59
4.3. Thuật toán String Matching on Ordered Alphabets	64
4.4. Thuật toán Optimal Mismatch algorithm.....	69
CHƯƠNG 5. THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ.....	74
5.1. Thuật toán Skip Search	74
5.2. Thuật toán Kmp Skip Search	78
5.3. Thuật toán Alpha Skip Search	83
KẾT LUẬN	92

CHƯƠNG 1. TỔNG QUAN THUẬT TOÁN ĐỐI SÁNH XÂU

1.1. Bài toán đối sánh chuỗi

Đối sánh chuỗi (String matching) là một chủ đề quan trọng trong lĩnh vực xử lý văn bản. Các thuật toán đối sánh chuỗi được xem là những thành phần cơ sở được cài đặt cho các hệ thống thực tế đang tồn tại trong hầu hết các hệ điều hành. Hơn thế nữa, các thuật toán đối sánh chuỗi cung cấp các mô hình cho nhiều lĩnh vực khác nhau của khoa học máy tính: xử lý ảnh, xử lý ngôn ngữ tự nhiên, tin sinh học và thiết kế phần mềm.

String-matching được hiểu là việc tìm một hoặc nhiều chuỗi mẫu (pattern) xuất hiện trong một văn bản (có thể là rất dài). Ký hiệu chuỗi mẫu hay chuỗi cần tìm là $X = (x_0, x_1, \dots, x_{m-1})$ có độ dài m . Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$ có độ dài n . Cả hai chuỗi được xây dựng từ một tập hữu hạn các ký tự Alphabet ký hiệu là Σ với kích cỡ là σ . Như vậy một chuỗi nhị phân có độ dài n ứng dụng trong mật mã học cũng được xem là một mẫu. Một chuỗi các ký tự ABD độ dài m biểu diễn các chuỗi AND cũng là một mẫu.

Input:

- Chuỗi mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_n)$, độ dài n .

Output:

- Tất cả vị trí xuất hiện của X trong Y

1.2. Các khái niệm cơ bản

Giả sử Alphabet là tập hợp (hoặc tập con) các mã ASCII. Một từ $w = (w_0, w_1, \dots, w_l)$ có độ dài l , $w_l = \text{null}$ giống như biểu diễn của ngôn ngữ C. Khi đó ta định nghĩa một số thuật ngữ sau:

- Prefix (tiền tố). Từ u được gọi là tiền tố của từ w nếu tồn tại một từ v để $w = uv$ (v có thể là rỗng). Ví dụ: $u = \text{"AB"}$ là tiền tố của $w = \text{"ABCDEF"}$ và $u = \text{"com"}$ là tiền tố của $w = \text{"communication"}$.
- Suffix (hậu tố). Từ v được gọi là hậu tố của từ w nếu tồn tại một từ u để $w = uv$ (u có thể là rỗng). Ví dụ: $v = \text{"EF"}$ là hậu tố của $w = \text{"ABCDEF"}$ và $v = \text{"tion"}$ là hậu tố của $w = \text{"communication"}$.
- Factor (substring, subword). Một từ z được gọi là một chuỗi con, từ con hay nhân tố của từ w nếu tồn tại hai từ u, v (u, v có thể rỗng) sao cho $w = uzv$. Ví dụ từ $z = \text{"CD"}$ là factor của từ $w = \text{"ABCDEF"}$ và $z = \text{"muni"}$ là factor của $w = \text{"communication"}$.
- Period (đoạn). Một số tự nhiên p được gọi là đoạn của từ w nếu với mọi i ($0 \leq i < m-1$) thì $w[i] = w[i+p]$. Giá trị đoạn nhỏ nhất của w được gọi là đoạn của w ký hiệu là $\text{pre}(w)$. Ví dụ $w = \text{"ABABCDEF"}$, khi đó tồn tại $p=2$.
- Periodic (tuần hoàn). Từ w được gọi là tuần hoàn nếu đoạn của từ nhỏ hơn hoặc bằng $l/2$. Trường hợp ngược lại được gọi là không tuần hoàn. Ví dụ từ $w = \text{"ABAB"}$ là từ tuần hoàn. Từ $w = \text{"ABABCDEF"}$ là không tuần hoàn.
- Basic word (từ cơ sở). Từ w được gọi là từ cơ sở nếu nó không thể viết như lũy thừa của một từ khác. Không tồn tại z và k để $z^k = w$.
- Boder word (từ biên). Từ z được gọi là boder của w nếu tồn tại hai từ u, v sao cho $w = uz = zv$. Khi đó z vừa là tiền tố vừa là hậu tố của w . Trong tình huống này $|u| = |v|$ là một đoạn của w .
- Reverse word (Từ đảo). Từ đảo của từ w có độ dài l ký hiệu là $wR = (w_{l-1}, w_{l-2}, \dots, w_1, w_0)$.

- Deterministic Finite Automata (DFA). Một automata hữu hạn A là bộ bốn (Q, q_0, T, E) trong đó:
 - Q là tập hữu hạn các trạng thái.
 - q_0 là trạng thái khởi đầu.
 - T là tập con của Q là tập trạng thái dừng.
- E là tập con của (Q, Σ, T) tập các chuyển dịch

1.3. Phân loại thuật toán phổ biến

- *Tìm kiếm mẫu từ bên trái qua bên phải*: Morris-Pratt Algorithm, Knuth-Morris-Pratt Algorithm, Finite Automata Algorithm, Karp-Rabin Algorithm, Shift Or Algorithm, Simon Algorithm, Colussi Algorithm, Galil-Giancarlo Algorithm, Apostolico-Crochemore Algorithm, Not So Naive Algorithm.
- *Tìm kiếm mẫu từ bên phải qua bên trái*: Horspool Algorithm, Quick Search Algorithm, Tuned Boyer-Moore Algorithm, Zhu-Takaoka Algorithm, Berry-Ravindran Algorithm, Smith Algorithm, Raita Algorithm, Turbo BM Algorithm, Apostolico-Giancarlo Algorithm, Reverse Colussi Algorithm.
- *Tìm kiếm mẫu theo thứ tự xác định*: Optimal Mismatch Algorithm, Maximal Shift Algorithm, Reverse Factor Algorithm, Turbo Reverse Factor Algorithm, Forward Dawg Matching Algorithm, Backward Nondeterministic Dawg Matching Algorithm, Backward Oracle Matching Algorithm, Two Way Algorithm, String Matching on Ordered Alphabets Algorithm.
- *Tìm kiếm mẫu từ vị trí bất kỳ*: Skip Search Algorithm, KMP Skip Search Algorithm, Alpha Skip Search Algorithm, Galil-Seiferas Algorithm.

CHƯƠNG 2. THUẬT TOÁN TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI

2.1. Thuật toán Knuth-Morris-Partt

2.1.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán KMP là "không bao giờ lùi con trỏ duyệt trên văn bản". Trong thuật toán tìm kiếm thông thường (Brute Force), khi xảy ra tình trạng lệch khớp (mismatch), ta phải dịch xâu mẫu lên 1 đơn vị và kiểm tra lại từ đầu, dẫn đến việc lặp đi lặp lại các phép so sánh vô ích.

KMP khắc phục điều này bằng cách: Tận dụng ngay những ký tự đã so khớp thành công trước đó để tự động tính toán xem xâu mẫu có thể "nhảy dịch" đi bao xa, giúp con trỏ trên văn bản luôn luôn tiến về phía trước ($O(1)$ bước tiến mỗi lần dịch). Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán KMP được triển khai tuần tự qua 2 bước sau:

Thuật toán PreKmp: //thực hiện bước tiền xử lý

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .

Output: Mảng giá trị kmpNext[].

Formats:

- PreKmp($X, m, \text{kmpNext}$);

Actions:

```
i = 1; kmpNext[0] = 0; len = 0; //kmpNex[0] luôn là 0
while (i < m) {
    if (X[i] == X[len] ) { //Nếu X[i] = X[len]
        len++; kmpNext[i] = len; i++;
    }
    else { // Nếu X[i] != X[len]
        if ( len != 0 ) { len = kmpNext[len-1]; }
        else { kmpNext[i] = 0; i++; }
    }
}
```

EndActions.

Thuật toán Knuth-Morris-Partt:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_n)$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: Knuth-Morris-Partt(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

preKmp($x, m, \text{kmpNext}$); //Tiền xử lý với độ phức tạp $O(m)$

Bước 2 (Lặp):

```
i = 0; j = 0;
while (i < n) {
```

```

if ( X[j] == Y[i] ) { i++; j++; }
if ( i == m ) {
    < Tìm thấy mẫu ở vị trí i-j>;
    j = kmpNext[j-1];
}
else if ( i < n && X[j] != Y[i] ) {
    if ( j != 0 ) j = kmpNext[ j-1 ];
    else i = i + 1;
}
}

```

EndActions.

2.1.2. Đặc điểm

- Thực hiện từ trái sang phải.
- Có pha tiền xử lý với độ phức tạp $O(m)$.
- Độ phức tạp thuật toán là $O(n + m)$;

2.1.3. Kiểm nghiệm thuật toán

*** Kiểm nghiệm PreKmp (X, m, kmpNext)**

Input: X[] = "AABAACAABAA", m = 11.

i=?	(X[i]== X[len])?	Len =?	kmpNext[i]=?
		Len=0	kmpNext[0]=0
i=1	('A'=='A'):Yes	Len=1	kmpNext[1]=1
i=2	('B'=='A'): No	Len=0	kmpNext[2]=chưa xác định
i=2	('B'=='A'): No	Len=0	kmpNext[2]=0
i=3	('A'=='A'):Yes	Len=1	kmpNext[3]=1
i=4	('A'=='A'):Yes	Len=2	kmpNext[4]=2
i=5	('C'=='B'): No	Len=1	kmpNext[5]=chưa xác định
i=5	('C'=='A'): No	Len=0	kmpNext[5]=chưa xác định
i=5	('C'=='A'): No	Len=0	kmpNext[5]=0
i=6	('A'=='A'):Yes	Len=1	kmpNext[6]=1
i=7	('A'=='A'):Yes	Len=2	kmpNext[7]=2
i=8	('B'=='B'):Yes	Len=3	kmpNext[8] = 3
i=9	('A'=='A'):Yes	Len=4	kmpNext[9] = 4

i=10	('A'='A'):Yes	Len=5	kmpNext[10] = 5
Kết luận: kmpNext = {0, 1, 0, 1, 2, 0, 1, 2, 3, 4, 5}			

Kiểm nghiệm Knuth-Moriss-Patt (X, m, Y, n):

- X[] = "ABABCABAB", m = 9.
- Y[] = "ABABDABACDABABCABAB", n = 19

Bước 1 (Tiền xử lý). Thực hiện Prekmp(X, m, kmpNext) ta nhận được:

kmpNext[] = { 0, 0, 1, 2, 0, 1, 2, 3, 4}

Bước 2 (Lặp):

(X[j] == Y[i])?	(j == 9)?	Giá trị con trỏ sau bước lặp
(X[0] == Y[0]): Yes ('A' == 'A')	No	i = 1, j = 1
(X[1] == Y[1]): Yes ('B' == 'B')	No	i = 2, j = 2
(X[2] == Y[2]): Yes ('A' == 'A')	No	i = 3, j = 3
(X[3] == Y[3]): Yes ('B' == 'B')	No	i = 4, j = 4
(X[4] == Y[4]): No ('C' != 'D')	No	i = 4, j = 2
(X[2] == Y[4]): No ('A' != 'D')	No	i = 4, j = 0
(X[0] == Y[4]): No ('A' != 'D')	No	i = 5, j = 0
(X[0] == Y[5]): Yes ('A' == 'A')	No	i = 6, j = 1
(X[1] == Y[6]): Yes ('B' == 'B')	No	i = 7, j = 2
(X[2] == Y[7]): Yes ('A' == 'A')	No	i = 8, j = 3
(X[3] == Y[8]): No ('B' != 'C')	No	i = 8, j = 1
(X[1] == Y[8]): No ('B' != 'C')	No	i = 8, j = 0
(X[0] == Y[8]): No ('A' != 'C')	No	i = 9, j = 0
(X[0] == Y[9]): No ('A' != 'D')	No	i = 10, j = 0
(X[0] == Y[10]): Yes ('A' == 'A')	No	i = 11, j = 1
(X[1] == Y[11]): Yes ('B' == 'B')	No	i = 12, j = 2
(X[2] == Y[12]): Yes ('A' == 'A')	No	i = 13, j = 3
(X[3] == Y[13]): Yes ('B' == 'B')	No	i = 14, j = 4
(X[4] == Y[14]): Yes ('C' == 'C')	No	i = 15, j = 5
(X[5] == Y[15]): Yes ('A' == 'A')	No	i = 16, j = 6
(X[6] == Y[16]): Yes ('B' == 'B')	No	i = 17, j = 7
(X[7] == Y[17]): Yes ('A' == 'A')	No	i = 18, j = 8
(X[8] == Y[18]): Yes ('B' == 'B')	No	i = 19, j = 9
(X[9] == Y[19]): Yes ('B' == 'B')	Yes	j = kmpNext[8] = 4
Tìm thấy mẫu tại vị trí: i - j = 19 - 9 = 10		

Kết luận kết quả kiểm nghiệm:

- Thuật toán kết thúc vòng lặp thành công khi i = 19 (đã duyệt hết chiều dài

- văn bản $n = 19$).
- Kết quả: Xâu mẫu X xuất hiện trong văn bản Y tại vị trí chỉ số 10 (bắt đầu đoạn "ABABCABAB" ở cuối văn bản Y).

2.1.4. Cài đặt thuật toán

Tiền xử lý preKMP

```
void preKmp(const char *x, int m, int kmpNext[]) {
    int i, j;
    i = 0;
    j = kmpNext[0] = -1;
    while (i < m) {
        while (j > -1 && x[i] != x[j]) {
            j = kmpNext[j];
        }
        i++;
        j++;
        if (x[i] == x[j]) {
            kmpNext[i] = kmpNext[j];
        }
        else {
            kmpNext[i] = j;
        }
    }
}
```

Thuật toán KMP

```
void KMP(const char *x, int m, const char *y, int n) {
    int i, j;
    int kmpNext[XSIZE]; // Mang kmpNext khai tạo từ preKmp

    /* Preprocessing */
    preKmp(x, m, kmpNext);

    /* Searching */
    i = j = 0;
    while (j < n) {
        while (i > -1 && x[i] != y[j]) {
            i = kmpNext[i];
        }
        i++;
        j++;
        if (i >= m) {
            cout << "-> Tìm thấy mẫu tại vị trí: " << i - j << endl;
            i = kmpNext[i];
        }
    }
}
```

2.2. Thuật toán Karp-Rabin

2.2.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Karp-Rabin là "sử dụng mã băm (hashing) để tăng tốc độ so khớp". Thay vì so khớp ký tự ngay từ đầu, thuật toán tính toán một giá trị băm cho xâu mẫu và cho từng "cửa sổ" ký tự trên văn bản. Nếu hai giá trị băm khác nhau, ta chắc chắn hai xâu không khớp. Chỉ khi giá trị băm trùng nhau ($h_x == h_y$), ta mới tiến hành so khớp chi tiết từng ký tự để tránh hiện tượng xung đột băm.

Thuật toán Karp-Rabin:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_n)$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: KarpRabin(X, m, Y, n);

Actions:

```
// 1. Tính toán giá trị dịch bit  $d = 2^{(m-1)}$ 
d = 1; i = 1;
while (i < m) {
    d = (d << 1); i++;
}
// 2. Tính mã băm ban đầu cho xâu mẫu (hx) và cửa sổ đầu tiên của văn bản (hy)
hy = 0; hx = 0; i = 0;
while (i < m) {
    hx = ((hx << 1) + X[i]);
    hy = ((hy << 1) + Y[i]);
    i++;
}

// 3. Vòng lặp tìm kiếm và cập nhật mã băm cuộn (Rolling Hash)
j = 0;
while (j <= n - m) {
    if ( hx == hy ) {
        if ( <Kiểm tra ký tự X[0..m-1] khớp hoàn toàn với Y[j..j+m-1]> ) {
            <Tìm thấy mẫu ở vị trí j>;
        }
    }
    j++;
    hy = ((hy - Y[j] * d) << 1) + Y[j + m];
}
```

EndActions.

2.2.2. Đặc điểm thuật toán

- Sử dụng một hàm băm (hashing function) để thực hiện so khớp.
- Pha tiền xử lý có độ phức tạp thời gian là $O(m)$ và tốn không gian bộ nhớ hằng số (constant space).
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$.
- Độ phức tạp thời gian trung bình kỳ vọng (expected running time) là $O(m + n)$.

2.2.3. Kiểm nghiệm thuật toán

Kiểm nghiệm Karp-Rabin (X, m, Y, n):

- $X = \{\text{"ABA"}\}$, $m = 3$.
- $Y = \{\text{"ABACABA"}\}$, $n = 7$.
- Mã ASCII giả định: 'A'=65, 'B'=66, 'C'=67.
- Tính toán khởi tạo: $d = 4$, mã băm mẫu $hx = 457$.

- Mã băm của số ban đầu tại vị trí đầu tiên: $hy = 457$.

$(hx == hy)?$	$(X[k] == Y[j+k])?$	$(k == 3)?$	$j = ? \quad hy = ?$
Bắt đầu ($j=0, hy=457$)			
$(457 == 457): \text{Yes}$	$(X[0] == Y[0]): \text{Yes}$	No	Đang xét $k=1$
	$(X[1] == Y[1]): \text{Yes}$	No	Đang xét $k=2$
	$(X[2] == Y[2]): \text{Yes}$	Yes	Tìm thấy mẫu ở vị trí $j=0$
			Cập nhật: $j=1, hy=460$
$(457 == 460): \text{No}$	(Bỏ qua so khớp ký tự)	No	Cập nhật: $j=2, hy=457$
$(457 == 457): \text{Yes}$	$(X[0] == Y[2]): \text{Yes}$	No	Đang xét $k=1$
	$(X[1] == Y[3]): \text{No}$	No	Sai lệch ký tự (Xung đột băm)
			Cập nhật: $j=3, hy=460$
$(457 == 460): \text{No}$	(Bỏ qua so khớp ký tự)	No	Cập nhật: $j=4, hy=457$
$(457 == 457): \text{Yes}$	$(X[0] == Y[4]): \text{Yes}$	No	Đang xét $k=1$
	$(X[1] == Y[5]): \text{Yes}$	No	Đang xét $k=2$
	$(X[2] == Y[6]): \text{Yes}$	Yes	Tìm thấy mẫu ở vị trí $j=4$
Kết thúc vòng lặp			Do $j=5 > (7-3)$

Kết luận: Xâu mẫu xuất hiện tại vị trí $j = \{0, 4\}$.

2.2.4. Cài đặt thuật toán

```
// Định nghĩa macro REHASH tối ưu bằng phép dịch bit tương tự trong ảnh mẫu
#define REHASH(a, b, h) (((h) - (a)*d) << 1) + (b))

// Hàm cài đặt thuật toán Karp-Rabin (KR) theo phong cách C++
void KR(const std::string& x, int m, const std::string& y, int n) {
    int d, hx, hy, i, j;

    /* Preprocessing */
    /* computes  $d = 2^{(m-1)}$  with the left-shift operator */
    for (d = i = 1; i < m; ++i) {
        d = (d << 1);
    }

    // Tính mã băm ban đầu cho xâu mẫu (hx) và của số đầu tiên của văn bản (hy)
    for (hy = hx = i = 0; i < m; ++i) {
        hx = ((hx << 1) + x[i]);
        hy = ((hy << 1) + y[i]);
    }

    /* Searching */
    j = 0;
    while (j <= n - m) {
```

```

        // Nếu mã băm trùng nhau, tiến hành so khớp chuỗi ký tự bằng compare
        (tương đương memcmp)
        if (hx == hy && y.compare(j, m, x) == 0) {
            cout << "Tìm thấy mau tại vị trí: " << j;
        }
        ++j;
        // Cập nhật mã băm cuộn (Rolling Hash) cho cửa sổ kế tiếp
        if (j <= n - m) {
            hy = REHASH(y[j - 1], y[j + m - 1], hy);
        }
    }
}

```

2.3. Thuật toán Shift Or

2.3.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Shift-Or là "sử dụng các kỹ thuật thao tác trên bit (bitwise) để thực hiện so khớp song song". Thay vì so khớp ký tự tuần tự, thuật toán mã hóa trạng thái khớp của các tiền tố thuộc mẫu thành một chuỗi bit. Điểm độc đáo của thuật toán này là quy ước đảo ngược trạng thái bit: giá trị 0 biểu thị trạng thái khớp (match), còn giá trị 1 biểu thị trạng thái lệch khớp (mismatch).

Khi duyệt qua văn bản, trạng thái bit tiếp theo được cập nhật siêu tốc chỉ bằng hai phép toán nền tảng cơ bản của phần cứng: Dịch bit trái ($\ll 1$) để dịch chuyển trạng thái khớp sang ký tự tiếp theo, và phép toán logic toán tử bitwise OR với mặt nạ mảng bit của ký tự hiện tại. Nếu bit tại vị trí cuối cùng dịch chuyển về giá trị 0, thuật toán ghi nhận mẫu xuất hiện hoàn toàn trên văn bản.

Thuật toán Shift-Or được triển khai tuần tự qua 2 bước sau:

Thuật toán PreShiftOr: //thực hiện bước tiền xử lý tạo mặt nạ bit và giá trị lẻ

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .

Output: Mảng mặt nạ ký tự $S[]$ và giá trị giới hạn kiểm tra lim .

Formats: PreShiftOr(X, m, S, lim);

Actions:

ASIZE = 256; $i = 0$;

// Khởi tạo tất cả các mặt nạ ký tự bằng chuỗi bit toàn 1

while ($i < ASIZE$) {

$S[i] = \sim 0$;

$i++$;

}

// Thiết lập bit 0 tại các vị trí ký tự xuất hiện tương ứng trong xâu mẫu X

$lim = 0$; $i = 0$; $j = 1$;

while ($i < m$) {

$S[X[i]] \&= \sim j$;

$lim |= j$;

$j = (j \ll 1)$;

$i++$;

}

$lim = \sim(lim \gg 1)$;

EndActions.

Thuật toán Shift-Or:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_m)$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_n)$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: ShiftOr(X, m, Y, n);**Actions:****Bước 1 (Tiền xử lý):**PreShiftOr(X, m, S, lim); //Tiền xử lý với độ phức tạp $O(m + \text{ASIZE})$ **Bước 2 (Lặp):**

```

state = ~0; j = 0;
while (j < n) {
    state = (state << 1) | S[Y[j]];
    if ( state < lim ) {
        <Tìm thấy mẫu ở vị trí j - m + 1>;
    }
    j++;
}

```

EndActions.**2.3.2. Đặc điểm thuật toán**

- Sử dụng các kỹ thuật bitwise xử lý song song trực tiếp trên thanh ghi máy tính.
- Đạt hiệu suất tối ưu tuyệt đối khi độ dài xâu mẫu m nhỏ hơn hoặc bằng kích thước từ nhớ (word-size) của kiến trúc phần cứng hệ thống (32-bit hoặc 64-bit).
- Pha tiền xử lý có độ phức tạp thời gian và không gian bộ nhớ là $O(m + \alpha)$ với α là kích thước bảng chữ cái hệ thống.
- Pha tìm kiếm có độ phức tạp thời gian tuyến tính lý tưởng là $O(n)$, hoàn toàn

2.3.3. Kiểm nghiệm thuật toánKiểm nghiệm Shift-Or (X, m, Y, n):

- $X = \text{"GAG"}; m = 3$.
- $Y = \text{"GCAGAG"}; n = 6$.

Bước 1 (Tiền xử lý): Thực hiện PreShiftOr(X, m, S, lim) ta nhận được:

- Mảng mặt nạ bit S (biểu diễn dạng nhị phân 3 bit thấp tương ứng $m=3$):
 - $S['G'] = 010$ (Bit 0 và Bit 2 bằng 0 vì khớp với ký tự 'G')
 - $S['A'] = 101$ (Bit 1 bằng 0 vì khớp với ký tự 'A')
 - Các ký tự khác không thuộc mẫu: $S['C'] = 111$
- Giá trị giới hạn kiểm tra điều kiện lẹ: $\text{lim} = 011$

Bước 2 (Lặp):

(state < lim)?	Ký tự Y[j] hiện tại	Thao tác bit cập nhật	j=? state=?
Bắt đầu (j=0, state=1112)			
(1112 < 0112): No	Duyệt Y[0]='G'	state=(1112<<1) 0102=1102	j=0, state=1102
(1102 < 0112): No	Duyệt Y[1]='C'	state=(1102<<1) 1112=1112	j=1, state=1112

(1112<0112): No	Duyệt Y[2]='A'	state=(1112<<1) 1012=1112	j=2,state=1112
(1112<0112): No	Duyệt Y[3]='G'	state=(1112<<1) 0102=1102	j=3,state=1102
(1102<0112): No	Duyệt Y[4]='A'	state=(1102<<1) 1012=1012	j=4,state=1012
(1012<0112): No	Duyệt Y[5]='G'	state=(1012<<1) 0102=0102	j=5,state=0102
(0102<0112): Yes		(Bit thứ m-1 dịch chuyển về số 0)	Tìm thấy mẫu ở vị trí j-m+1=3
Kết thúc vòng lặp			Do j=6≥6

Kết luận: Xâu mẫu xuất hiện tại vị trí $j = \{3\}$.

2.3.4. Cài đặt thuật toán

```
// Định nghĩa kích thước bảng chữ cái ASCII tiêu chuẩn
#define ASIZE 256

// Hàm Bước 1: Tiền xử lý (PreShiftOr)
unsigned int PreShiftOr(const std::string& x, int m, unsigned int S[]) {
    int i;
    unsigned int j, lim;

    // Khởi tạo mảng mặt nạ bằng chuỗi bit toàn 1 (~0)
    for (i = 0; i < ASIZE; ++i) {
        S[i] = ~0;
    }

    // Đặt bit 0 vào vị trí các ký tự có mặt trong mẫu x
    for (lim = i = 0, j = 1; i < m; ++i, j <= 1) {
        S[x[i]] &= ~j;
        lim |= j;
    }

    // Tính toán và trả về giá trị lẻ lim dùng để kiểm tra trạng thái khớp hoàn toàn
    lim = ~(lim >> 1);
    return lim;
}

// Hàm Bước 2: Tìm kiếm (Shift-Or chính)
void SO(const std::string& x, int m, const std::string& y, int n) {
    if (m > 32) {
        std::cerr << "SO: Use pattern size <= word size (32-bit)" <<
std::endl;
        return;
    }

    unsigned int lim, state;
    unsigned int S[ASIZE];
    int j;

    /* Bước 1: Gọi hàm tiền xử lý */
    lim = PreShiftOr(x, m, S);

    /* Bước 2: Vòng lặp tìm kiếm tuyến tính */
    for (state = ~0, j = 0; j < n; ++j) {
        state = (state << 1) | S[y[j]];

        // Điều kiện state < lim tương đương với việc bit thứ (m-1) bằng 0
        if (state < lim) {
```

```

std::cout << "Tìm thay mau tại vị trí: " << (j - m + 1) <<
std::endl;
    }
}
}

```

2.4. Thuật toán Not So Naïve

2.4.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Not So Naïve là "tối ưu hóa thuật toán vét cạn (Brute Force) bằng một bước tiền xử lý siêu nhanh với độ phức tạp $O(1)$. Thay vì luôn dịch xâu mẫu lên 1 đơn vị khi xảy ra lệch khớp, thuật toán tiến hành kiểm tra mối quan hệ giữa hai ký tự đầu tiên của xâu mẫu ($X[0]$ và $X[1]$) để quyết định hai hằng số dịch chuyển an toàn: $k1$ và $k2$.

Khi tiến hành tìm kiếm, tại mỗi cửa sổ, thuật toán thực hiện so khớp ký tự thứ hai ($X[1]$ với $Y[j+1]$) trước tiên. Nếu có sự sai lệch ký tự ngay tại vị trí này, cửa sổ tìm kiếm sẽ được dịch chuyển nhanh một khoảng bằng $k1$ hoặc $k2$ đơn vị dựa trên kết quả phân nhánh tiền xử lý. Nhờ cơ chế hoán đổi thứ tự so sánh (xét $X[1]$ trước $X[0]$), thuật toán loại bỏ được nhiều phép so khớp thừa ở các vị trí không khả thi. Để triển khai, thuật toán được chia làm 2 bước rõ ràng.

Thuật toán Not So Naïve được triển khai tuần tự qua 2 bước sau:

Thuật toán PreNotSoNaive: // Thực hiện tiền xử lý xác định bước nhảy ban đầu

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

Output: Hai giá trị bước nhảy dịch chuyển $k1$ và $k2$.

Formats: PreNotSoNaive($X, m, k1, k2$);

Actions:

```

if ( $X[0] == X[1]$ ) {
     $k1 = 2$ ;
     $k2 = 1$ ;
} else {
     $k1 = 1$ ;
     $k2 = 2$ ;
}
EndActions.

```

Thuật toán NotSoNaive:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: NotSoNaive(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreNotSoNaive($X, m, k1, k2$); // Xác định $k1, k2$ với độ phức tạp $O(1)$

Bước 2 (Lặp tìm kiếm):

$j = 0$;

```

while (j <= n - m) {
// Kiểm tra ký tự thứ hai của mẫu trước
if (X[1] != Y[j + 1]) {
    j = j + k1;
}
else {
// Nếu X[1] khớp, tiến hành so khớp ký tự đầu tiên X[0] và đoạn còn lại X[2..m-1]
if (X[0] == Y[j] && <Kiểm tra đoạn X[2..m-1] khớp hoàn toàn với Y[j+2..j+m-1]>) {
    <Tìm thấy mẫu ở vị trí j>;
}
j = j + k2;
}
}
}
EndActions.

```

2.4.2. Đặc điểm thuật toán

- Thực hiện việc dịch chuyển và so khớp cửa sổ tuần tự từ trái sang phải trên văn bản.
- Pha tiền xử lý cực kỳ tối ưu, diễn ra tức thời với độ phức tạp thời gian hằng số $O(1)$ và không tốn không gian bộ nhớ lưu trữ bổ sung (độ phức tạp không gian $O(1)$).
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$, tuy nhiên thời gian thực thi trung bình thực tế đạt hiệu năng tiệm cận tuyến tính, nhanh hơn đáng kể so với thuật toán Brute Force nhờ giảm số lượng phép so sánh ký tự đầu tiên.

2.4.3. Kiểm nghiệm thuật toán

Kiểm nghiệm NotSoNaive (X, m, Y, n):

Input:

- $X = \{\text{"ABAB"}\}$, $m = 4$.
- $Y = \{\text{"ABACABABAAB"}\}$, $n = 11$.
-

Bước 1 (Tiền xử lý): Gọi PreNotSoNaive(X, 4, k1, k2).

Vì $X[0] = \text{'A'}$ và $X[1] = \text{'B'}$ ($X[0] \neq X[1]$) $\Rightarrow$ Kết quả xác định bước nhảy: **k1 = 1, k2 = 2**.

Bước 2 (Lặp):

(X[1] == Y[j+1])?	So khớp ký tự chi tiết	(Khớp hoàn toàn)?	j=?
Bắt đầu (j=0)			
(Y[1] == 'B') $\Rightarrow$ Yes	$X[0] == Y[0]$ ('A' == 'A') và $X[2...3] \neq Y[2...3]$ ("AB" $\neq$ "AC")	No	Lệch khớp $\Rightarrow j = j + k2$ $= 0 + 2 = 2$
(Y[3] == 'C') $\Rightarrow$ No	(Bỏ qua so khớp ký tự còn lại)	No	Lệch tại X[1] $\Rightarrow j = j + k1$ $= 2 + 1 = 3$
(Y[4] == 'A')	(Bỏ qua so khớp ký tự còn lại)	No	Lệch tại X[1] $\Rightarrow j = j + k1$

⇒ No			=3+1=4
(Y[5]=='B') ⇒ Yes	X[0]==Y[4] ('A'=='A') và X[2...3]==Y[6...7] ("AB"=="AB")	Yes	Tìm thấy mẫu ở vị trí j=4Cập nhật: j=j+k2=4+2=6
(Y[7]=='B') ⇒ Yes	X[0]==Y[6] ('A'=='A') và X[2...3]≠Y[8...9] ("AB"≠"AA")	No	Lệch khớp ⇒ j=j+k2 =6+2=8
Kết thúc vòng lặp			Do j=8>(11-4)

2.4.4. Cài đặt thuật toán

Cài đặt thuật toán Not So Naive

```
// Bước 1: Pha tiền xử lý (Preprocessing)
void PreNotSoNaive(const std::string& x, int& k1, int& k2) {
    if (x[0] == x[1]) {
        k1 = 2;
        k2 = 1;
    } else {
        k1 = 1;
        k2 = 2;
    }
}

// Bước 2: Pha tìm kiếm (Searching)
void NotSoNaive(const std::string& x, int m, const std::string& y, int n) {
    int j = 0;
    int k1, k2;

    /* Gọi Bước 1: Tiền xử lý tính toán bước nhảy */
    PreNotSoNaive(x, k1, k2);

    /* Thực hiện Bước 2: Tìm kiếm chuỗi */
    while (j <= n - m) {
        // So khớp ký tự thứ hai trước (X[1] với Y[j + 1])
        if (x[1] != y[j + 1]) {
            j += k1;
        } else {
            // Nếu X[1] khớp, tiến hành đối sánh ký tự đầu tiên X[0] và phần
            // thân còn lại X[2...m-1]
            if (x[0] == y[j] && y.compare(j + 2, m - 2, x, 2, m - 2) == 0) {
                std::cout << "Tìm thấy mẫu tại vị trí: " << j << std::endl;
            }
            j += k2;
        }
    }
}
```

2.5. Thuật toán Morris-Pratt

2.5.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Morris-Pratt là "tận dụng tối đa thông tin thu được từ các ký tự đã so khớp trước đó để tránh việc lùi con trỏ duyệt trên văn bản". Trong thuật toán vét cạn, khi xảy ra tình trạng lệch khớp, ta phải dịch xâu mẫu lên 1 đơn vị và kiểm tra lại từ đầu, làm lãng phí các phép so sánh hợp lệ.

Morris-Pratt khắc phục điều này bằng cách: Tính toán trước một mảng bộ nhớ gọi là `mpNext[]`. Giá trị `mpNext[i]` lưu trữ độ dài của "biên" (border) dài nhất của chuỗi con $X[0 \dots i-1]$ (chuỗi vừa là tiền tố, vừa là hậu tố của đoạn đã khớp). Khi xảy ra lệch khớp giữa ký tự văn bản $Y[j]$ và ký tự mẫu $X[i]$, thay vì quay ngược con trỏ văn bản j , thuật toán chỉ cần chuyển sang so sánh ký tự $Y[j]$ với ký tự mẫu tại vị trí mới là $X[\text{mpNext}[i]]$. Nhờ vậy, con trỏ trên văn bản luôn tiến về phía trước một cách tuyến tính. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Morris-Pratt được triển khai tuần tự qua 2 bước sau:

Thuật toán PreMp: // Thực hiện bước tiền xử lý tính bảng nhảy biên

Input : - Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

Output: Mảng giá trị dịch chuyển `mpNext[]`.

Formats: `PreMp(X, m, mpNext);`

Actions:

```
i = 0;
j = -1;
mpNext[0] = -1;
while (i < m) {
    while (j > -1 && X[i] != X[j]) {
        j = mpNext[j];
    }
    i++;
    j++;
    mpNext[i] = j;
}
EndActions.
```

Thuật toán MorrisPratt:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: `MorrisPratt(X, m, Y, n);`

Actions:

Bước 1 (Tiền xử lý):

`PreMp(X, m, mpNext);` // Tiền xử lý tính mảng `mpNext` với độ phức tạp $O(m)$

Bước 2 (Lặp tìm kiếm):

```
i = 0; // Con trỏ vào xâu mẫu X
j = 0; // Con trỏ vào văn bản Y
while (j < n) {
    while (i > -1 && X[i] != Y[j]) {
        i = mpNext[i];
    }
    i++;
    j++;
    if (i >= m) {
        <Tìm thấy mẫu ở vị trí j - i>;
    }
}
```

```

        i = mpNext[i];
    }
}
EndActions.

```

2.5.2. Đặc điểm thuật toán

- Thực hiện việc so khớp ký tự tuần tự theo chiều từ trái sang phải trên văn bản.
- Pha tiền xử lý có độ phức tạp thời gian và không gian bộ nhớ đều là $O(m)$ nhằm khởi tạo bảng tra cứu `mpNext`.
- Pha tìm kiếm có độ phức tạp thời gian tuyến tính lý tưởng là $O(m + n)$, hoàn toàn độc lập với kích thước của bảng chữ cái.
- Số lượng phép so sánh ký tự văn bản tối đa trong pha tìm kiếm được giới hạn nghiêm ngặt ở mức $2n - 1$, và độ trễ tối đa (delay) cho một ký tự văn bản đơn lẻ được chặn bởi độ dài mẫu m .

2.5.3. Kiểm nghiệm thuật toán

Kiểm nghiệm MorrisPratt (X, m, Y, n):

- $X = \{\text{"GCAGAGAG"}\}$, $m = 8$.
- $Y = \{\text{"GCATCGCAGAGAG"}\}$, $n = 13$.

Bước 1 (Tiền xử lý): Thực hiện PreMp($X, 8, \text{mpNext}$) ta nhận được mảng dịch chuyển:

- $\text{mpNext}[] = \{-1, 0, 0, 1, 0, 1, 0, 1, 0\}$

Bước 2 (Lặp tìm kiếm):

$(X[i]==Y[j])?$	Điều kiện trạng thái khớp	$(i \geq 8)?$	Hành động dịch chuyển
Bắt đầu ($j=0, i=0$)			
Khớp liên tục đến $i=3$	$(X[0..2]==Y[0..2]): \text{Yes}$	No	Lệch tại $Y[3]='T' \neq X[3]='G'$
$(X[3] \neq Y[3]): \text{No}$	$i > -1 \Rightarrow i = \text{mpNext}[3] = 1$	No	Xét tiếp với ký tự mẫu $X[1]='C'$
$(X[1] \neq Y[3]): \text{No}$	$i > -1 \Rightarrow i = \text{mpNext}[1] = 0$	No	Xét tiếp with ký tự mẫu $X[0]='G'$
$(X[0] \neq Y[3]): \text{No}$	$i > -1 \Rightarrow i = \text{mpNext}[0] = -1$	No	Vòng lặp dừng $\Rightarrow$ Tăng $i++, j++$
Dịch sang $j=4, i=0$	Ký tự văn bản tiếp theo $Y[4]='C'$		
$(X[0] \neq Y[4]): \text{No}$	$i > -1 \Rightarrow i = \text{mpNext}[0] = -1$	No	Lệch ngay từ đầu $\Rightarrow$ Tăng $i++, j++$
Dịch sang $j=5, i=0$	Ký tự văn bản tiếp theo $Y[5]='G'$		
Khớp liên tục đến hết	$(X[0..7]==Y[5..12]): \text{Yes}$	Yes	Tìm thấy mẫu ở vị trí $j-i=13-8=5$
Kết thúc vòng lặp			Do con trỏ văn bản $j=13 \geq 13$

Kết luận: Xâu mẫu xuất hiện tại vị trí $j = \{5\}$.

2.5.4. Cài đặt thuật toán

Cài đặt thuật toán Morris-Pratt

```
// Bước 1: Pha tiền xử lý (Preprocessing)
void PreMp(const std::string& x, int m, std::vector<int>& mpNext) {
    int i = 0;
    int j = mpNext[0] = -1;

    while (i < m) {
        while (j > -1 && x[i] != x[j]) {
            j = mpNext[j];
        }
        i++;
        j++;
        mpNext[i] = j;
    }
}

// Bước 2: Pha tìm kiếm (Searching)
void MorrisPratt(const std::string& x, int m, const std::string& y, int n) {
    // Khởi tạo mảng mpNext có kích thước m + 1
    std::vector<int> mpNext(m + 1);

    /* Gọi Bước 1: Tiền xử lý dữ liệu xâu mẫu */
    PreMp(x, m, mpNext);

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự */
    int i = 0; // Con trỏ trên xâu mẫu x
    int j = 0; // Con trỏ trên xâu văn bản y

    while (j < n) {
        // Hoạt động lùi vị trí con trỏ i trên xâu mẫu khi xảy ra lệch khớp
        ký tự
        while (i > -1 && x[i] != y[j]) {
            i = mpNext[i];
        }
        i++;
        j++;

        // Nếu con trỏ i duyệt hết toàn bộ độ dài mẫu m
        if (i >= m) {
            std::cout << "Tìm thay mau tai vi tri: " << (j - i) << std::endl;
            i = mpNext[i]; // Tiếp tục dịch chuyển mẫu để tìm các vị trí
            trùng tiếp theo
        }
    }
}
```

2.6. Thuật toán Automat hữu hạn

2.6.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Automat hữu hạn là "xây dựng một bộ máy trạng thái (Deterministic Finite Automaton - DFA) để xử lý văn bản một cách tối ưu, đạt tốc độ kiểm tra $O(1)$ cho mỗi ký tự". Trong thuật toán này, mỗi trạng thái của Automat đại diện

cho độ dài của tiền tố thuộc mẫu X đã khớp được với văn bản cho đến thời điểm hiện tại.

Pha tiền xử lý dựa vào cấu trúc mẫu X và bảng chữ cái để xây dựng một ma trận chuyển trạng thái nhị phân không gian hai chiều $\delta[trạng_thái][ký_tự]$. Khi duyệt qua văn bản Y, thuật toán không thực hiện các phép so sánh ký tự trực tiếp mà chỉ đơn thuần dùng ký tự hiện tại làm chỉ số cấu trúc để "nhảy" sang trạng thái kế tiếp trong ma trận. Nếu trạng thái chuyển đổi chạm đến mốc giá trị bằng độ dài mẫu m, thuật toán ghi nhận ngay lập tức một vị trí trùng khớp hoàn toàn. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Automat hữu hạn được triển khai tuần tự qua 2 bước sau:

Thuật toán PreFiniteAutomata: // Thực hiện bước tiền xử lý xây dựng hàm chuyển trạng thái delta

Input :

- Mẫu X = (x₀, x₁, ..., x_{m-1}), độ dài m.
- Kích thước bảng chữ cái hệ thống 'ASIZE'.

Output: Ma trận chuyển trạng thái hai chiều delta[[]].

Formats: PreFiniteAutomata(X, m, ASIZE, delta);

Actions:

```
q = 0;
while (q <= m) {
    i = 0;
    while (i < ASIZE) {
        c = <Ký tự thứ i trong bảng chữ cái>;

        // Tìm tiền tố dài nhất của X đồng thời là hậu tố của đoạn (X[0..q-1] + c)
        k = min(m, q + 1);
        while (k > 0 && <Đoạn X[0..k-1] không phải là hậu tố của X[0..q-1] + c>) {
            k--;
        }
        delta[q][c] = k;
        i++;
    }
    q++;
}
EndActions.
```

Thuật toán FiniteAutomata:

Input :

- Mẫu X = (x₀, x₁, ..., x_{m-1}), độ dài m.
- Văn bản Y = (y₀, y₁, ..., y_{n-1}), độ dài n.

Output:

- Tất cả vị trí xuất hiện X trong Y.

Formats: FiniteAutomata(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreFiniteAutomata(X, m, 256, delta); // Xây dựng ma trận delta với độ phức tạp $O(m^3 * ASIZE)$

Bước 2 (Lặp tìm kiếm):

state = 0; // Trạng thái khởi đầu (chưa khớp ký tự nào)

```

j = 0;
while (j < n) {
    state = delta[state][Y[j]]; // Chuyển trạng thái tức thời với O(1)
    if (state == m) {
        <Tìm thấy mẫu ở vị trí j - m + 1>;
    }
    j++;
}
EndActions.

```

2.6.2. Đặc điểm thuật toán

- Thực hiện việc tìm kiếm chuỗi tuần tự theo chiều từ trái sang phải trên văn bản Y.
- Pha tiền xử lý tốn thời gian xây dựng ma trận khá lớn với độ phức tạp thời gian trường hợp cơ bản là $O(m^3 \cdot \sigma)$ (có thể tối ưu xuống $O(m \cdot \sigma)$ bằng cấu trúc bảng KMP) và không gian bộ nhớ lưu trữ ma trận là $O(m \cdot \sigma)$, với σ là kích thước bảng chữ cái.
- Pha tìm kiếm đạt hiệu năng tối ưu tuyệt đối với độ phức tạp thời gian tuyến tính nghiêm ngặt là $O(n)$, hoàn toàn độc lập với cấu trúc độ dài của xâu mẫu m và kích thước bảng chữ cái.

2.6.3. Kiểm nghiệm thuật toán

Kiểm nghiệm FiniteAutomata (X, m, Y, n):

- $X = \{GAG\}$, $m = 3$.
- $Y = \{GCAGAG\}$, $n = 6$.

Bước 1 (Tiền xử lý): Sau khi thực hiện hàm dựng PreFiniteAutomata, ta thu được ma trận hàm chuyển trạng thái $\delta[state][Y[j]]$ (chỉ biểu diễn các ký tự liên quan trong ví dụ):

Trạng thái hiện tại (state)	Nhận ký tự 'G'	Nhận ký tự 'A'	Nhận ký tự 'C'	Các ký tự khác
Trạng thái 0 (Chưa khớp)	1	0	0	0
Trạng thái 1 (Đã khớp "G")	1	2	0	0
Trạng thái 2 (Đã khớp "GA")	3	0	0	0
Trạng thái 3 (Khớp hoàn toàn "GAG")	1	2	0	0

Bước 2 (Lặp tìm kiếm):

(state == m)?	Ký tự Y[j] hiện tại	Chuyển đổi trạng thái: state = $\delta[state][Y[j]]$	j=? state=?
Bắt đầu (j=0,			

(state == m)?	Ký tự Y[j] hiện tại	Chuyển đổi trạng thái: state = delta[state][Y[j]]	j=? state=?
state=0)			
(0 == 3): No	Duyệt Y[0] = 'G'	state = delta[0]['G'] = 1	j=0, state=1
(1 == 3): No	Duyệt Y[1] = 'C'	state = delta[1]['C'] = 0	j=1, state=0
(0 == 3): No	Duyệt Y[2] = 'A'	state = delta[0]['A'] = 0	j=2, state=0
(0 == 3): No	Duyệt Y[3] = 'G'	state = delta[0]['G'] = 1	j=3, state=1
(1 == 3): No	Duyệt Y[4] = 'A'	state = delta[1]['A'] = 2	j=4, state=2
(2 == 3): No	Duyệt Y[5] = 'G'	state = delta[2]['G'] = 3	j=5, state=3
(3 == 3): Yes		(Trạng thái đạt mốc độ dài mẫu m = 3)	Tìm thấy mẫu ở vị trí j-m+1 = 3
Kết thúc vòng lặp			Do j=6 ≥ n

Kết luận: Xâu mẫu xuất hiện tại vị trí $j = \{3\}$.

2.6.4. Cài đặt thuật toán

Cài đặt thuật toán Automata hữu hạn

```
// Bước 1: Pha tiền xử lý (Preprocessing)
void PreFiniteAutomata(const std::string& x, int m,
std::vector<std::vector<int>>& delta) {
    int q, c;

    for (q = 0; q <= m; ++q) {
        for (c = 0; c < ASIZE; ++c) {
            // Khởi tạo trạng thái dịch chuyển tiếp theo lớn nhất có thể đạt
            int k = std::min(m, q + 1);

            // Vòng lặp kiểm tra tính chất hậu tố trùng khít tiền tố mẫu
            while (k > 0) {
                if (x[k - 1] == (char)c) {
                    int i = 0;
                    while (i < k - 1) {
                        if (x[i] != x[q - (k - 1) + i]) break;
                        i++;
                    }
                    if (i == k - 1) break; // Tìm thấy hậu tố hợp lệ lớn nhất
                }
                k--;
            }
            delta[q][c] = k;
        }
    }
}
```

```

}

// Bước 2: Pha tìm kiếm (Searching)
void FiniteAutomata(const std::string& x, int m, const std::string& y, int n)
{
    // Khởi tạo ma trận delta với kích thước (m + 1) x ASIZE
    std::vector<std::vector<int>> delta(m + 1, std::vector<int>(ASIZE, 0));

    /* Gọi Bước 1: Tiền xử lý dữ liệu chuỗi mẫu */
    PreFiniteAutomata(x, m, delta);

    /* Thực hiện Bước 2: Vòng lặp tìm kiếm trên văn bản Y */
    int state = 0; // Khởi tạo trạng thái ban đầu bằng 0
    for (int j = 0; j < n; ++j) {
        state = delta[state][((unsigned char)y[j])];

        // Nếu trạng thái bằng độ dài mẫu m, ghi nhận kết quả tìm kiếm thành
        // công
        if (state == m) {
            std::cout << "Tim thay mau tai vi tri: " << (j - m + 1) <<
std::endl;
        }
    }
}

```

2.7. Thuật toán Simon

2.7.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Simon là "tối ưu hóa Automat hữu hạn bằng cách chỉ lưu trữ và xử lý các trạng thái chuyển đổi thực sự có khả năng xảy ra lỗi (mismatch)". Trong Automat hữu hạn thông thường, ta phải duy trì một ma trận khổng lồ kích thước $O(m \times \sigma)$ Thuật toán Simon cải tiến điều này bằng cách định nghĩa một cấu trúc danh sách liên kết danh sách các phần tử gọi là các "cạnh" (edges) dịch chuyển lỗi hoặc sử dụng mảng phụ thông minh.

Cụ thể, pha tiền xử lý xây dựng một mảng bảng nhảy gọi là *simonNext[]* phối hợp cùng một danh sách liên kết danh sách các cạnh lỗi L. Khi một ký tự tại văn bản $Y[j]$ bị lệch khớp với mẫu tại vị trí i , thuật toán không cần tra bảng hai chiều mà chỉ cần kiểm tra xem cặp trạng thái $(i, Y[j])$ có nằm trong tập các cạnh chuyển đặc biệt L hay không để nhảy trực tiếp, nếu không có thì lùi về giá trị an toàn được tính toán sẵn trong $simonNext[i]$. Nhờ vậy, không gian lưu trữ được tiết kiệm tối đa mà vẫn giữ nguyên tốc độ chuyển đổi trạng thái tức thời. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Simon được triển khai tuần tự qua 2 bước sau:

Thuật toán PreSimon: // Thực hiện tiền xử lý tính toán bảng dịch chuyển Simon

Input : - Chuỗi mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

Output: Mảng lùi trạng thái $simonNext[]$ và danh sách các cạnh chuyển đổi đặc biệt L.

Formats: PreSimon($X, m, simonNext, L$);

Actions:

// Khởi tạo trạng thái ban đầu cho các điều kiện biên

$simonNext[0] = -1$;

$simonNext[1] = 0$;

// Tính toán các hàm biên dựa trên cấu trúc tuần hoàn của mẫu X

$i = 2$;

```

while (i <= m) {
    <Xác định tiền tố lặp trùng với hậu tố của đoạn X[0..i-1]>;
    simonNext[i] = <Độ dài biên trùng khớp tương ứng>;
    i++;
}
<Xây dựng tập các cạnh chuyển trạng thái lỗi L đối với các ký tự đặc biệt>;
EndActions.

```

Thuật toán Simon:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: Simon(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreSimon($X, m, \text{simonNext}, L$); // Tiền xử lý với độ phức tạp tối ưu $O(m)$

Bước 2 (Lặp tìm kiếm):

$i = 0$; // Chỉ số trên xâu mẫu X

$j = 0$; // Chỉ số trên văn bản Y

```

while (j < n) {
    // Kiểm tra xem có tồn tại cạnh chuyển trạng thái đặc biệt cho cặp (i, Y[j]) trong L
    // hay không
    if (<Tồn tại cạnh chuyển (i, Y[j] -> k) trong tập L>) {
        i = k;
        j++;
    }
    else if (X[i] == Y[j]) {
        i++;
        j++;
    }
    else {
        i = simonNext[i];
        if (i == -1) {
            i = 0;
            j++;
        }
    }
}
// Nếu con trỏ i duyệt qua hết độ dài mẫu m
if (i == m) {
    <Tìm thấy mẫu ở vị trí j - m>;
    i = simonNext[i];
}
}
EndActions.

```


2.7.2. Đặc điểm thuật toán

- Thực hiện cơ chế tìm kiếm, đối sánh ký tự tuần tự từ trái sang phải trên văn bản Y.
- Pha tiền xử lý vô cùng gọn nhẹ, chỉ tốn độ phức tạp thời gian là $O(m)$ và độ phức tạp không gian lưu trữ bộ nhớ được tối giản nghiêm ngặt về mức $O(m)$, loại bỏ hoàn toàn sự phụ thuộc vào kích thước bảng chữ cái σ so với Automat hữu hạn nguyên bản.
- Pha tìm kiếm thực thi với hiệu năng rất cao, độ phức tạp thời gian trong trường hợp xấu nhất được khống chế tuyến tính là $O(n)$
- Số lượng phép so sánh ký tự tối đa trên văn bản được giới hạn chặt chẽ ở mức là $2n - 1$ phép toán.

2.7.3. Kiểm nghiệm thuật toán

Kiểm nghiệm Simon (X, m, Y, n):

- X chuỗi mẫu: "abab", độ dài m = 4.
- Y văn bản: "ababcabab", độ dài n = 9.

Bước 1 (Tiền xử lý): Thực hiện hàm PreSimon, ta thu được mảng lùi simonNext và danh sách cạnh chuyển lỗi đặc biệt L:

- Mảng simonNext[] = {-1, 0, 0, 1, 2}
- Tập các cạnh lỗi L: Ở trạng thái thứ 2 nếu gặp ký tự 'a' sẽ nhảy thẳng về trạng thái 1 thay vì lùi tuần tự.

Bước 2 (Lập tìm kiếm):

Lượt quét (Attempt)	Vị trí văn bản (j)	Con trỏ mẫu (i)	Ký tự đối sánh (X[i]==Y[j])?	Kết quả xử lý / Hành động dịch chuyển trạng thái
Bắt đầu (j=0)				
First attempt	j=0→3	i=0→3	Khớp liên tục 4 ký tự: "abab" == "abab"	Con trỏ mẫu chạm mốc I = 4 == m. ⇒ KẾT LUẬN: Tìm thấy mẫu tại vị trí j - m = 4 - 4 = 0. Dịch chuyển lùi trạng thái: I = simonNext[4] = 2. Vị trí văn bản tiếp theo kế tiếp: j = 4.
Second attempt	j=4	i=2	X[2] ≠ Y[4] ('a' ≠ 'c')	Lệch khớp ký tự. Không có cạnh đặc biệt. Trạng thái lùi: i=simonNext[2]=0. Giữ nguyên vị trí văn bản j=4.
Third attempt	j=4	i=0	X[0] ≠ Y[4] ('a' ≠ 'c')	Lệch khớp ký tự đầu tiên (i=0). ⇒ i=simonNext[0]=-1. Kích hoạt điều kiện biên: đặt lại i=0 và tăng j++ ⇒ j=5.
Fourth attempt	j=5→8	i=0→3	Khớp liên tục đến hết chuỗi: "abab"=="abab"	Con trỏ mẫu chạm mốc i=4==m. ⇒ KẾT LUẬN: Tìm thấy mẫu tại vị trí j - m = 9 - 4 = 5. Dịch chuyển lùi trạng thái: i = simonNext[4] = 2. Cửa sổ mới dịch chuyển đến: j = 9.

Kết thúc	j=9		Vòng lặp dừng hoàn toàn	Do con trỏ văn bản hiện tại đã duyệt hết $j=9 \geq n$.
----------	-----	--	-------------------------	---

Kết luận: Xâu mẫu xuất hiện tại hai vị trí chỉ số $j = \{0, 5\}$.

2.7.4. Cài đặt thuật toán

Cài đặt thuật toán Simon

```
// Cấu trúc biểu diễn một cạnh chuyển trạng thái lỗi đặc biệt trong tập L
struct SimonEdge {
    int from_state;
    char character;
    int to_state;
};

// Bước 1: Pha tiền xử lý (Preprocessing)
void PreSimon(const std::string& x, int m, std::vector<int>& simonNext,
std::vector<SimonEdge>& L) {
    // Khởi tạo giá trị cơ sở ban đầu
    simonNext[0] = -1;
    if (m > 0) simonNext[1] = 0;

    int i = 2;
    int j = 0;

    // Vòng lặp xây dựng bảng nhảy tương đồng với cơ chế KMP cải tiến
    while (i <= m) {
        while (j > -1 && x[i - 1] != x[j]) {
            j = simonNext[j];
        }
        j++;
        simonNext[i] = j;
        i++;
    }

    // Xây dựng tập các cạnh chuyển đặc biệt L khi phát hiện các liên kết
    tuần hoàn trùng lặp
    for (int state = 1; state < m; ++state) {
        if (state >= 2 && x[state] == x[simonNext[state]]) {
            SimonEdge edge;
            edge.from_state = state;
            edge.character = x[simonNext[state]];
            edge.to_state = simonNext[simonNext[state]] + 1;
            L.push_back(edge);
        }
    }
}

// Bước 2: Pha tìm kiếm (Searching)
void Simon(const std::string& x, int m, const std::string& y, int n) {
    std::vector<int> simonNext(m + 1, 0);
    std::vector<SimonEdge> L;

    /* Gọi Bước 1: Tiền xử lý dữ liệu xâu mẫu */
    PreSimon(x, m, simonNext, L);

    int i = 0; // Con trỏ chỉ số mẫu X
    int j = 0; // Con trỏ chỉ số văn bản Y

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự trên văn bản Y */
```

```

while (j < n) {
    bool edge_found = false;

    // Kiểm tra xem trạng thái hiện tại và ký tự văn bản có nằm trong tập
    cạnh đặc biệt L hay không
    for (const auto& edge : L) {
        if (edge.from_state == i && edge.character == y[j]) {
            i = edge.to_state;
            j++;
            edge_found = true;
            break;
        }
    }

    // Nếu không thuộc tập cạnh đặc biệt, xử lý theo logic so khớp thông
    thường
    if (!edge_found) {
        if (x[i] == y[j]) {
            i++;
            j++;
        } else {
            i = simonNext[i];
            if (i == -1) {
                i = 0;
                j++;
            }
        }
    }

    // Nếu con trỏ i khớp hoàn toàn xâu mẫu đạt độ dài m
    if (i == m) {
        std::cout << "Tìm thay mau tai vi tri: " << (j - m) << std::endl;
        i = simonNext[i]; // Lùi trạng thái để sẵn sàng tìm kiếm các vị
        trí tiếp theo
    }
}
}

```

CHƯƠNG 3. THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

3.1. Thuật toán Horspool

3.1.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Horspool là "đơn giản hóa thuật toán Boyer-Moore bằng cách chỉ sử dụng duy nhất bảng dịch chuyển ký tự xấu (bad-character shift) tính toán dựa trên ký tự cuối cùng của cửa sổ hiện tại". Thuật toán thực hiện đối sánh các ký tự trong cửa sổ theo chiều từ phải sang trái (bắt đầu từ ký tự cuối cùng của mẫu $X[m-1]$ ngược dần về đầu).

Pha tiền xử lý xây dựng một mảng bảng nhảy một chiều mang tên $bmBc[]$ dựa trên kích thước bảng chữ cái. Điểm thông minh của Horspool là: cho dù quá trình so khớp trong cửa sổ bị lỗi ở bất kỳ vị trí nào, khoảng cách dịch chuyển xâu mẫu cho lượt quét tiếp theo sẽ luôn luôn được quyết định một cách nhất quán dựa trên giá trị tra cứu của ký tự văn bản nằm trùng khớp với vị trí cuối cùng của cửa sổ ($c = Y[j + m - 1]$). Nhờ vậy, thuật toán loại bỏ được bảng nhảy "khớp tốt" (good-suffix) phức tạp của Boyer-Moore mà vẫn giữ được khả năng nhảy cóc cực xa trên văn bản trong thực tế. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Horspool được triển khai tuần tự qua 2 bước sau:

Thuật toán PreBmBc: // Thực hiện bước tiền xử lý tính bảng dịch chuyển ký tự xấu

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống `ASIZE`.

Output: Mảng giá trị dịch chuyển $bmBc[]$.

Formats: `PreBmBc(X, m, bmBc);`

Actions:

$i = 0;$

while ($i < ASIZE$) {

$bmBc[i] = m;$ // Mặc định nếu ký tự không nằm trong mẫu, bước nhảy bằng m
 $i++;$

}

$i = 0;$

while ($i < m - 1$) {

$bmBc[X[i]] = m - 1 - i;$ // Khoảng cách từ vị trí ký tự đến đuôi mẫu
 $i++;$

}

EndActions.

Thuật toán Horspool:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: `Horspool(X, m, Y, n);`

Actions:

Bước 1 (Tiền xử lý):

PreBmBc(X, m, bmBc); // Tiền xử lý tính mảng bmBc với độ phức tạp $O(m + \text{ASIZE})$

Bước 2 (Lập tìm kiếm):

$j = 0$; // Chỉ số mốc vị trí bắt đầu của cửa sổ trên văn bản Y

while ($j \leq n - m$) {

$c = Y[j + m - 1]$; // Lấy ký tự văn bản tại vị trí cuối cùng của cửa sổ hiện tại

 // So sánh ký tự cuối cùng trước, khớp thì dùng memcmp kiểm tra đoạn còn lại

 if ($(X[m - 1] == c \ \&\& \text{Kiểm tra đoạn } X[0..m-2] \text{ khớp hoàn toàn với } Y[j..j+m-2]) >$) {
 <Tìm thấy mẫu ở vị trí j >;

 }

$j = j + \text{bmBc}[c]$; // Dịch chuyển cửa sổ dựa trên ký tự cuối cùng c

}

EndActions.

3.1.2. Đặc điểm thuật toán

- Thực hiện việc đối sánh ký tự bên trong cửa sổ theo chiều từ phải sang trái, nhưng cửa sổ tịnh tiến từ trái sang phải trên văn bản.
- Dễ dàng cài đặt và tối ưu hóa cấu trúc vòng lặp trong thực tế.
- Pha tiền xử lý có độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ là $O(\sigma)$, với σ là kích thước bảng chữ cái (ví dụ $\sigma = 256$ với mã ASCII).
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$, tuy nhiên số phép so sánh trung bình trên một ký tự văn bản trong thực tế cực kỳ thấp, dao động tối ưu trong khoảng từ $1/\sigma$ đến $2/(\sigma + 1)$. Thuật toán đạt hiệu năng đặc biệt cao khi chạy trên các bảng chữ cái kích thước lớn (như bảng mã ASCII thông thường).

3.1.3. Kiểm nghiệm thuật toán

Kiểm nghiệm Horspool (X, m, Y, n):

- X xâu mẫu: "GCAGAGAG", độ dài $m = 8$.

- Y văn bản: "GCATCGCAGAGAGTATACAGTACG", độ dài $n = 24$.

Bước 1 (Tiền xử lý): Thực hiện hàm PreBmBc, ta thu được mảng dịch chuyển ký tự xấu của các ký tự liên quan (tính khoảng cách từ vị trí xuất hiện muộn nhất của ký tự đến cuối xâu mẫu, lưu ý không xét ký tự cuối cùng $X[7]$):

- $\text{bmBc}['A'] = 1$ (chữ 'A' xuất hiện muộn nhất ở chỉ số 6 $\rightarrow 8 - 1 - 6 = 1$)

- $\text{bmBc}['G'] = 2$ (chữ 'G' xuất hiện muộn nhất trước đuôi ở chỉ số 5 $\rightarrow 8 - 1 - 5 = 2$)

- $\text{bmBc}['C'] = 6$ (chữ 'C' xuất hiện tại chỉ số 1 $\rightarrow 8 - 1 - 1 = 6$)

- $\text{bmBc}['T'] = 8$ (ký tự 'T' không nằm trong mẫu $\rightarrow$ giá trị mặc định bằng $m = 8$)

Bước 2 (Lập tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Ký tự cuối cửa sổ c = $Y[j+7]$	Tiến trình so khớp ký tự (Từ phải sang trái)	Hành động dịch chuyển tiếp theo
First	$j = 0$	$Y[7] = 'A'$	Phép so sánh 1:	$\text{Shift} = \text{bmBc}['A'] = 1.$

attempt			$X[7]='G' \neq Y[7]='A'$ → Lệch ngay lập tức.	Cửa sổ mới: $j = 0 + 1 = 1$.
Second attempt	$j = 1$	$Y[8] = 'G'$	Phép so sánh 1: $X[7]='G'$ (Khớp). Phép so sánh 2: $X[6]='A' \neq Y[7]='A' \rightarrow$ Lệch.	Shift = $\text{bmBc}['G'] = 2$ Cửa sổ mới: $j = 1 + 2 = 3$.
Third attempt	$j = 3$	$Y[10] = 'G'$	Phép so sánh 1: $X[7]='G'$ (Khớp). Phép so sánh 2: $X[6]='A' \neq Y[9]='G' \rightarrow$ Lệch.	Shift = $\text{bmBc}['G'] = 2$ Cửa sổ mới: $j = 3 + 2 = 5$.
Fourth attempt	$j = 5$	$Y[12] = 'G'$	Khớp hoàn toàn chuỗi ký tự thông qua 8 phép đối sánh: $X[0..7] == Y[5..12] \rightarrow \{ "GCAGAGAG" \}$	→ KẾT LUẬN: Tìm thấy mẫu tại $j = 5$. Shift = $\text{bmBc}['G'] = 2$ Cửa sổ mới: $j = 5 + 2 = 7$.
Fifth attempt	$j = 7$	$Y[14] = 'A'$	Phép so sánh 1: $X[7]='G' \neq Y[14]='A' \rightarrow$ Lệch.	Shift = $\text{bmBc}['A'] = 1$ Cửa sổ mới: $j = 7 + 1 = 8$.
Sixth attempt	$j = 8$	$Y[15] = 'T'$	Phép so sánh 1: $X[7]='G' \neq Y[15]='T' \rightarrow$ Lệch.	Shift = $\text{bmBc}['T'] = 8$. Cửa sổ mới: $j = 8 + 8 = 16$.
Seventh attempt	$j = 16$	$Y[23] = 'G'$	Phép so sánh 1: $X[7]='G'$ (Khớp). Phép so sánh 2: $X[6]='A' \neq Y[22]='C' \rightarrow$ Lệch.	Shift = $\text{bmBc}['G'] = 2$ Vòng lặp dừng vì lượt kế tiếp $j = 18 > (24 - 8)$.

Kết luận: Qua tiến trình kiểm nghiệm, thuật toán Horspool phát hiện xâu mẫu xuất hiện hoàn chỉnh tại vị trí chỉ số $j = \{5\}$.

3.1.4. Cài đặt thuật toán

```
// Định nghĩa kích thước bảng chữ cái ASCII tiêu chuẩn
#define ASIZE 256

//Bước 1: Pha tiền xử lý (Preprocessing)
void PreBmBc(const std::string& x, int m, std::vector<int>& bmBc) {
    int i;

    // Khởi tạo tất cả các phần tử mặc định bằng độ dài m của xâu mẫu
```

```

    for (i = 0; i < ASIZE; ++i) {
        bmBc[i] = m;
    }

    // Tính toán khoảng cách dịch chuyển cho các ký tự có mặt trong mẫu (loại
    trừ ký tự cuối cùng)
    for (i = 0; i < m - 1; ++i) {
        bmBc[(unsigned char)x[i]] = m - 1 - i;
    }
}

//Bước 2: Pha tìm kiếm (Searching)
void Horspool(const std::string& x, int m, const std::string& y, int n) {
    // Cấp phát mảng bảng nhảy bmBc với kích thước ASIZE
    std::vector<int> bmBc(ASIZE);

    /* Gọi Bước 1: Tiền xử lý tính toán mảng dịch chuyển chuỗi */
    PreBmBc(x, m, bmBc);

    int j = 0;
    char c;

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự trên văn bản Y */
    while (j <= n - m) {
        // Lấy ký tự văn bản tại vị trí cuối cùng tương ứng với cửa sổ hiện
        tại
        c = y[j + m - 1];

        // Kiểm tra điều kiện: Ký tự cuối cùng khớp VÀ đoạn thân phía trước
        khớp hoàn toàn (compare tương đương memcmp)
        if (x[m - 1] == c && y.compare(j, m - 1, x, 0, m - 1) == 0) {
            std::cout << "Tìm thấy mẫu tại vị trí: " << j << std::endl;
        }

        // Tiến hành dịch chuyển cửa sổ tịnh tiến sang phải dựa trên giá trị
        tra cứu của ký tự c
        j += bmBc[(unsigned char)c];
    }
}

```

3.2. Thuật toán Quick search

3.2.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Quick Search là "tối ưu hóa bước nhảy của Boyer-Moore bằng cách quyết định khoảng cách dịch chuyển dựa vào ký tự nằm ngay sau cửa sổ hiện tại". Trong các thuật toán khác, khi xảy ra lỗi so khớp bên trong cửa sổ, ta thường xét các ký tự thuộc phạm vi hiện thời của cửa sổ.

Quick Search nhận ra một đặc điểm thông minh: Khi cửa sổ đang nằm ở đoạn $Y[j \dots j+m-1]$, sau lượt quét này, dù có khớp hay không thì xâu mẫu chắc chắn phải dịch lên ít nhất 1 đơn vị. Điều đó có nghĩa là ký tự tại vị trí $Y[j+m]$ (ký tự ngay sát sau cửa sổ) **bắt buộc** sẽ tham gia vào lượt quét kế tiếp. Do đó, pha tiền xử lý xây dựng mảng bảng nhảy một chiều $qsBc[]$. Khi kết thúc một lượt quét, thuật toán không cần quan tâm việc so khớp bị lỗi ở đâu hay thứ tự nào, nó chỉ nhìn vào ký tự ngay sau cửa sổ ($y[j + m]$) để tra cứu bước nhảy. Cơ chế này giúp Quick Search có cấu trúc code gọn nhẹ và đạt bước nhảy rất xa. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Quick Search được triển khai tuần tự qua 2 bước sau:

Thuật toán PreQsBc: // Thực hiện bước tiền xử lý tính bảng dịch chuyển Quick Search

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống 'ASIZE'.

Output: Bảng giá trị dịch chuyển $qsBc[]$.

Formats: PreQsBc($X, m, qsBc$);

Actions:

$i = 0$;

while ($i < ASIZE$) {

$qsBc[i] = m + 1$; // Mặc định nếu ký tự không nằm trong mẫu, bước nhảy bằng $m+1$
 $i++$;

}

$i = 0$;

while ($i < m$) {

$qsBc[X[i]] = m - i$; // Khoảng cách từ ký tự đến vị trí ngay sau đuôi mẫu
 $i++$;

}

EndActions.

Thuật toán QuickSearch:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất X trong Y .

Formats: QuickSearch(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreQsBc($X, m, qsBc$); // Tiền xử lý tính mảng $qsBc$ với độ phức tạp $O(m + ASIZE)$

Bước 2 (Lặp tìm kiếm):

$j = 0$; // Vị trí mốc bắt đầu của cửa sổ trên văn bản Y

while ($j \leq n - m$) {

 // Tiến hành đối sánh toàn bộ cửa sổ (dùng memcmp)

 if ($\langle$ Kiểm tra đoạn $X[0..m-1]$ khớp hoàn toàn với $Y[j..j+m-1]\rangle$) {
 $\langle$ Tìm thấy mẫu ở vị trí j $\rangle$;

 }

 // Bước nhảy quyết định bởi ký tự nằm ngay sau cửa sổ hiện tại

$j = j + qsBc[Y[j + m]]$;

}

EndActions.

3.2.2. Đặc điểm thuật toán

- Quá trình đối sánh ký tự bên trong cửa sổ có thể thực hiện theo ****bất kỳ thứ tự nào**** (any order) tùy ý, tuy nhiên cửa sổ dịch chuyển tuần tiên từ trái sang phải trên văn bản Y .
- Pha tiền xử lý cực kỳ gọn nhẹ với độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ lưu trữ mảng nhảy là $O(\sigma)$, với σ là kích thước bảng chữ

cái.

- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m * \sigma)$, nhưng có hiệu năng thực tế (practical behaviour) rất tốt. Thuật toán chạy cực nhanh đối với các trường hợp xấu mẫu ngắn (short patterns) và bảng chữ cái có kích thước lớn (large alphabets).

3.2.3. Kiểm nghiệm thuật toán

Kiểm nghiệm QuickSearch(X, m, Y, n):

- **X mẫu:** "GCAGAGAG", độ dài m = 8.
- **Y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài n = 24.

Bước 1 (Tiền xử lý): Thực hiện hàm PreQsBc, ta thu được mảng dịch chuyển dựa theo khoảng cách đến chỉ số vị trí m = 8:

- $qsBc['G'] = 1$ (Chữ 'G' xuất hiện muộn nhất ở chỉ số cuối $7 \rightarrow 8 - 7 = 1$)
- $qsBc['A'] = 2$ (Chữ 'A' xuất hiện muộn nhất ở chỉ số 6 $\rightarrow 8 - 6 = 2$)
- $qsBc['C'] = 7$ (Chữ 'C' xuất hiện muộn nhất ở chỉ số 1 $\rightarrow 8 - 1 = 7$)
- $qsBc['T'] = 9$ (Ký tự 'T' không nằm trong mẫu $\rightarrow$ giá trị mặc định bằng $m + 1 = 8 + 1 = 9$)

Bước 2 (Lặp tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Đoạn văn bản trong cửa sổ $Y[j...j+7]$	Kết quả so khớp hoàn hảo	Ký tự ngay sau cửa sổ $Y[j + 8]$	Khoảng cách dịch chuyển kế tiếp
First attempt	j = 0	"GCATCGCA"	Không khớp (Lệch ở ký tự thứ 4: 'T')	$Y[8] = 'G'$	Shift = $qsBc['G'] = 1$ Cửa sổ mới: $j=0 + 1 = 1$.
Second attempt	j = 1	"CATCGCAG"	Không khớp	$Y[9] = 'G'$	Shift = $qsBc['G'] = 1$ Cửa sổ mới: $j=1 + 1 = 2$.
Third attempt	j = 2	"ATCGCAGA"	Không khớp	$Y[10] = 'G'$	Shift = $qsBc['G'] = 1$ Cửa sổ mới: $j = 2 + 1 = 3$.
Fourth attempt	j = 3	"TCGCAGAG"	Không khớp	$Y[11] = 'A'$	Shift = $qsBc['A'] = 2$ Cửa sổ mới: $j = 3 + 2 = 5$.
Fifth attempt	j = 5	"GCAGAGAG"	Khớp hoàn toàn! $\rightarrow$ Tìm thấy mẫu tại j = 5	$Y[13] = 'T'$	Shift = $qsBc['T'] = 9$ Cửa sổ mới: $j = 5 + 9 = 14$.
Sixth attempt	j = 14	"ATACAGTA"	Không khớp	$Y[22] = 'C'$	Shift = $qsBc['C'] = 7$ Vòng lặp dừng vì lượt kế tiếp $j = 21 > (24 - 8)$.

Kết luận: Mẫu xuất hiện hoàn chỉnh tại vị trí chỉ số $j = \{5\}$ trong văn bản Y.

3.2.4. Cài đặt thuật toán

Cài đặt thuật toán Simon

```
// Định nghĩa kích thước bảng chữ cái ASCII tiêu chuẩn
#define ASIZE 256
//Bước 1: Pha tiền xử lý (Preprocessing)
void PreQsBc(const std::string& x, int m, std::vector<int>& qsBc) {
    int i;

    // Khởi tạo tất cả các phần tử mặc định bằng độ dài m + 1
    for (i = 0; i < ASIZE; ++i) {
        qsBc[i] = m + 1;
    }

    // Tính toán khoảng cách dịch chuyển từ vị trí ký tự đến điểm m sát sau
    // mẫu
    for (i = 0; i < m; ++i) {
        qsBc[(unsigned char)x[i]] = m - i;
    }
}

//Bước 2: Pha tìm kiếm (Searching)
void QuickSearch(const std::string& x, int m, const std::string& y, int n) {
    // Cấp phát mảng bảng nhảy qsBc với kích thước ASIZE
    std::vector<int> qsBc(ASIZE);

    /* Gọi Bước 1: Tiền xử lý tính toán mảng dịch chuyển chuỗi */
    PreQsBc(x, m, qsBc);

    int j = 0;

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự trên văn bản Y */
    while (j <= n - m) {
        // So khớp toàn bộ xâu mẫu độ dài m tại vị trí j (sử dụng y.compare
        // tương đương memcmp)
        if (y.compare(j, m, x) == 0) {
            std::cout << "Tìm thay mau tai vi tri: " << j << std::endl;
        }

        // Điều kiện biên để tránh truy cập vượt quá chỉ số mảng của Y khi j
        // đạt mốc cuối
        if (j + m < n) {
            // Điểm cốt lõi của Quick Search: Nhảy dịch dựa vào ký tự y[j +
            // m] ngay sau cửa sổ
            j += qsBc[(unsigned char)y[j + m]];
        } else {
            break;
        }
    }
}
```

3.3. Thuật toán Smith

3.3.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Smith là "kết hợp sức mạnh của hai thuật toán Horspool và Quick Search bằng cách lấy giá trị dịch chuyển lớn nhất (MAX) giữa hai

hàm dịch chuyển ký tự xấu". Tác giả Smith nhận thấy rằng việc tính toán bước nhảy dựa trên ký tự văn bản nằm ngay sau cửa sổ ($Y[j + m]$) đôi khi cho kết quả ngắn hơn so với việc tính toán dựa trên ký tự văn bản cuối cùng của cửa sổ ($Y[j + m - 1]$), và ngược lại. Để tối ưu hóa, pha tiền xử lý của thuật toán Smith sẽ đồng thời xây dựng cả hai mảng bảng nhảy một chiều: $bmBc[]$ (của thuật toán Horspool) và $qsBc[]$ (của thuật toán Quick Search). Khi tiến hành tìm kiếm tại mỗi cửa sổ, thuật toán thực hiện so khớp chuỗi ký tự.

Nếu xảy ra lệch khớp hoặc sau khi tìm thấy mẫu, khoảng cách tịnh tiến cửa sổ mới sẽ lấy giá trị cực đại giữa bước nhảy ký tự xấu của Horspool tại vị trí cuối cửa sổ và bước nhảy của Quick Search tại vị trí liền sau cửa sổ: $MAX(bmBc[Y[j + m - 1]], qsBc[Y[j + m]])$. Sự kết hợp này giúp thuật toán luôn đạt được bước nhảy xa nhất có thể trong mọi tình huống. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Smith được triển khai tuần tự qua 2 bước sau:

Thuật toán PreSmith: // Thực hiện tiền xử lý tính toán hai bảng dịch chuyển phụ trợ

Input : - Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

- Kích thước bảng chữ cái hệ thống 'ASIZE'.
-

Output: Mảng dịch chuyển $bmBc[]$ và mảng dịch chuyển $qsBc[]$.

Formats: PreSmith($X, m, bmBc, qsBc$);

Actions:

// 1. Gọi hàm tiền xử lý của thuật toán Horspool để tính mảng $bmBc$
PreBmBc($X, m, bmBc$);

// 2. Gọi hàm tiền xử lý của thuật toán Quick Search để tính mảng $qsBc$
PreQsBc($X, m, qsBc$);

EndActions.

Thuật toán Smith:

Input :

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả vị trí xuất hiện X trong Y .

Formats: Smith(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreSmith($X, m, bmBc, qsBc$); // Tiền xử lý song song với độ phức tạp $O(m + ASIZE)$

Bước 2 (Lặp tìm kiếm):

$j = 0$; // Vị trí mốc bắt đầu của cửa sổ trên văn bản Y

while ($j \leq n - m$) {

// Tiến hành đối sánh toàn bộ cửa sổ thông qua hàm so khớp bộ nhớ memcmp
if (<Kiểm tra đoạn $X[0..m-1]$ khớp hoàn toàn với $Y[j..j+m-1]$ >) {
 <Tìm thấy mẫu ở vị trí j >;

}

// Tính toán bước nhảy tối ưu bằng cách lấy giá trị cực đại (MAX) giữa hai bảng
 $j = j + MAX(bmBc[Y[j + m - 1]], qsBc[Y[j + m]]);$

}

EndActions.

3.3.2. Đặc điểm thuật toán

- Quá trình đối sánh ký tự bên trong cửa sổ sử dụng hàm kiểm tra vùng nhớ 'memcmp' (duyệt từ trái sang phải), nhưng cơ chế tính bước nhảy kết hợp cả hai phần tử cuối cửa sổ và liền sau cửa sổ.
- Pha tiền xử lý có độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ lưu trữ là $O(\sigma)$, với σ là kích thước bảng chữ cái.
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m * \sigma)$, tuy nhiên do luôn chọn được bước nhảy cực đại nên số lượng cửa sổ cần quét thực tế được giảm thiểu tối đa, giúp thuật toán vận hành cực kỳ hiệu quả trong thực tế.

3.3.3. Kiểm nghiệm thuật toán

Kiểm nghiệm Smith(X, m, Y, n):

- X chuỗi mẫu: "GCAGAGAG", độ dài m = 8.
- Y văn bản: "GCATCGCAGAGAGTATACAGTACG", độ dài n = 24.

Bước 1 (Tiền xử lý): Thực hiện kết hợp hai hàm PreBmBc và PreQsBc, ta thu được hai mảng dịch chuyển tương ứng của các ký tự:

- Bảng tra cứu Horspool: bmBc['A'] = 1, bmBc['G'] = 2, bmBc['C'] = 6, bmBc['T'] = 8.
- Bảng tra cứu Quick Search: qsBc['G'] = 1, qsBc['A'] = 2, qsBc['C'] = 7, qsBc['T'] = 9.

Bước 2 (Lặp tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Ký tự cuối cửa sổ y[j+7]	Ký tự sau cửa sổ y[j+8]	Công thức tính bước nhảy dịch chuyển cửa sổ kế tiếp	Khoảng cách dịch chuyển
First attempt	j = 0	'A'	'G'	shift = max(bmBc['A'], qsBc['G']) = max(1, 1)	shift = 1 -> j = 1
Second attempt	j = 1	'G'	'G'	shift = max(bmBc['G'], qsBc['G']) = max(2, 1)	shift = 2 -> j = 3
Third attempt	j = 3	'G'	'A'	shift = max(bmBc['G'], qsBc['A']) = max(2, 2)	shift = 2 -> j = 5
Fourth attempt	j = 5	'G'	'T'	Khớp hoàn toàn! Tìm thấy mẫu tại j = 5. shift = max(bmBc['G'], qsBc['T']) = max(2, 9)	shift = 9 -> j = 14
Fifth attempt	j = 14	'A'	'C'	Không khớp. shift = max(bmBc['A'], qsBc['C']) = max(1, 7)	shift = 7 -> j = 21

Kết thúc	j = 21			Vòng lặp dừng vì vị trí của số tiếp theo vượt quá điều kiện (n - m).	
----------	--------	--	--	--	--

Kết luận: Qua tiến trình kiểm nghiệm, thuật toán Smith phát hiện xâu mẫu xuất hiện hoàn chỉnh tại vị trí chỉ số **j = {5}**.

3.3.4. Cài đặt thuật toán

```
#define ASIZE 256

// Hàm tiền xử lý của Horspool
void PreBmBc(const std::string& x, int m, std::vector<int>& bmBc) {
    for (int i = 0; i < ASIZE; ++i) {
        bmBc[i] = m;
    }
    for (int i = 0; i < m - 1; ++i) {
        bmBc[(unsigned char)x[i]] = m - 1 - i;
    }
}

// Hàm tiền xử lý của Quick Search
void PreQsBc(const std::string& x, int m, std::vector<int>& qsBc) {
    for (int i = 0; i < ASIZE; ++i) {
        qsBc[i] = m + 1;
    }
    for (int i = 0; i < m; ++i) {
        qsBc[(unsigned char)x[i]] = m - i;
    }
}

// Cài đặt thuật toán Smith
void Smith(const std::string& x, int m, const std::string& y, int n) {
    std::vector<int> bmBc(ASIZE);
    std::vector<int> qsBc(ASIZE);

    /* Bước 1: Pha tiền xử lý kết hợp */
    PreBmBc(x, m, bmBc);
    PreQsBc(x, m, qsBc);

    int j = 0;

    /* Bước 2: Pha tìm kiếm sử dụng MAX bước nhảy */
    while (j <= n - m) {
        // So khớp toàn bộ của số bằng hàm tương đương memcmp
        if (y.compare(j, m, x) == 0) {
            std::cout << "Tìm thay mau tai vi tri: " << j << std::endl;
        }

        // Điều kiện biên an toàn khi tính toán vị trí j + m trên văn bản
        // Y
        if (j + m < n) {
            j += std::max(bmBc[(unsigned char)y[j + m - 1]],
                qsBc[(unsigned char)y[j + m]]);
        } else {
            break;
        }
    }
}
```

3.4. Thuật toán Raita

3.4.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Raita là "tối ưu hóa giai đoạn đối sánh ký tự trong cửa sổ của thuật toán Horspool bằng cách thay đổi thứ tự kiểm tra: ưu tiên so sánh ký tự cuối cùng, ký tự đầu tiên, rồi đến ký tự chính giữa của mẫu trước khi quét các ký tự còn lại". Tác giả Raita nhận thấy rằng trong thực tế, nếu ba vị trí chiến lược này (cuối, đầu, giữa) đã khớp thì tỷ lệ mẫu khớp hoàn toàn tăng lên rất cao. Ngược lại, nếu một trong ba ký tự này lệch, thuật toán có thể bác bỏ ngay cửa sổ đó mà không cần tốn công so sánh các ký tự khác.

Để thực hiện điều này, pha tiền xử lý của thuật toán Raita xây dựng mảng bảng nhảy một chiều $bmBc[]$ hoàn toàn giống với thuật toán Horspool. Khi tiến hành tìm kiếm tại mỗi cửa sổ j , thuật toán thực hiện kiểm tra tuần tự: ký tự cuối cửa sổ $y[j + m - 1]$, ký tự đầu cửa sổ $y[j]$, và ký tự giữa cửa sổ $y[j + (m / 2)]$. Nếu cả ba ký tự này đều trùng khớp với các ký tự tương ứng trên mẫu, thuật toán mới tiến hành một vòng lặp đối sánh các ký tự còn lại từ vị trí kế cuối $(m - 2)$ lùi dần về đầu. Sau khi kiểm tra xong (dù khớp hay lệch), cửa sổ sẽ được dịch chuyển dựa trên giá trị tra cứu ký tự cuối cửa sổ trong bảng $bmBc[y[j + m - 1]]$. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Raita được triển khai tuần tự qua 2 bước sau:

Thuật toán PreBmBc: // Tiền xử lý tính bảng dịch chuyển ký tự xấu của Horspool

Input:

- Mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống ASIZE. **Output:** Mảng giá trị dịch chuyển $bmBc[ASIZE]$. **Formats:** PreBmBc($X, m, bmBc$);

Actions:

```
i = 0;
while (i < ASIZE) {
    bmBc[i] = m;
    i++;
}

i = 0;
while (i < m - 1) {
    bmBc[X[i]] = m - 1 - i;
    i++;
}
EndActions.
```

Thuật toán Raita: Input:

- Mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .

Formats: Raita(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreBmBc($X, m, bmBc$);

Bước 2 (Lặp tìm kiếm):

```

j = 0;
firstCh = X[0];
middleCh = X[m / 2];
lastCh = X[m - 1];

while (j <= n - m) {
    if (lastCh == Y[j + m - 1] && firstCh == Y[j] && middleCh == Y[j + (m / 2)]) {
        i = m - 2;
        while (i > 0 && X[i] == Y[j + i]) {
            i--;
        }
        if (i <= 0) {
            <Tìm thấy mẫu ở vị trí j>;
        }
    }
    j = j + bmBc[Y[j + m - 1]];
}
EndActions.

```

3.4.2. Đặc điểm thuật toán

- Quá trình đối sánh ký tự bên trong cửa sổ được thực hiện theo một thứ tự cố định đặc biệt: Cuối -> Đầu -> Giữa -> Các ký tự còn lại từ phải sang trái. Cửa sổ tịnh tiến tuần tiến từ trái sang phải trên văn bản y.
- Pha tiền xử lý cực kỳ đơn giản và gọn nhẹ vì chỉ kế thừa lại mảng tra cứu ký tự xấu một chiều bmBc của Horspool. Độ phức tạp thời gian tiền xử lý là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ là $O(\sigma)$, với σ là kích thước bảng chữ cái.
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$. Tuy nhiên, nhờ cơ chế lọc trước 3 vị trí trọng yếu (cuối, đầu, giữa) bằng các phép so sánh riêng lẻ trực tiếp, thuật toán giảm thiểu tối đa việc phải gọi vòng lặp kiểm tra chuỗi, mang lại tốc độ thực thi rất nhanh trong thực tế đối với các hệ thống cơ sở dữ liệu văn bản thông thường.

3.4.3. Kiểm nghiệm thuật toán

Kiểm nghiệm Raita(x, m, y, n):

- **x xâu mẫu:** "GCAGAGAG", độ dài m = 8.
- Ký tự đầu: firstCh = x[0] = 'G'
- Ký tự giữa: middleCh = x[8 / 2] = x[4] = 'G'
- Ký tự cuối: lastCh = x[8 - 1] = x[7] = 'G'
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài n = 24.

Bước 1 (Tiền xử lý): Thực hiện hàm preBmBc, ta thu được mảng dịch chuyển dựa theo khoảng cách đến ký tự kế cuối của xâu mẫu:

- bmBc['A'] = 1 (Chữ 'A' xuất hiện muộn nhất ở chỉ số 6 -> 8 - 1 - 6 = 1)
- bmBc['G'] = 2 (Chữ 'G' xuất hiện muộn nhất ở chỉ số 5 tại đoạn trước -> 8 - 1 - 5 = 2. Lưu ý không xét x[7])
- bmBc['C'] = 6 (Chữ 'C' xuất hiện muộn nhất ở chỉ số 1 -> 8 - 1 - 1 = 6)

- $\text{bmBc}['T'] = 8$ (Ký tự 'T' không nằm trong mẫu $\rightarrow$ giá trị mặc định bằng $m = 8$)

Bước 2 (Lập tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Đoạn văn bản trong cửa sổ $y[j \dots j+7]$	Tiến trình kiểm tra điều kiện Raita	Khoảng cách dịch chuyển kế tiếp
First attempt	$j = 0$	"GCATCGC A"	1. Kiểm tra cuối: $\text{lastCh} = 'G'$ khác $y[7] = 'A'$ $\rightarrow$ Sai luôn . Không cần check đầu, giữa và thân.	$\text{shift} = \text{bmBc}[y[7]] = \text{bmBc}['A'] = 1 \rightarrow j = 1$
Second attempt	$j = 1$	"CATCGCA G"	1. Kiểm tra cuối: $\text{lastCh} = 'G'$ bằng $y[8] = 'G'$ $\rightarrow$ Đúng. 2. Kiểm tra đầu: $\text{firstCh} = 'G'$ khác $y[1] = 'C'$ $\rightarrow$ Sai .	$\text{shift} = \text{bmBc}[y[8]] = \text{bmBc}['G'] = 2 \rightarrow j = 3$
Third attempt	$j = 3$	"TCGCAGA G"	1. Kiểm tra cuối: $\text{lastCh} = 'G'$ bằng $y[10] = 'G'$ $\rightarrow$ Đúng. 2. Kiểm tra đầu: $\text{firstCh} = 'G'$ khác $y[3] = 'T'$ $\rightarrow$ Sai .	$\text{shift} = \text{bmBc}[y[10]] = \text{bmBc}['G'] = 2 \rightarrow j = 5$
Fourth attempt	$j = 5$	"GCAGAGA G"	1. Cuối: $\text{lastCh} = 'G' == y[12] = 'G'$ (Đúng). 2. Đầu: $\text{firstCh} = 'G' == y[5] = 'G'$ (Đúng). 3. Giữa: $\text{middleCh} = 'G' == y[9] = 'G'$ (Đúng). $\rightarrow$ Tiến hành quét thân từ $i = 6$ lùi về 1: khớp tất cả. Tìm thấy mẫu tại $j = 5$.	$\text{shift} = \text{bmBc}[y[12]] = \text{bmBc}['G'] = 2 \rightarrow j = 7$
Fifth attempt	$j = 7$	"AGAGAGT A"	1. Kiểm tra cuối: $\text{lastCh} = 'G'$ khác $y[14] = 'A'$ $\rightarrow$ Sai .	$\text{shift} = \text{bmBc}[y[14]] = \text{bmBc}['A'] = 1 \rightarrow j = 8$
Sixth attempt	$j = 8$	"GAGAGTA T"	1. Kiểm tra cuối: $\text{lastCh} = 'G'$ khác $y[15] = 'T'$ $\rightarrow$ Sai .	$\text{shift} = \text{bmBc}[y[15]] = \text{bmBc}['T'] = 8 \rightarrow j = 16$

Seventh attempt	j = 16	"TACAGTA C"	1. Kiểm tra cuối: lastCh='G' khác y[23]='C' -> Sai.	shift = bmBc[y[23]] = bmBc['C'] = 6 -> j = 22
Kết thúc	j = 22		Vòng lặp dừng vì j = 22 vượt quá điều kiện giới hạn (n - m = 16).	

Kết luận: Qua tiến trình kiểm nghiệm, nhờ cơ chế sàng lọc nhanh 3 ký tự, thuật toán Raita bỏ qua rất nhiều phép so sánh thừa và tìm thấy mẫu tại vị trí chỉ số **j = {5}**.

3.4.4. Cài đặt thuật toán #define ASIZE 256

```
// Bước 1: Pha tiền xử lý (Preprocessing)
void PreBmBc(const std::string& x, int m, std::vector<int>& bmBc) {
    int i;

    // Khởi tạo tất cả các phần tử mặc định bằng độ dài m
    for (i = 0; i < ASIZE; ++i) {
        bmBc[i] = m;
    }

    // Tính toán khoảng cách đến chỉ số sát cuối xâu mẫu
    for (i = 0; i < m - 1; ++i) {
        bmBc[(unsigned char)x[i]] = m - 1 - i;
    }
}

// Bước 2: Pha tìm kiếm (Searching)
void Raita(const std::string& x, int m, const std::string& y, int n) {
    std::vector<int> bmBc(ASIZE);

    /* Gọi Bước 1: Pha tiền xử lý tính toán mảng nhảy bmBc */
    PreBmBc(x, m, bmBc);

    int j = 0;
    int i;

    // Trích xuất các ký tự mốc chiến lược của xâu mẫu x
    unsigned char firstCh = x[0];
    unsigned char middleCh = x[m / 2];
    unsigned char lastCh = x[m - 1];

    /* Thực hiện Bước 2: Duyệt vòng lặp tìm kiếm của sô trên văn bản y */
    while (j <= n - m) {
        // Áp dụng cơ chế lọc Raita: Kiểm tra Cuối, Đầu, Giữa độc lập trước
        if (lastCh == (unsigned char)y[j + m - 1] &&
            firstCh == (unsigned char)y[j] &&
            middleCh == (unsigned char)y[j + (m / 2)]) {

            // Nếu cả 3 mốc đều khớp, tiến hành so sánh các ký tự thân từ
            // phải sang trái
            i = m - 2;
            while (i > 0 && x[i] == y[j + i]) {
                i--;
            }

            // Nếu chỉ số chạy i lùi được về mốc <= 0 tức là khớp hoàn toàn
            // xâu mẫu
        }
    }
}
```

```

        if (i <= 0) {
            std::cout << "Tim thay mau tai vi tri: " << j << std::endl;
        }
    }

    // Luôn dịch chuyển của sổ dựa trên ký tự nằm ở cuối của sổ hiện tại
    j += bmBc[(unsigned char)y[j + m - 1]];
}
}

```

3.5. Thuật toán Zhu-Takaoka

3.5.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Zhu-Takaoka là "mở rộng cơ chế dịch chuyển ký tự xấu của Boyer-Moore bằng cách xét đồng thời một cặp 2 ký tự liên tiếp để tăng tối đa khoảng cách dịch chuyển của sổ". Trong thuật toán Horspool, ta chỉ nhìn vào 1 ký tự cuối cùng của cửa sổ để tính bước nhảy. Việc này dễ bị hạn chế nếu ký tự đó xuất hiện quá nhiều trong mẫu.

Zhu-Takaoka cải tiến bằng cách: Pha tiên xử lý xây dựng một ma trận hai chiều mang tên `ztBc[ASIZE][ASIZE]`. Tại mỗi lượt quét, thuật toán thực hiện đối sánh các ký tự trong cửa sổ từ phải sang trái. Nếu xảy ra lệch khớp hoặc sau khi tìm thấy mẫu, thuật toán sẽ nhìn vào cặp 2 ký tự cuối cùng của cửa sổ hiện tại (ký tự tại vị trí $j + m - 2$ và $j + m - 1$) để tra cứu bước nhảy trong ma trận hai chiều này. Ngoài ra, thuật toán còn kết hợp song song với bảng dịch chuyển khớp tốt (good-suffix shift) giống như Boyer-Moore để đảm bảo luôn chọn được bước nhảy lớn nhất (MAX). Nhờ cơ chế xét cặp ký tự này, thuật toán đạt được hiệu năng nhảy cóc cực kỳ ấn tượng trên văn bản. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Zhu-Takaoka được triển khai tuần tự qua 2 bước sau:

Thuật toán PreZtBc:

Input:

- Mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống $ASIZE$.

Output: Ma trận hai chiều `ztBc[ASIZE][ASIZE]`.

Formats: `PreZtBc(X, m, ztBc);`

Actions:

```

i = 0;
while (i < ASIZE) {
    j = 0;
    while (j < ASIZE) {
        ztBc[i][j] = m;
        j++;
    }
    i++;
}
i = 0;
while (i < ASIZE) {
    ztBc[i][X[0]] = m - 1;
    i++;
}

```

```

}
i = 1;
while (i < m - 1) {
    ztBc[X[i - 1]][X[i]] = m - 1 - i;
    i++;
}

```

EndActions.

Thuật toán ZhuTakaoka:

Input: * Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .

- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .

Formats: ZhuTakaoka(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý): ``text PreZtBc(X, m, ztBc); PreBmGs(X, m, bmGs);

Bước 2 (Lặp tìm kiếm):

```

j = 0;
while (j <= n - m) {
    i = m - 1;
    while (i >= 0 && X[i] == Y[i + j]) {
        --i;
    }

    if (i < 0) {
        <Tìm thấy mẫu ở vị trí j>;
        j = j + bmGs[0];
    } else {
        c1 = Y[j + m - 2];
        c2 = Y[j + m - 1];
        j = j + MAX(bmGs[i], ztBc[c1][c2]);
    }
}

```

EndActions.

3.5.2. Đặc điểm thuật toán

- Thực hiện việc đối sánh ký tự bên trong cửa sổ theo chiều từ phải sang trái, cửa sổ tịnh tiến dần từ trái sang phải trên văn bản y .
- Pha tiền xử lý tốn nhiều bộ nhớ hơn một chút do cần khởi tạo ma trận hai chiều kích thước $\text{ASIZE} \times \text{ASIZE}$. Độ phức tạp thời gian tiền xử lý là $O(m + \sigma^2)$ và độ phức tạp không gian bộ nhớ là $O(\sigma^2)$, với σ là kích thước bảng chữ cái (ví dụ $\sigma = 256$ với mã ASCII).
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m * n)$, nhưng thời gian thực thi trung bình thực tế cực kỳ nhanh, đạt hiệu quả vượt trội so với Boyer-Moore thuần túy trên các văn bản có bảng chữ cái nhỏ hoặc chứa nhiều đoạn lặp lại.

3.5.3. Kiểm nghiệm thuật toán

Kiểm nghiệm ZhuTakaoka(x, m, y, n):

- **x mẫu:** "GCAGAGAG", độ dài m = 8.
- **Y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài n = 24.

Bước 1 (Tiền xử lý): Thực hiện tính toán ma trận hai chiều ztBc dựa trên khoảng cách của cặp ký tự kề nhau đến đuôi mẫu (chỉ liệt kê các cặp ký tự xuất hiện thực tế trong ví dụ):

- $ztBc['A']['G'] = 1$ (Cặp "AG" xuất hiện muộn nhất ở chỉ số 5 và 6 $\rightarrow 8 - 2 - 5 = 1$)
- $ztBc['G']['A'] = 2$ (Cặp "GA" xuất hiện muộn nhất ở chỉ số 4 và 5 $\rightarrow 8 - 2 - 4 = 2$)
- $ztBc['C']['A'] = 5$ (Cặp "CA" xuất hiện muộn nhất ở chỉ số 1 và 2 $\rightarrow 8 - 2 - 1 = 5$)
- $ztBc['G']['C'] = 6$ (Cặp "GC" xuất hiện muộn nhất ở chỉ số 0 và 1 $\rightarrow 8 - 2 - 0 = 6$)
- Tất cả các cặp chứa ký tự 'T' hoặc không nằm trong mẫu (nhũ "TC", "AT") đều nhận giá trị mặc định bằng m = 8.
- Giả định bảng khớp tốt bmGs tại các vị trí lệch tương ứng đều nhỏ hơn hoặc bằng bước nhảy của ztBc trong ví dụ này để tiện theo dõi nhánh xử lý chính.

Bước 2 (Lặp tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Cặp ký tự cuối cửa sổ $y[j+6]$ và $y[j+7]$	Tiến trình so khớp ký tự (Từ phải sang trái)	Công thức tính bước nhảy dịch chuyển cửa sổ kế tiếp	Khoảng cách dịch chuyển
First attempt	j = 0	"CA"	Lệch ngay từ đầu: $x[7]='G'$ khác $y[7]='A'$	$shift = \max(ztBc['C']['A'], bmGs[7]) = \max(5, bmGs[7])$	$shift = 5 \rightarrow j = 5$
Second attempt	j = 5	"AG"	Khớp hoàn toàn cả 8 ký tự: "GCAGAGAG"	Tìm thấy mẫu tại j = 5. $shift = bmGs[0]$	$shift = 2 \rightarrow j = 7$
Third attempt	j = 7	"TA"	Lệch ngay từ đầu: $x[7]='G'$ khác $y[14]='A'$	$shift = \max(ztBc['T']['A'], bmGs[7]) = \max(8, bmGs[7])$	$shift = 8 \rightarrow j = 15$
Fourth attempt	j = 15	"GT"	Lệch ngay từ đầu: $x[7]='G'$ khác $y[22]='C'$	Cặp "GT" không có trong mẫu $\rightarrow$ $ztBc['G']['T'] = 8$	$shift = 8 \rightarrow j = 23$
Kết thúc	j = 23			Vòng lặp dừng vì j = 23 vượt quá điều kiện ($n - m = 16$).	

3.5.4. Cài đặt thuật toán

```
#define ASIZE 256

// Hàm giả lập tiền xử lý bảng khớp tốt bmGs của Boyer-Moore
void PreBmGs(const std::string& x, int m, std::vector<int>& bmGs) {
    for (int i = 0; i < m; ++i) {
        bmGs[i] = m;
    }
    bmGs[0] = 2;
}

// Bước 1: Pha tiền xử lý (Preprocessing)
void PreZtBc(const std::string& x, int m, std::vector<std::vector<int>>&
ztBc) {
    int i, j;

    for (i = 0; i < ASIZE; ++i) {
        for (j = 0; j < ASIZE; ++j) {
            ztBc[i][j] = m;
        }
    }

    for (i = 0; i < ASIZE; ++i) {
        ztBc[i][(unsigned char)x[0]] = m - 1;
    }

    for (i = 0; i < m - 2; ++i) {
        ztBc[(unsigned char)x[i]][(unsigned char)x[i + 1]] = m - 2 - i;
    }
}

// Bước 2: Pha tìm kiếm (Searching)
void ZhuTakaoka(const std::string& x, int m, const std::string& y, int n) {
    std::vector<std::vector<int>> ztBc(ASIZE, std::vector<int>(ASIZE));
    std::vector<int> bmGs(m);

    /* Gọi Bước 1: Pha tiền xử lý dữ liệu mẫu */
    PreZtBc(x, m, ztBc);
    PreBmGs(x, m, bmGs);

    int j = 0;
    int i;

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự trên văn bản y */
    while (j <= n - m) {
        i = m - 1;

        while (i >= 0 && x[i] == y[j + i]) {
            i--;
        }

        if (i < 0) {
            std::cout << "Tìm thay mẫu tại vị trí: " << j << std::endl;
            j += bmGs[0];
        } else {
            unsigned char c1 = (unsigned char)y[j + m - 2];
            unsigned char c2 = (unsigned char)y[j + m - 1];

            j += std::max(ztBc[c1][c2], bmGs[i]);
        }
    }
}
```

3.6. Thuật toán Turbo BM

3.6.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Turbo-BM là "tăng tốc thuật toán Boyer-Moore bằng cách ghi nhớ đoạn văn bản đã khớp hoàn toàn ở lượt quét ngay trước đó nhằm bỏ qua các phép so sánh trùng lặp và kích hoạt các bước nhảy tăng tốc (turbo-shifts)". Trong thuật toán Boyer-Moore nguyên bản, thông tin về đoạn ký tự đã khớp ở cửa sổ trước sẽ bị quên đi khi dịch chuyển sang cửa sổ mới, dẫn đến việc máy tính phải so sánh lại một số ký tự thừa.

Turbo-BM giải quyết triệt để vấn đề này bằng cách duy trì hai biến nhớ: biến u lưu độ dài đoạn ký tự đã khớp ở lượt trước (chỉ ghi nhận khi bước nhảy trước đó là bước nhảy khớp tốt - good-suffix shift), và biến v lưu độ dài đoạn ký tự khớp được ở lượt hiện tại. Khi tiến hành đối sánh từ phải sang trái trong cửa sổ mới, nếu con trỏ mẫu i lùi chạm đến ranh giới của đoạn đã biết, thuật toán có thể nhảy cóc qua đoạn này để kiểm tra tiếp phần phía trước. Hơn nữa, nếu xảy ra tình trạng lệch khớp mà đoạn khớp hiện tại v ngắn hơn đoạn khớp cũ u , thuật toán sẽ kích hoạt cơ chế "turbo-shift" với bước nhảy tối thiểu bằng hiệu số ($u - v$) hoặc ép dịch chuyển an toàn một khoảng lớn hơn nhờ việc phát hiện các ký tự xung đột không thể trùng khít nhau. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Turbo-BM được triển khai tuần tự qua 2 bước sau:

Thuật toán PreTBM: // Thực hiện các bước tiền xử lý tính mảng ký tự xấu và khớp tốt
Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống ASIZE.

Output: Mảng ký tự xấu $bmBc[ASIZE]$ và mảng khớp tốt $bmGs[m]$.

Formats: PreTBM($X, m, bmBc, bmGs$);

Actions:

// Sử dụng nguyên vẹn hai hàm tiền xử lý của thuật toán Boyer-Moore tiêu chuẩn
PreBmBc($X, m, bmBc$);
PreBmGs($X, m, bmGs$);

EndActions.

Thuật toán TBM: Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .

Formats: TBM(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreTBM($X, m, bmBc, bmGs$);

Bước 2 (Lặp tìm kiếm):

```
j = 0;  
u = 0;  
shift = m;  
  
while (j <= n - m) {
```

```

i = m - 1;
while (i >= 0 && X[i] == Y[i + j]) {
    --i;
    if (u != 0 && i == m - 1 - shift) {
        i = i - u;
    }
}

if (i < 0) {
    <Tìm thấy mẫu ở vị trí j>;
    shift = bmGs[0];
    u = m - shift;
} else {
    v = m - 1 - i;
    turboShift = u - v;
    bcShift = bmBc[Y[i + j]] - m + 1 + i;
    shift = MAX(turboShift, bcShift);
    shift = MAX(shift, bmGs[i]);

    if (shift == bmGs[i]) {
        u = MIN(m - shift, v);
    } else {
        if (turboShift < bcShift) {
            shift = MAX(shift, u + 1);
        }
        u = 0;
    }
}
j = j + shift;
}
EndActions.

```

3.6.2. Đặc điểm thuật toán

- Quá trình đối sánh ký tự bên trong cửa sổ thực hiện theo chiều từ phải sang trái, có khả năng nhảy cóc qua đoạn ký tự đã biết. Cửa sổ dịch chuyển tuần tiến từ trái sang phải trên văn bản y.
- Pha tiền xử lý hoàn toàn không tốn thêm bất kỳ chi phí thời gian hay không gian bộ nhớ nào so với Boyer-Moore truyền thống. Độ phức tạp thời gian tiền xử lý là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ là $O(m + \sigma)$, với σ là kích thước bảng chữ cái.
- Pha tìm kiếm đạt hiệu năng tuyến tính tối ưu vượt bậc với độ phức tạp thời gian trong trường hợp xấu nhất được khống chế nghiêm ngặt ở mức $O(n)$ (trong khi Boyer-Moore thuần túy có thể bị suy biến thành $O(m \times n)$).
- Số lượng phép so sánh ký tự tối đa trên toàn bộ văn bản trong trường hợp xấu nhất được chặn trên bởi $2n$ phép toán, mang lại sự ổn định tuyệt đối cho hệ thống tìm kiếm thời gian thực.

3.6.3. Kiểm nghiệm thuật toán

Kiểm nghiệm TBM(x, m, y, n):

- **x mẫu:** "GCAGAGAG", độ dài m = 8.
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài n = 24.

Bước 1 (Tiền xử lý): Kết quả tính toán mảng ký tự xấu bmBc và mảng khớp tốt bmGs chuẩn theo dữ liệu hình ảnh số 5 (image_d8a609.png):

- $bmBc['A'] = 1, bmBc['G'] = 2, bmBc['C'] = 6, bmBc['T'] = 8.$
- $bmGs = \{7, 7, 7, 2, 7, 4, 7, 1\}$

Bước 2 (Lập tìm kiếm):

Lượt quét	Vị trí cửa sổ (j)	Biến u hiện tại	Biến v hiện tại	Tiến trình đối sánh ký tự và kích hoạt Turbo	Công thức và khoảng cách dịch chuyển kế tiếp
First attempt	j = 0	u = 0	v = 0	So sánh $x[7] \neq 'G'$ khác $y[7] = 'A'$ (Lệch tại i = 7). Tính toán bước nhảy thông thường: $bcShift = bmBc['A'] - 8 + 1 + 7 = 1 - 8 + 1 + 7 = 1.$	$shift = \max(bmGs[7], bcShift) = \max(1, 1) = 1.$ Vì dịch chuyển theo bmGs nên cập nhật biến nhớ: $u = \min(8 - 1, 0) = 0.$ Cửa sổ mới: $j = 0 + 1 = 1.$
Second attempt	j = 1	u = 0	v = 1	So sánh $x[7] \neq 'G'$ (Khớp). Tiếp tục lùi sang $x[6] = 'A'$ khác $y[7] = 'A'$ (Lệch tại i = 6). Tính toán: $v = 8 - 1 - 6 = 1. bcShift = bmBc['A'] - 8 + 1 + 6 = 1 - 8 + 1 + 6 = 0.$	$shift = \max(bmGs[6], bcShift) = \max(7, 0) = 7.$ Do dịch chuyển theo bmGs nên cập nhật biến nhớ: $u = \min(8 - 7, 1) = 1.$ Cửa sổ mới: $j = 1 + 7 = 8.$
Third attempt	j = 8	u = 1	v = 0	Bắt đầu tại i = 7: $x[7] \neq 'G'$ trùng $y[15] = 'T' \rightarrow$ Sai ngay lập tức. Lúc này $v = 0,$ $turboShift = 1 - 0 = 1.$ $bcShift = bmBc['T'] - 8 + 1 + 7 = 8 - 8 + 1 +$	Do shift chọn $bcShift = 8,$ điều kiện $turboShift < bcShift$ thỏa mãn $\rightarrow$ ép $shift = \max(8, 1 + 1) = 8.$ Đặt lại $u = 0.$ Cửa sổ mới: $j = 8 + 8 = 16.$

				7 = 8.	
Kết thúc	j = 16			Vòng lặp kết thúc vì vị trí của số kế tiếp vượt quá điều kiện biên an toàn (n - m = 16).	

Kết luận: Xâu mẫu được quét qua các chặng dịch chuyển tăng tốc an toàn và cho ra kết quả tìm kiếm tuyến tính tối ưu.

3.6.4. Cài đặt thuật toán

```
// Hàm tiền xử lý ký tự xâu bmBc tiêu chuẩn của Boyer-Moore
void PreBmBc(const std::string& x, int m, std::vector<int>& bmBc) {
    int i;
    for (i = 0; i < ASIZE; ++i) {
        bmBc[i] = m;
    }
    for (i = 0; i < m - 1; ++i) {
        bmBc[(unsigned char)x[i]] = m - 1 - i;
    }
}

// Hàm tiền xử lý khớp tốt bmGs tiêu chuẩn của Boyer-Moore
void PreBmGs(const std::string& x, int m, std::vector<int>& bmGs) {
    int i, j;
    std::vector<int> suff(m);

    suff[m - 1] = m;
    int g = m - 1;
    int f = m - 1;
    for (i = m - 2; i >= 0; --i) {
        if (i > g && suff[i + m - 1 - f] < i - g) {
            suff[i] = suff[i + m - 1 - f];
        } else {
            if (i < g) g = i;
            f = i;
            while (g >= 0 && x[g] == x[g + m - 1 - f]) {
                --g;
            }
            suff[i] = f - g;
        }
    }

    for (i = 0; i < m; ++i) {
        bmGs[i] = m;
    }

    j = 0;
    for (i = m - 1; i >= 0; --i) {
        if (suff[i] == i + 1) {
            for (; j < m - 1 - i; ++j) {
                if (bmGs[j] == m) {
                    bmGs[j] = m - 1 - i;
                }
            }
        }
    }
}
```

```

        for (i = 0; i < m - 1; ++i) {
            bmGs[m - 1 - suff[i]] = m - 1 - i;
        }
    }

/**
 * Hàm tìm kiếm chuỗi sử dụng giải thuật nâng cao Turbo-BM
 */
void TBM(const std::string& x, int m, const std::string& y, int n) {
    std::vector<int> bmGs(XSIZE);
    std::vector<int> bmBc(ASIZE);

    /* Bước 1: Gọi pha tiền xử lý cấu trúc dữ liệu mẫu */
    PreBmGs(x, m, bmGs);
    PreBmBc(x, m, bmBc);

    /* Bước 2: Vòng lặp duyệt tìm kiếm tăng tốc trên văn bản Y */
    int j = 0;
    int u = 0;
    int shift = m;
    int i, v, turboShift, bcShift;

    while (j <= n - m) {
        i = m - 1;

        // Tiến hành so khớp ngược từ phải sang trái kết hợp nhảy cóc qua
        // đoạn u đã biết
        while (i >= 0 && x[i] == y[i + j]) {
            --i;
            if (u != 0 && i == m - 1 - shift) {
                i -= u;
            }
        }

        // Kiểm tra trạng thái khớp hoàn toàn của sỏ
        if (i < 0) {
            std::cout << "Tim thay mau tai vi tri: " << j << std::endl;
            shift = bmGs[0];
            u = m - shift;
        } else {
            // Trường hợp xảy ra lệch khớp ký tự: Tính toán các giá trị bước
            // nhảy tối ưu
            v = m - 1 - i;
            turboShift = u - v;
            bcShift = bmBc[(unsigned char)y[i + j]] - m + 1 + i;

            shift = std::max(turboShift, bcShift);
            shift = std::max(shift, bmGs[i]);

            // Cập nhật trạng thái biến nhớ u cho lượt quét kế tiếp
            if (shift == bmGs[i]) {
                u = std::min(m - shift, v);
            } else {
                if (turboShift < bcShift) {
                    shift = std::max(shift, u + 1);
                }
                u = 0;
            }
        }

        // Dịch chuyển của sỏ tìm kiếm
        j += shift;
    }
}

```

3.7. Thuật toán Berry-Ravindran

3.7.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Berry-Ravindran là "mở rộng cơ chế dịch chuyển ký tự xấu bằng cách xét đồng thời một cặp 2 ký tự liên tiếp nằm ngay phía sau cửa sổ so khớp hiện tại để tối đa hóa khoảng cách tịnh tiến". Khác với các thuật toán như Boyer-Moore hay Horspool chỉ xét ký tự trong phạm vi cửa sổ, Berry-Ravindran chuyển góc nhìn sang hai ký tự kế tiếp trên văn bản y (ký tự tại vị trí $j + m$ và $j + m + 1$) ngay khi phát hiện lệch khớp hoặc sau khi tìm thấy xấu mẫu.

Để thực hiện cơ chế này, pha tiền xử lý của thuật toán xây dựng một ma trận hai chiều $brBc[ASIZE][ASIZE]$. Giá trị trong ma trận được tính toán dựa trên khoảng cách từ vị trí xuất hiện muộn nhất của cặp ký tự đó đến đầu hoặc đuôi xấu mẫu. Khi tìm kiếm, thuật toán đối sánh xấu mẫu với văn bản từ trái sang phải hoặc từ phải sang trái (hoặc kết hợp). Nếu có sự lệch khớp, thuật toán không cần quan tâm đến vị trí lệch mà lập tức tra cứu cặp ký tự $y[j + m]$ và $y[j + m + 1]$ trong ma trận $brBc$ để thực hiện bước nhảy. Nhờ việc xét cặp ký tự nằm ngoài cửa sổ này, thuật toán luôn đạt được bước nhảy tối thiểu là 1 và tối đa lên tới $m + 2$, giúp tốc độ lướt trên văn bản cực kỳ nhanh. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Berry-Ravindran được triển khai tuần tự qua 2 bước sau:

Thuật toán PreBrBc: // Thực hiện bước tiền xử lý tính ma trận hai chiều ký tự xấu kế tiếp **Input:**

- Xấu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống $ASIZE$.

Output: Ma trận hai chiều $brBc[ASIZE][ASIZE]$.

Formats: $PreBrBc(X, m, brBc)$;

Actions:

$i = 0$;

```
while (i < ASIZE) {  
    j = 0;  
    while (j < ASIZE) {  
         $brBc[i][j] = m + 2$ ;  
        j++;  
    }  
    i++;  
}
```

$i = 0$;

```
while (i < ASIZE) {  
     $brBc[i][X[0]] = m + 1$ ;  
    i++;  
}
```

$i = 0$;

```
while (i < m - 1) {  
     $brBc[X[i]][X[i + 1]] = m - 1 - i$ ;  
    i++;  
}
```

```

}

i = 0;
while (i < ASIZE) {
    brBc[X[m - 1]][i] = 1;
    i++;
}
EndActions.

```

Thuật toán BerryRavindran: Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .

Formats: BerryRavindran(X, m, Y, n);

Actions: Bước 1 (Tiền xử lý):

PreBrBc($X, m, brBc$);

Bước 2 (Lặp tìm kiếm):

$Y[n] = 0$;

$Y[n + 1] = 0$;

$j = 0$;

```

while (j <= n - m) {
    i = 0;
    while (i < m && X[i] == Y[j + i]) {
        i++;
    }

    if (i >= m) {
        <Tìm thấy mẫu ở vị trí j>;
    }

    c1 = Y[j + m];
    c2 = Y[j + m + 1];
    j = j + brBc[c1][c2];
}
EndActions.

```

3.7.2. Đặc điểm thuật toán

- Tiến trình so khớp ký tự bên trong cửa sổ có thể thực hiện theo chiều từ trái sang phải, cửa sổ tịnh tiến dần từ trái sang phải trên văn bản y .
- Pha tiền xử lý yêu cầu khởi tạo và tính toán trên lưới ma trận hai chiều kích thước $ASIZE \times ASIZE$. Độ phức tạp thời gian tiền xử lý là $O(m + \sigma^2)$ và độ phức tạp không gian bộ nhớ là $O(\sigma^2)$, với σ là kích thước bảng chữ cái (ví dụ $\sigma = 256$ với mã ASCII).
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m * n)$. Tuy nhiên, trên thực tế và trong trường hợp trung bình, nhờ tận dụng cặp ký tự đứng ngay sau cửa sổ, thuật toán Berry-Ravindran cho phép thực hiện những bước

nhảy rất lớn (lên tới $m + 2$), giúp giảm đáng kể số lần kiểm tra cửa sổ và mang lại hiệu năng thực thi vượt trội trên các văn bản lớn.

3.7.3 Kiểm nghiệm thuật toán

Kiểm nghiệm BerryRavindran(x, m, y, n):

- **x chuỗi mẫu:** "GCAGAGAG", độ dài $m = 8$.
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài $n = 24$.
- Thêm 2 ký tự lính canh vào cuối văn bản để tránh vượt quá chỉ số mảng: $y[24] = 0, y[25] = 0$.

Bước 1 (Tiền xử lý): Thực hiện tính toán ma trận hai chiều brBc dựa theo logic các vòng lặp của chuỗi mẫu "GCAGAGAG":

- Khởi tạo toàn bộ ma trận bằng $m + 2 = 10$.
- Với vòng lặp gán $brBc[i][x[0]] = m + 1$, mọi cặp có ký tự thứ hai là 'G' (như "AG", "CG", "GG", "TG") tạm thời nhận giá trị 9.
- Vòng lặp quét thân mẫu $m - 1 - i$ cập nhật cho các cặp xuất hiện thực tế:
 - o Cặp "GC" ($i=0$) $\rightarrow brBc['G']['C'] = 8 - 1 - 0 = 7$
 - o Cặp "CA" ($i=1$) $\rightarrow brBc['C']['A'] = 8 - 1 - 1 = 6$
 - o Cặp "AG" ($i=2, i=4, i=6$) $\rightarrow$ Lấy vị trí muộn nhất ở $i=6$: $brBc['A']['G'] = 8 - 1 - 6 = 1$
 - o Cặp "GA" ($i=3, i=5$) $\rightarrow$ Lấy vị trí muộn nhất ở $i=5$: $brBc['G']['A'] = 8 - 1 - 5 = 2$
- Vòng lặp cuối cùng $brBc[x[m-1]][i] = 1$, mọi cặp có ký tự đầu tiên là 'G' (do $x[7] = 'G'$) đều được cập nhật bằng 1 (ví dụ: "GA", "GC", "GT", "GG" đều bằng 1). Do vòng lặp này chạy sau cùng nên nó ghi đè lên các giá trị trước đó.
- Kết quả tra cứu một số cặp ký tự cần dùng trong bài toán mẫu:
 - o $brBc['A']['G'] = 1$ (do bắt đầu bằng 'A', giữ nguyên giá trị vòng lặp thân).
 - o $brBc['G']['A'] = 1$ (bị vòng lặp cuối ghi đè vì bắt đầu bằng 'G').
 - o $brBc['T']['A'] = 10$ (mặc định vì không xuất hiện trong các điều kiện mẫu).

Bước 2 (Lặp tìm kiếm):

Vị trí cửa sổ (j)	Cặp ký tự ngay sau cửa sổ $y[j+8]$ và $y[j+9]$	Tiến trình so khớp ký tự (Từ trái sang phải)	Công thức tính bước nhảy dịch chuyển cửa sổ kế tiếp	Khoảng cách dịch chuyển
$j = 0$	"AG"	Lệch ở vị trí thứ 3: $x[2]='A'$ khác $y[2]='C'$	$shift = brBc[y[8]][y[9]]$ $= brBc['A']['G']$	$shift = 1 \rightarrow j = 1$
$j = 1$	"GA"	Lệch ngay từ đầu: $x[0]='G'$ khác $y[1]='C'$	$shift = brBc[y[9]][y[10]]$ $= brBc['G']['A']$	$shift = 1 \rightarrow j = 2$
$j = 2$	"AG"	Lệch ngay từ đầu: $x[0]='G'$ khác	$shift =$ $brBc[y[10]][y[11]] =$	$shift = 1 \rightarrow j = 3$

		y[2]='C'	brBc['A']['G']	
j = 3	"GA"	Lệch ngay từ đầu: x[0]='G' khác y[3]='T'	shift = brBc[y[11]][y[12]] = brBc['G']['A']	shift = 1 -> j = 4
j = 4	"AG"	Lệch ngay từ đầu: x[0]='G' khác y[4]='C'	shift = brBc[y[12]][y[13]] = brBc['A']['G']	shift = 1 -> j = 5
j = 5	"TA"	Khớp hoàn toàn cả 8 ký tự trên cửa sổ văn bản.	Tìm thấy mẫu tại j = 5. shift = brBc[y[13]][y[14]] = brBc['T']['A']	shift = 10 -> j = 15
j = 15	"CG"	Lệch ngay từ đầu: x[0]='G' khác y[15]='T'	Cặp y[23] và y[24] là "C" và ký tự kế tiếp. Tra bảng brBc['C']['G'] = 9.	shift = 9 -> j = 24
j = 24			Vòng lặp dừng vì j = 24 vượt quá điều kiện biên (n - m = 16).	

Kết luận: Thuật toán Berry-Ravindran tìm thấy xâu mẫu xuất hiện hoàn chỉnh tại vị trí chỉ số **j = {5}**.

3.7.4. Cài đặt thuật toán

```
//Bước 1: Pha tiền xử lý (Preprocessing)
void PreBrBc(const std::string& x, int m, std::vector<std::vector<int>>&
brBc) {
    int i, j;

    // 1. Khởi tạo toàn bộ lưới ma trận với giá trị mặc định là m + 2
    for (i = 0; i < ASIZE; ++i) {
        for (j = 0; j < ASIZE; ++j) {
            brBc[i][j] = m + 2;
        }
    }

    // 2. Đặt giá trị m + 1 cho các cặp có ký tự thứ hai trùng với ký tự đầu
    của mẫu
    for (i = 0; i < ASIZE; ++i) {
        brBc[i][(unsigned char)x[0]] = m + 1;
    }

    // 3. Tính khoảng cách từ các cặp ký tự trong thân mẫu đến đầu mẫu
    for (i = 0; i < m - 1; ++i) {
        brBc[(unsigned char)x[i]][(unsigned char)x[i + 1]] = m - 1 - i;
    }

    // 4. Đặt giá trị bằng 1 cho các cặp có ký tự đầu tiên trùng với ký tự
    cuối của mẫu
```

```

        for (i = 0; i < ASIZE; ++i) {
            brBc[(unsigned char)x[m - 1]][i] = 1;
        }
    }

//Bước 2: Pha tìm kiếm chuỗi (Searching)
void BerryRavindran(const std::string& x, int m, std::string y, int n) {
    // Khởi tạo ma trận hai chiều brBc kích thước ASIZE x ASIZE
    std::vector<std::vector<int>> brBc(ASIZE, std::vector<int>(ASIZE));

    /* Gọi Bước 1: Pha tiền xử lý dữ liệu chuỗi mẫu x */
    PreBrBc(x, m, brBc);

    // Bổ sung các ký tự lính canh vào cuối văn bản để tránh lỗi tràn mảng
    // khi kiểm tra j + m
    y.push_back('\0');
    y.push_back('\0');

    int j = 0;
    int i;

    /* Thực hiện Bước 2: Lặp tìm kiếm chuỗi ký tự trên văn bản y */
    while (j <= n - m) {
        i = 0;

        // Tiến hành so khớp thuận từ trái sang phải trong cửa sổ hiện tại
        while (i < m && x[i] == y[j + i]) {
            i++;
        }

        // Nếu chỉ số i đạt bằng độ dài m tức là đã khớp hoàn toàn
        if (i >= m) {
            std::cout << "Tìm thấy mẫu tại vị trí: " << j << std::endl;
        }

        // Trích xuất cặp 2 ký tự nằm ngay kế tiếp sau cửa sổ so khớp hiện tại
        unsigned char c1 = (unsigned char)y[j + m];
        unsigned char c2 = (unsigned char)y[j + m + 1];

        // Dịch chuyển mốc j bằng cách tra cứu giá trị trong ma trận brBc
        j += brBc[c1][c2];
    }
}

```

CHƯƠNG 4. THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH

4.1. Thuật toán Galil-Seiferas

4.1.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Galil-Giancarlo là "tối ưu hóa thuật toán Colussi bằng cách ghi nhớ các đoạn ký tự đã khớp và xử lý đặc biệt khi xâu mẫu không phải là lũy thừa của một ký tự đơn đơn lẻ ($x \neq c^m$)". Thuật toán chia xâu mẫu thành các phần dựa trên chỉ số ell (chỉ số cuối cùng mà tại đó các ký tự liên tiếp trùng nhau $x[0] == x[i]$).

Pha tiền xử lý gọi lại hàm preColussi để tính toán các mảng phụ trợ bao gồm h, next, và shift để xác định thứ tự đối sánh các vị trí "không lỗ" (noholes) và các bước nhảy. Khi tìm kiếm trên văn bản y, thuật toán sử dụng một biến cờ hiệu mang tên heavy để ghi nhớ trạng thái của lượt quét trước. Nếu lượt quét trước tất cả các vị trí không lỗ đều khớp và một phần hậu tố của mẫu đã khớp, thuật toán ở lượt quét kế tiếp sẽ không so sánh lại từ đầu mà nhảy cóc qua đoạn văn bản đã biết, tiếp tục quét từ vị trí kế tiếp ($last + 1$) cho đến khi chạm tới cuối đoạn hoặc phát hiện ký tự xung đột với $x[0]$. Nhờ cơ chế quản lý bộ nhớ đệm này, thuật toán không chế số phép so sánh ở mức cực thấp. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Galil-Giancarlo được triển khai tuần tự qua 2 bước sau:

Thuật toán PreGG: // Thực hiện bước tiền xử lý tính toán cấu trúc mảng của Colussi

Input: * Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.

Output: Chỉ số nd và các mảng $h[]$, $next[]$, $shift[]$ tính từ hàm preColussi.

Formats: PreGG(X, m, h, next, shift);

Actions:

ell = 0;

```
while (X[ell] == X[ell + 1]) {  
    ell++;  
}
```

```
if (ell != m - 1) {
```

```
    nd = preColussi(X, m, h, next, shift);
```

```
}
```

EndActions.

Thuật toán GalilGiancarlo: Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.

- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n.

Output: Tất cả vị trí xuất hiện X trong Y.

Formats: GalilGiancarlo(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreGG(X, m, h, next, shift);

Bước 2 (Lặp tìm kiếm):

```
if (ell == m - 1) {
```

```
    // Trường hợp đặc biệt x xâu mẫu là lũy thừa của 1 ký tự đơn (ví dụ: "aaaa")
```

```
    j = ell = 0;
```



```

while (j < n) {
    if (X[0] == Y[j]) {
        ell++;
        if (ell >= m) {
            <Tìm thấy mẫu ở vị trí j - m + 1>;
        }
    } else {
        ell = 0;
    }
    j++;
}
} else {
    // Trường hợp tổng quát sử dụng máy trạng thái Colussi cải tiến
    i = 0;
    j = 0;
    heavy = 0;
    last = -1;

    while (j <= n - m) {
        if (heavy != 0 && i == 0) {
            k = last - j + 1;
            while (X[0] == Y[j + k]) {
                k++;
            }
            if (k <= ell || X[ell + 1] != Y[j + k]) {
                i = 0;
                j = j + (k + 1);
                last = j - 1;
            } else {
                i = 1;
                last = j + k;
                j = last - (ell + 1);
            }
            heavy = 0;
        } else {
            while (i < m && last < j + h[i] && X[h[i]] == Y[j + h[i]]) {
                i++;
            }
            if (i >= m || last >= j + h[i]) {
                <Tìm thấy mẫu ở vị trí j>;
                i = m;
            }
            if (i > nd) {
                last = j + m - 1;
            }
            j = j + shift[i];
            i = next[i];
        }
    }
}

```

```

    heavy = (j > last ? 0 : 1);
  }
}
EndActions.

```

4.1.2. Đặc điểm thuật toán

- Thứ tự so sánh ký tự bên trong cửa sổ không tuân tiến theo chiều tự nhiên mà nhảy cóc phụ thuộc vào cấu trúc mảng tuần tự $h[i]$ được tính sẵn từ thuật toán Colussi, cửa sổ dịch chuyển tuần tiến từ trái sang phải trên văn bản y.
- Pha tiền xử lý thừa kế toàn bộ cấu trúc của thuật toán Colussi nâng cao. Độ phức tạp thời gian tiền xử lý là $O(m)$ và độ phức tạp không gian bộ nhớ lưu trữ các mảng tra cứu một chiều là $O(m)$.
- Pha tìm kiếm đạt hiệu năng tuyến tính tối ưu nghiêm ngặt trong mọi tình huống với độ phức tạp thời gian trong trường hợp xấu nhất là $O(n)$.
- Số lượng phép so sánh ký tự tối đa trên văn bản trong trường hợp xấu nhất được chặn trên chặt chẽ ở mức $(4/3)n$ phép toán, giúp nó trở thành một trong những thuật toán có số phép đối sánh ít nhất thuộc nhóm thuật toán xử lý chuỗi ký tự độ dài cố định.

4.1.3. Kiểm nghiệm thuật toán

Kiểm nghiệm GG(x, m, y, n):

- **x mẫu:** "GCAGAGAG", độ dài $m = 8$.
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG", độ dài $n = 24$.

Bước 1 (Tiền xử lý): Do $x[0]='G'$ khác $x[1]='C'$ nên $ell = 0$. Giá trị $nd = 3$ và kết quả sau bước tiền xử lý:

- $h = \{1, 2, 4, 6, 7, 5, 3, 0\}$ (Thứ tự kiểm tra vị trí ký tự trong cửa sổ)
- $shift = \{1, 2, 3, 5, 8, 7, 7, 7\}$
- $next = \{0, 0, 0, 0, 0, 0, 0, 0\}$

Bước 2 (Lặp tìm kiếm):

Lượt quét	j	heavy	Tiến trình đối sánh (vị trí $h[i]$)	Chỉ số i kết thúc	Cập nhật chỉ số kế tiếp
1	0	0	$x[1]==y[1]$ (Khớp) $x[2]==y[2]$ (Khớp) $x[4]!=y[4]$ (Lệch)	2	$last = -1$ $j = 0 + shift[2] = 3$ $i = next[2] = 0$
2	3	0	$x[1]!=y[4]$ (Lệch)	0	$last = -1$ $j = 3 + shift[0] = 4$ $i = next[0] = 0$
3	4	0	Khớp tuần tự cả 8 ký tự theo mảng h.	8	Tìm thấy mẫu tại j = 4 $last = 4 + 8 - 1 = 11$ $j = 4 + shift[8] = 11$ $i = next[8] = 0$

4	11	1	<i>Kích hoạt nhánh rẽ heavy:</i> $k = 11 - 11 + 1 = 1$ $x[0]! = y[12]$ (Lệch)	0	<i>Nhảy trực tiếp:</i> $i = 0$ $j = 11 + 1 + 1 = 13$ $last = 13 - 1 = 12$
Dừng	13		Vòng lặp dừng vì $j = 13 > (n - m = 16)$		

Kết luận: Xâu mẫu xuất hiện hoàn chỉnh tại vị trí chỉ số $j = \{4\}$.

4.1.4. Cài đặt thuật toán

```

int preColussi(const std::string& x, int m, std::vector<int>& h,
std::vector<int>& next, std::vector<int>& shift) {
    int i, k, nd, q, r, s;
    std::vector<int> hmax(XSIZE, 0);
    std::vector<int> kmin(XSIZE, 0);
    std::vector<int> nhd0(XSIZE, 0);
    std::vector<int> rmin(XSIZE, 0);

    i = k = 1;
    do {
        while (i < m && x[i] == x[i - k]) {
            i++;
        }
        hmax[k] = i;
        q = k + 1;
        while (q < m && hmax[q - k] + k < i) {
            hmax[q] = hmax[q - k] + k;
            q++;
        }
        k = q;
    } while (k < m && k <= i);

    // Xử lý các giá trị mặc định cho phần còn lại của hmax, kmin, rmin
    for (i = 0; i < m; ++i) {
        kmin[i] = 0;
        rmin[i] = m;
    }

    for (i = m - 1; i >= 1; --i) {
        if (hmax[i] < m) {
            kmin[hmax[i]] = i;
            rmin[hmax[i]] = std::min(rmin[hmax[i]], i);
        }
    }

    r = m;
    s = -1;
    for (i = 0; i < m; ++i) {
        if (kmin[i] == 0) {
            h[--r] = i;
        } else {
            h[++s] = i; // Đưa các phần tử có kmin (holes) lên phía trước
        }
    }
    nd = s; // nd ghi nhận số lượng phần tử thuộc tập noholes
}

```

```

    for (i = 0; i <= nd; ++i) {
        shift[i] = kmin[h[i]];
    }
    for (i = nd + 1; i < m; ++i) {
        shift[i] = rmin[h[i]];
    }
    shift[m] = rmin[0];

    s = 0;
    for (i = 0; i < m; ++i) {
        nhd0[i] = s;
        if (kmin[i] > 0) {
            ++s;
        }
    }

    for (i = 0; i <= nd; ++i) {
        next[i] = nhd0[h[i] - kmin[h[i]]];
    }
    for (i = nd + 1; i < m; ++i) {
        next[i] = nhd0[m - rmin[h[i]]];
    }
    next[m] = nhd0[m - rmin[h[m - 1]]];

    return nd; // Trả về chỉ số nd phân tách tập
}

void GG(const std::string& x, int m, const std::string& y, int n) {
    int i, j, k, ell, last, nd;
    std::vector<int> h(XSIZE, 0);
    std::vector<int> next(XSIZE, 0);
    std::vector<int> shift(XSIZE, 0);
    int heavy;

    for (ell = 0; ell < m - 1 && x[ell] == x[ell + 1]; ell++);

    if (ell == m - 1) {
        for (j = ell = 0; j < n; ++j) {
            if (x[0] == y[j]) {
                ++ell;
                if (ell >= m) {
                    std::cout << "Tim thay mau tai vi tri: " << (j - m +
1) << std::endl;
                }
            } else {
                ell = 0;
            }
        }
    } else {
        // Gọi hàm preColussi đầy đủ vừa dựng ở trên
        nd = preColussi(x, m, h, next, shift);

        i = 0;
        j = 0;
        heavy = 0;
        last = -1;

        while (j <= n - m) {
            if (heavy && i == 0) {
                k = last - j + 1;
                while (k < m && x[0] == y[j + k]) {

```

```

        k++;
    }
    if (k <= ell || x[ell + 1] != y[j + k]) {
        i = 0;
        j += (k + 1);
        last = j - 1;
    } else {
        i = 1;
        last = j + k;
        j = last - (ell + 1);
    }
    heavy = 0;
} else {
    while (i < m && last < j + h[i] && x[h[i]] == y[j + h[i]]) {
        ++i;
    }
    if (i >= m || last >= j + h[i]) {
        std::cout << "Tìm thay mau tai vi tri: " << j;
        i = m;
    }
    if (i > nd) {
        last = j + m - 1;
    }
    j += shift[i];
    i = next[i];
}
heavy = (j > last ? 0 : 1);
}
}
}

```

4.2. Thuật toán Two way

4.2.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Two-Way là "phân rã xâu mẫu thành hai phần biệt lập xl (trái) và xr (phải) dựa trên điểm cắt tới hạn (critical factorization) nhằm tối ưu hóa trật tự đối sánh: duyệt phần bên phải trước từ trái sang phải, sau đó mới duyệt ngược phần bên trái từ phải sang trái". Thuật toán tận dụng tính tuần hoàn cục bộ (local period) tại điểm cắt tới hạn để đạt được bước nhảy cóc an toàn và cực xa.

Pha tiền xử lý thực hiện tìm điểm cắt tới hạn bằng cách tính toán hậu tố lớn nhất (maximal suffix) của xâu mẫu dựa trên hai trật tự sắp xếp ngược nhau (thông qua hàm maxSuf và maxSufTilde). Điểm phân tách ell được chọn làm mốc chia xâu mẫu thành $xl = X[0..ell]$ và $xr = X[ell+1..m-1]$. Khi tiến hành tìm kiếm, nếu quá trình quét xr bên phải xảy ra lỗi ở ký tự thứ k, cửa sổ dịch chuyển lên k vị trí. Nếu xr khớp hoàn toàn nhưng phần bên trái xl bị lỗi (hoặc khi tìm thấy mẫu), thuật toán sẽ dịch cửa sổ một khoảng bằng chu kỳ tới hạn per. Đặc biệt, để tránh bị suy biến, thuật toán tích hợp một biến nhớ memory ghi lại đoạn tiền tố đã khớp của cửa sổ trước để nhảy cóc qua, không cần so sánh lại. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Two-Way được triển khai tuần tự qua 2 bước sau:

Thuật toán PreTwoWay: // Tìm điểm cắt tới hạn ell và chu kỳ per của xâu mẫu

Input: * Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.

Output: Vị trí phân tách ell và độ dài chu kỳ per.

Formats: PreTwoWay(X, m, ell, per);

Actions:

```
i = maxSuf(X, m, p);    // Tính hậu tố lớn nhất theo trật tự thuận
j = maxSufTilde(X, m, q); // Tính hậu tố lớn nhất theo trật tự nghịch
```

```
if (i > j) {
    ell = i;
    per = p;
} else {
    ell = j;
    per = q;
}
```

EndActions.**Thuật toán TwoWay: Input:**

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .**Formats:** TwoWay(X, m, Y, n);**Actions: Bước 1 (Tiền xử lý):**

```
PreTwoWay(X, m, ell, per);
```

Bước 2 (Lặp tìm kiếm):

```
if (<Đoạn  $X[0..per-1]$  trùng khớp hoàn toàn với  $X[per..ell]$ >) {
    // Nhánh tuần hoàn:  $per \leq (ell + 1) / 2$ 
    j = 0;
    memory = -1;
    while (j <= n - m) {
        i = MAX(ell, memory) + 1;
        while (i < m && X[i] == Y[i + j]) {
            ++i;
        }
        if (i >= m) {
            i = ell;
            while (i > memory && X[i] == Y[i + j]) {
                --i;
            }
            if (i <= memory) {
                <Tìm thấy mẫu ở vị trí j>;
            }
            j = j + per;
            memory = m - per - 1;
        } else {
            j = j + (i - ell);
            memory = -1;
        }
    }
} else {
```

```

// Nhánh không tuần hoàn: per > (ell + 1) / 2
per = MAX(ell + 1, m - ell - 1) + 1;
j = 0;
while (j <= n - m) {
    i = ell + 1;
    while (i < m && X[i] == Y[i + j]) {
        ++i;
    }
    if (i >= m) {
        i = ell;
        while (i >= 0 && X[i] == Y[i + j]) {
            --i;
        }
        if (i < 0) {
            <Tìm thấy mẫu ở vị trí j>;
        }
        j = j + per;
    } else {
        j = j + (i - ell);
    }
}
}
EndActions.

```

4.2.2. Đặc điểm thuật toán

- Tiến trình so khớp ký tự thực hiện qua hai giai đoạn ngược chiều nhau: Duyệt phần xr bên phải xuôi chiều (từ trái sang phải) rồi mới duyệt phần xl bên trái ngược chiều (từ phải sang trái). Cửa sổ dịch tuần tiến từ trái sang phải trên văn bản y.
- Pha tiền xử lý vận hành gọn nhẹ với độ phức tạp thời gian là $O(m)$ và chỉ tốn không gian bộ nhớ hằng số $O(1)$ do không cần dựng các bảng ma trận lớn.
- Pha tìm kiếm đạt hiệu năng tuyến tính tối ưu tuyệt đối trong mọi tình huống với độ phức tạp thời gian trường hợp xấu nhất là $O(n)$.
- Số lượng phép so sánh ký tự trên văn bản trong trường hợp xấu nhất được chặn trên nghiêm ngặt ở mức $2n - m$ phép toán, đảm bảo tốc độ lướt dữ liệu cực kỳ ổn định.

4.2.3. Kiểm nghiệm thuật toán

Kiểm nghiệm TwoWay(x, m, y, n):

- **x chuỗi mẫu:** "GCAGAGAG" ($m = 8$),
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" ($n = 24$).

Bước 1 (Tiền xử lý): Chạy hàm PreTwoWay xác định điểm cắt tối hạn:

- Điểm phân tách: $ell = 1$. Chuỗi mẫu được phân rã thành $xl = "GC"$, $xr = "AGAGAG"$.
- Chu kỳ tối hạn: $per = 7$. Thuộc nhánh không tuần hoàn (nhánh 2). Đặt lại giá trị dịch chuyển: $per = \max(1 + 1, 8 - 1 - 1) + 1 = \max(2, 6) + 1 = 7$.

Sơ đồ chặng dịch chuyển tìm kiếm:

Lượt quét	Vị trí j	Tiến trình kiểm tra trên cửa sổ văn bản	Chỉ số i dừng	Khoảng cách dịch chuyển cửa sổ kế tiếp
1	0	Giai đoạn 1 (xr xuôi từ i=2 đến 7): i=2: x[2]='A' == y[2]='A' (Khớp). i=3: x[3]='G' == y[3]='G' (Khớp). i=4: x[4]='A' != y[4]='T' (Lệch).	4	Lệch ở phần xr bên phải. shift = i - ell = 4 - 1 = 3. Cửa sổ mới: j = 0 + 3 = 3.
2	3	Giai đoạn 1 (xr xuôi từ i=2): i=2: x[2]='A' != y[5]='G' (Lệch).	2	Lệch ở phần xr bên phải. shift = i - ell = 2 - 1 = 1. Cửa sổ mới: j = 3 + 1 = 4.
3	4	Giai đoạn 1 (xr xuôi): Khớp hoàn toàn đoạn "AGAGAG". Giai đoạn 2 (xl ngược từ i=1 lùi về 0): Khớp hoàn toàn đoạn "GC".	-1	Tìm thấy mẫu tại vị trí j = 4. Do khớp hoàn toàn, dịch theo chu kỳ tới hạn: shift = per = 7. Cửa sổ mới: j = 4 + 7 = 11.
Kết thúc	11	Vòng lặp kết thúc vì vị trí cửa sổ kế tiếp vượt quá giới hạn (n - m = 16).		

4.2.3. Cài đặt thuật toán

```
// Hàm tính hậu tố lớn nhất của xâu mẫu theo trật tự sắp xếp thuận <=
int maxSuf(const std::string& x, int m, int& p) {
    int ms = -1;
    int j = 0;
    int k = p = 1;
    char a, b;

    while (j + k < m) {
        a = x[j + k];
        b = x[ms + k];
        if (a < b) {
            j += k;
            k = 1;
            p = j - ms;
        } else if (a == b) {
            if (k != p) {
                ++k;
            } else {
                j += p;
                k = 1;
            }
        } else { // a > b
            ms = j;
            j = ms + 1;
            k = p = 1;
        }
    }
}
```



```

    }
}
return ms;
}

// Hàm tính hậu tố lớn nhất của xâu mẫu theo trật tự sắp xếp nghịch >=
int maxSufTilde(const std::string& x, int m, int& q) {
    int ms = -1;
    int j = 0;
    int k = q = 1;
    char a, b;

    while (j + k < m) {
        a = x[j + k];
        b = x[ms + k];
        if (a > b) {
            j += k;
            k = 1;
            q = j - ms;
        } else if (a == b) {
            if (k != q) {
                ++k;
            } else {
                j += q;
                k = 1;
            }
        } else { // a < b
            ms = j;
            j = ms + 1;
            k = q = 1;
        }
    }
    return ms;
}

/**
 * Hàm tìm kiếm chuỗi chính sử dụng giải thuật Two-Way
 */
void TW(const std::string& x, int m, const std::string& y, int n) {
    int i, j, ell, memory, p, per, q;

    /* Bước 1: Tiền xử lý tìm điểm phân tách ell và chu kỳ per */
    p = 1;
    i = maxSuf(x, m, p);
    q = 1;
    j = maxSufTilde(x, m, q);

    if (i > j) {
        ell = i;
        per = p;
    } else {
        ell = j;
        per = q;
    }

    /* Bước 2: Tiến trình tìm kiếm đối sánh trong vòng lặp */
    if (std::memcmp(x.c_str(), x.c_str() + per, ell + 1) == 0) {
        // Nhánh 1: Xâu mẫu có tính tuần hoàn cục bộ
        j = 0;
        memory = -1;
        while (j <= n - m) {
            i = std::max(ell, memory) + 1;
            while (i < m && x[i] == y[i + j]) {

```

```

        ++i;
    }
    if (i >= m) {
        i = ell;
        while (i > memory && x[i] == y[i + j]) {
            --i;
        }
        if (i <= memory) {
            std::cout << "Tim thay mau tai vi tri: " << j <<
std::endl;
        }
        j += per;
        memory = m - per - 1;
    } else {
        j += (i - ell);
        memory = -1;
    }
}
} else {
    // Nhánh 2: Xâu mẫu không tuần hoàn cục bộ
    per = std::max(ell + 1, m - ell - 1) + 1;
    j = 0;
    while (j <= n - m) {
        i = ell + 1;
        while (i < m && x[i] == y[i + j]) {
            ++i;
        }
        if (i >= m) {
            i = ell;
            while (i >= 0 && x[i] == y[i + j]) {
                --i;
            }
            if (i < 0) {
                std::cout << "Tim thay mau tai vi tri: " << j <<
std::endl;
            }
            j += per;
        } else {
            j += (i - ell);
        }
    }
}
}
}

```

4.3. Thuật toán String Matching on Ordered Alphabets

4.3.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Tìm kiếm chuỗi trên bảng chữ cái được sắp thứ tự (SMOA) là "tận dụng tính chất thứ tự của các ký tự trong bảng chữ cái để thực hiện phân rã xâu mẫu theo cấu trúc hậu tố lớn nhất (Maximal-Suffix decomposition), từ đó tìm ra chu kỳ cục bộ mà không cần bất kỳ pha tiền xử lý dữ liệu nào". Thuật toán phân tích xâu mẫu x dưới dạng chuỗi lặp có cấu trúc $v = w^e w'$, trong đó w là từ cơ sở và w' là tiền tố của w .

Khi tiến hành tìm kiếm, thuật toán so khớp xâu mẫu với văn bản từ trái sang phải. Nếu xảy ra lệch khớp tại vị trí i , thuật toán không dùng bảng tra cứu tính sẵn mà gọi trực tiếp hàm `nextMaximalSuffix` để phân tích tính tuần hoàn của đoạn văn bản vừa khớp trên cửa sổ hiện tại $(y + j)$. Hàm này trả về các chỉ số trạng thái bao gồm ip , jp , k , p để tính

toán điểm cắt tối hạn tức thời. Dựa vào kết quả đối sánh vùng nhớ (memcmp), cửa sổ sẽ dịch chuyển một khoảng bằng bội số chu kỳ p hoặc nhảy cóc an toàn qua đoạn xung đột lớn nhất bằng hàm cực đại (MAX). Chính nhờ việc tính toán động này, SMOA loại bỏ hoàn toàn pha tiền xử lý giúp tiết kiệm bộ nhớ đệm tối đa. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán SMOA được triển khai tuần tự qua 2 bước sau:

Thuật toán nextMaximalSuffix: // Phân tích động hậu tố lớn nhất kế tiếp trên cửa sổ văn bản

Input:

- Xâu văn bản tại cửa sổ hiện thời ($y + j$), chỉ số mốc đoạn khớp $i + 1$.
- Các con trỏ địa chỉ biến lưu trạng thái: $\&ip$, $\&jp$, $\&k$, $\&p$.

Output: Cập nhật giá trị trực tiếp vào các biến trạng thái ip , jp , k , p .

Formats: nextMaximalSuffix($y + j$, $i + 1$, $\&ip$, $\&jp$, $\&k$, $\&p$);

Actions:

```
while (*jp + *k < m) {
    a = x[*i + *k];
    b = x[*j + *k];
    if (a == b) {
        if (*k == *p) {
            *jp = *jp + *p;
            *k = 1;
        } else {
            ++(*k);
        }
    } else {
        if (a > b) {
            *jp = *jp + *k;
            *k = 1;
            *p = *jp - *i;
        } else {
            *i = *jp;
            ++(*jp);
            *k = *p = 1;
        }
    }
}
```

EndActions.

Thuật toán SMOA:Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .

Formats: SMOA(X , m , Y , n);

Actions:

Bước 1 (Tiền xử lý):

// Thuật toán SMOA hoàn toàn không có pha tiền xử lý dữ liệu (No preprocessing phase)

Bước 2 (Lặp tìm kiếm):

Plaintext

ip = -1;

i = 0;

j = 0;

jp = 0;

k = 1;

p = 1;

```

while (j <= n - m) {
    while (i + j < n && i < m && X[i] == Y[i + j]) {
        ++i;
    }
    if (i == 0) {
        ++j;
        ip = -1;
        jp = 0;
        k = p = 1;
    } else {
        if (i >= m) {
            <Tìm thấy mẫu ở vị trí j>;
        }
        nextMaximalSuffix(Y + j, i + 1, &ip, &jp, &k, &p);

        if (ip < 0 || (ip < p && memcmp(Y + j, Y + j + p, ip + 1) == 0)) {
            j = j + p;
            i = i - p;
            if (i < 0) {
                i = 0;
            }
            if (jp - ip > p) {
                jp = jp - p;
            } else {
                ip = -1;
                jp = 0;
                k = p = 1;
            }
        } else {
            j = j + (MAX(ip + 1, MIN(i - ip - 1, jp + 1)) + 1);
            i = jp = 0;
            ip = -1;
            k = p = 1;
        }
    }
}
EndActions.

```

4.3.2. Đặc điểm thuật toán

- Tiến trình kiểm tra ký tự thực hiện tuần tiến từ trái sang phải trên văn bản y. Khi xảy ra sai lệch, thứ tự dịch chuyển được tính toán động dựa vào hàm phân rã chu kỳ.
- Không có pha tiền xử lý, do đó độ phức tạp thời gian tiền xử lý bằng 0 và không gian bộ nhớ phụ trợ đạt mức hằng số tối ưu $O(1)$.
- Pha tìm kiếm đạt hiệu năng tuyến tính với độ phức tạp thời gian trường hợp xấu nhất là $O(n)$.
- Số lượng phép so sánh ký tự trên văn bản trong trường hợp xấu nhất được chặn trên nghiêm ngặt ở mức $6n + 5$ phép toán, hoạt động rất hiệu quả trên các hệ thống có tài nguyên bộ nhớ bị hạn chế.

4.3.3. Kiểm nghiệm thuật toán

Kiểm nghiệm SMOA(x, m, y, n):

- **x mẫu:** "GCAGAGAG" (m = 8),
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" (n = 24).

Bước 1 (Tiền xử lý): Không thực hiện (Bỏ qua).

Bảng chặng dịch chuyển tìm kiếm:

Lượt quét	Vị trí j	Tiến trình đối sánh ký tự (Từ trái sang phải)	Kết quả gọi hàm phân rã chỉ số động	Khoảng cách dịch chuyển cửa sổ kế tiếp
1	0	- Khớp liên tục 4 ký tự đầu: x[0]=='G', x[1]=='C', x[2]=='A', x[3]=='G'. - Lệch tại vị trí thứ 5: x[4]='A' != y[4]='T'. Chỉ số dừng: i = 4.	- Gọi hàm nextMaximalSuffix: - Kết quả trả về các tham số: ip = 2, jp = 3, k = 1, p = 1. - Thực hiện 6 phép đối sánh ký tự bên trong hàm.	shift = 4. Cửa sổ mới tịnh tiến sang phải: j = 0 + 4 = 4.
2	4	- Bắt đầu vòng quét mới tại mốc j = 4. - Khớp liên tục cả 8 ký tự: "GCAGAGAG". Chỉ số đạt: i = 8.	Tìm thấy mẫu tại vị trí j = 4 Gọi hàm cập nhật chu kỳ động.	shift = 1. Cửa sổ mới tịnh tiến: j = 4 + 1 = 5.
3	5	- Khớp liên tục 8 ký tự tiếp theo từ y[5] đến y[12]: "GCAGAGAG". Chỉ số đạt: i = 8.	Tìm thấy mẫu tại vị trí j = 5 Tính toán bước nhảy chu kỳ tới hạn kế tiếp.	shift = 1. Vòng lặp dừng vì chặng kế tiếp vượt quá điều kiện biên (n - m = 16).

Kết luận: Qua tiến trình kiểm nghiệm thực tế từ slide, thuật toán SMOA tìm thấy mẫu xuất hiện hoàn chỉnh tại hai chỉ số **j = {4, 5}**.

4.3.4. Cài đặt thuật toán

```

// Hàm phân tích và tính toán động hậu tố lớn nhất kế tiếp trên văn bản
void nextMaximalSuffix(const char* y, int m, int* i, int* j, int* k, int* p)
{
    char a, b;
    // Vòng lặp quét tính toán chỉ số dựa trên mốc ký tự văn bản tại cửa sổ
    while (*j + *k < m) {
        a = y[*i + *k];
        b = y[*j + *k];
        if (a == b) {
            if (*k == *p) {
                *j += *p;
                *k = 1;
            } else {
                ++(*k);
            }
        } else {
            if (a > b) {
                *j += *k;
                *k = 1;
                *p = *j - *i;
            } else {
                *i = *j;
                ++(*j);
                *k = *p = 1;
            }
        }
    }
}

/**
 * Hàm tìm kiếm chuỗi chính theo giải thuật SMOA (Ordered Alphabets)
 */
void SMOA(const std::string& x, int m, const std::string& y, int n) {
    int i, ip, j, jp, k, p;
    const char* x_ptr = x.c_str();
    const char* y_ptr = y.c_str();

    /* Bước 1: Pha tiền xử lý (Không có cấu trúc tiền xử lý) */

    /* Bước 2: Vòng lặp điều khiển tiến trình tìm kiếm */
    ip = -1;
    i = j = jp = 0;
    k = p = 1;

    while (j <= n - m) {
        // So khớp ký tự thuận từ trái sang phải trong cửa sổ hiện tại
        while (i + j < n && i < m && x_ptr[i] == y_ptr[i + j]) {
            ++i;
        }

        if (i == 0) {
            ++j;
            ip = -1;
            jp = 0;
            k = p = 1;
        } else {
            if (i >= m) {
                std::cout << "Tim thay mau tai vi tri: " << j << std::endl;
            }

            // Gọi hàm tính toán chu kỳ động nextMaximalSuffix
            nextMaximalSuffix(y_ptr + j, i + 1, &ip, &jp, &k, &p);
        }
    }
}

```

```

// Điều kiện tính toán bước nhảy tịnh tiến của số j
if (ip < 0 || (ip < p && std::memcmp(y_ptr + j, y_ptr + j + p, ip
+ 1) == 0)) {
    j += p;
    i -= p;
    if (i < 0) {
        i = 0;
    }
    if (jp - ip > p) {
        jp -= p;
    } else {
        ip = -1;
        jp = 0;
        k = p = 1;
    }
} else {
    j += (std::max(ip + 1, std::min(i - ip - 1, jp + 1)) + 1);
    i = jp = 0;
    ip = -1;
    k = p = 1;
}
}
}
}
}

```

4.4. Thuật toán Optimal Mismatch algorithm

4.4.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Lệch khớp tối ưu (Optimal Mismatch) là "tối ưu hóa tốc độ loại bỏ cửa sổ bằng cách thay đổi thứ tự đối sánh ký tự bên trong mẫu: ưu tiên so sánh những ký tự có tần suất xuất hiện ít nhất trong ngôn ngữ trước, nhằm tăng tối đa tỷ lệ xảy ra lệch khớp (mismatch) ngay từ những phép toán đầu tiên". Ý tưởng này dựa trên thiết kế cải tiến của tác giả Sunday.

Pha tiền xử lý của thuật toán thực hiện thống kê mảng tần suất freq[] của các ký tự trong bảng chữ cái, sau đó tiến hành sắp xếp lại các ký tự của xâu mẫu x theo thứ tự tần suất xuất hiện giảm dần (từ ít gặp nhất đến hay gặp nhất) và lưu vào mảng cấu trúc pat[]. Tiếp theo, thuật toán xây dựng song song hai bảng dịch chuyển: bảng ký tự xâu qsBc[] của Quick Search và bảng khớp tốt cải tiến adaptedGs[] được điều chỉnh tương thích với thứ tự quét mới của xâu mẫu. Khi tìm kiếm, thuật toán quét văn bản từ trái sang phải, tại mỗi cửa sổ j sẽ đối sánh ký tự dựa theo vị trí gốc được lưu trong pat[i].loc. Nếu phát hiện sai lệch hoặc khi khớp hoàn toàn, thuật toán sẽ lấy giá trị cực đại (MAX) giữa hai bảng dịch chuyển adaptedGs[i] và qsBc[Y[j + m]] để nhảy chặng. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Lệch khớp tối ưu được triển khai tuần tự qua 2 bước sau:

Thuật toán PreOM: // Thực hiện tiền xử lý sắp xếp mẫu và dựng các bảng dịch chuyển thích ứng

Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.
- Kích thước bảng chữ cái hệ thống ASIZE. Mảng thống kê tần suất ký tự freq[ASIZE].

Output: Mảng cấu trúc sắp thứ tự pat[], mảng qsBc[] và mảng adaptedGs[].

Formats: PreOM(X, m, pat, qsBc, adaptedGs);

Actions:

```
// 1. Khởi tạo và sắp xếp cấu trúc pat dựa vào hàm so sánh tần suất optimalPcmp
for (i = 0; i <= m; ++i) {
    pat[i].loc = i;
    pat[i].c = X[i];
}
qsort(pat, m, sizeof(pattern), optimalPcmp);

// 2. Gọi hàm tiền xử lý ký tự xấu của Quick Search
PreQsBc(X, m, qsBc);

// 3. Gọi hàm dựng bảng khớp tốt thích ứng
preAdaptedGs(X, m, adaptedGs, pat);
EndActions.
```

Thuật toán OptimalMismatch: Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y .**Formats:** OptimalMismatch(X, m, Y, n);**Actions:****Bước 1 (Tiền xử lý):**PreOM($X, m, pat, qsBc, adaptedGs$);**Bước 2 (Lặp tìm kiếm):**

Plaintext

 $j = 0;$ while ($j \leq n - m$) { $i = 0;$ while ($i < m \ \&\& \ pat[i].c == Y[j + pat[i].loc]$) { $++i;$

}

 if ($i \geq m$) { <Tìm thấy mẫu ở vị trí j >;

}

```
// Tra cứu và dịch chuyển dựa trên giá trị cực đại (MAX) giữa hai bảng tra cứu
j = j + MAX(adaptedGs[i], qsBc[Y[j + m]]);
```

}

EndActions.

4.4.2. Đặc điểm thuật toán

- Thứ tự đối sánh ký tự bên trong cửa sổ tuân thủ nghiêm ngặt theo trật tự tần suất tăng dần được lưu trong cấu trúc $pat[i].loc$, cửa sổ dịch chuyển tuần tiến từ trái sang phải trên văn bản y .
- Pha tiền xử lý đòi hỏi thao tác sắp xếp mảng và tính toán bảng nhảy thích ứng. Độ

phức tạp thời gian tiền xử lý là $O(m^2 + \sigma)$ và độ phức tạp không gian bộ nhớ lưu trữ là $O(m + \sigma)$, với σ là kích thước bảng chữ cái.

- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$. Tuy nhiên, nhờ cơ chế lọc thông minh dựa vào tần suất ký tự, số phép đối sánh ký tự trung bình thực tế được giảm thiểu tối đa, giúp thuật toán vận hành với tốc độ vượt trội trên các văn bản tự nhiên có sự phân hóa tần suất ký tự rõ rệt (như văn bản tiếng Anh).

4.4.3 Kiểm nghiệm thuật toán

Kiểm nghiệm OM(x, m, y, n):

- **x chuỗi mẫu:** "GCAGAGAG" (m = 8),
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" (n = 24).

Bước 1 (Tiền xử lý): Kết quả tính toán mảng cấu trúc sắp xếp pat, bảng nhảy ký tự xâu qsBc

- Mảng tần suất ngôn ngữ thực tế: $\text{freq}['A'] = 8$, $\text{freq}['C'] = 5$, $\text{freq}['G'] = 7$, $\text{freq}['T'] = 4$
- Bảng Quick Search: $\text{qsBc}['A'] = 2$, $\text{qsBc}['C'] = 7$, $\text{qsBc}['G'] = 1$, $\text{qsBc}['T'] = 9$
- Trật tự mảng pat sau sắp xếp (tần suất giảm dần):
 - o $\text{pat}[0] = \{\text{loc}: 1, \text{c}: 'C'\}$, $\text{pat}[1] = \{\text{loc}: 7, \text{c}: 'G'\}$, $\text{pat}[2] = \{\text{loc}: 5, \text{c}: 'G'\}$, $\text{pat}[3] = \{\text{loc}: 3, \text{c}: 'G'\}$, $\text{pat}[4] = \{\text{loc}: 0, \text{c}: 'G'\}$, $\text{pat}[5] = \{\text{loc}: 6, \text{c}: 'A'\}$, $\text{pat}[6] = \{\text{loc}: 4, \text{c}: 'A'\}$, $\text{pat}[7] = \{\text{loc}: 2, \text{c}: 'A'\}$
- Bảng thích ứng adaptedGs = {1, 3, 4, 2, 7, 7, 7, 7, 7}

Bảng chặng dịch chuyển tìm kiếm:

Lượt quét	Vị trí j	Tiến trình đối sánh ký tự dựa theo mảng pat	Chỉ số i dừng	Công thức và khoảng cách dịch chuyển kế tiếp
1	0	$i=0: \text{pat}[0].\text{c}='C' == y[0+1]='C'$ (Khớp). $i=1: \text{pat}[1].\text{c}='G' != y[0+7]='A'$ (Lệch).	1	Lệch tại chỉ số $i = 1$, ký tự sau cửa sổ là $y[8] = 'G'$. $\text{shift} = \max(\text{adaptedGs}[1], \text{qsBc}['G']) = \max(3, 1) = 3$. Cửa sổ mới: $j = 0 + 3 = 3$.
2	3	$i=0: \text{pat}[0].\text{c}='C' == y[3+1]='C'$ (Khớp). $i=1: \text{pat}[1].\text{c}='G' != y[3+7]='A'$ (Lệch tại $y[10]$). $i=2: \text{pat}[2].\text{c}='G' == y[3+5]='G'$ (Khớp). $i=3: \text{pat}[3].\text{c}='G' == y[3+3]='G'$ (Khớp).	1	Lệch tại chỉ số $i = 1$, ký tự sau cửa sổ là $y[11] = 'A'$. $\text{shift} = \max(\text{adaptedGs}[1], \text{qsBc}['A']) = \max(3, 2) = 3$. Cửa sổ mới: $j = 3 + 3 = 6$.

Kết thúc	6	Vòng lặp dừng vì vượt quá điều kiện biên an toàn ($n - m = 16$).		
---------------------	---	--	--	--

4.4.4. Cài đặt thuật toán

```
// Cấu trúc lưu trữ thông tin trật tự quét của ký tự trong mẫu x
struct patternScanOrder {
    int loc;
    char c;
};

// Mảng toàn cục lưu trữ tần suất xuất hiện của ký tự
int freq[ASIZE];

// Hàm tiền xử lý ký tự xấu phụ trợ của Quick Search
void PreQsBc(const std::string& x, int m, std::vector<int>& qsBc) {
    for (int i = 0; i < ASIZE; ++i) {
        qsBc[i] = m + 1;
    }
    for (int i = 0; i < m; ++i) {
        qsBc[(unsigned char)x[i]] = m - i;
    }
}

// Hàm so sánh dùng cho thuật toán sắp xếp nhanh qsort
int optimalPcmp(const void* pat1, const void* pat2) {
    float fx;
    patternScanOrder* p1 = (patternScanOrder*)pat1;
    patternScanOrder* p2 = (patternScanOrder*)pat2;

    fx = freq[(unsigned char)p1->c] - freq[(unsigned char)p2->c];
    if (fx != 0) {
        return (fx > 0 ? 1 : -1);
    }
    return (p2->loc - p1->loc);
}

// Hàm tính toán độ dịch chuyển khớp trái tiếp theo
int matchShift(const std::string& x, int m, int ploc, int lshift, const
std::vector<patternScanOrder>& pat) {
    int i, j;
    for (; lshift < m; ++lshift) {
        i = ploc;
        while (--i >= 0) {
            j = pat[i].loc - lshift;
            if (j < 0) continue;
            if (pat[i].c != x[j]) break;
        }
        if (i < 0) break;
    }
    return lshift;
}

// Hàm xây dựng bảng dịch chuyển khớp tốt thích ứng
void preAdaptedGs(const std::string& x, int m, std::vector<int>& adaptedGs,
const std::vector<patternScanOrder>& pat) {
    int lshift, i, ploc;

    adaptedGs[0] = lshift = 1;
```

```

    for (ploc = 1; ploc <= m; ++ploc) {
        lshift = matchShift(x, m, ploc, lshift, pat);
        adaptedGs[ploc] = lshift;
    }

    for (ploc = 0; ploc <= m; ++ploc) {
        lshift = adaptedGs[ploc];
        while (lshift < m) {
            i = pat[ploc].loc - lshift;
            if (i < 0 || pat[ploc].c != x[i]) break;
            ++lshift;
            lshift = matchShift(x, m, ploc, lshift, pat);
        }
        adaptedGs[ploc] = lshift;
    }
}

/**
 * Hàm tìm kiếm chính của giải thuật Optimal Mismatch (OM)
 */
void OM(const std::string& x, int m, const std::string& y, int n) {
    std::vector<int> adaptedGs(XSIZE);
    std::vector<int> qsBc(ASIZE);
    std::vector<patternScanOrder> pat(XSIZE);

    // Khởi tạo mảng cấu trúc pat phục vụ cho việc sắp xếp trật tự tần suất
    for (int i = 0; i <= m; ++i) {
        pat[i].loc = i;
        pat[i].c = x[i];
    }

    /* Bước 1: Pha tiền xử lý đồng bộ dữ liệu */
    std::qsort(pat.data(), m, sizeof(patternScanOrder), optimalPcmp);
    PreQsBc(x, m, qsBc);
    preAdaptedGs(x, m, adaptedGs, pat);

    /* Bước 2: Vòng lặp tìm kiếm chặng dịch chuyển trên văn bản y */
    int j = 0;
    int i;
    while (j <= n - m) {
        i = 0;
        // So khớp ký tự dựa theo chỉ số vị trí thực tế được sắp đặt trong
        // cấu trúc dữ liệu pat
        while (i < m && pat[i].c == y[j + pat[i].loc]) {
            ++i;
        }
        if (i >= m) {
            std::cout << "Tìm thay mau tai vi tri: " << j << std::endl;
        }

        // Tiến hành dịch chuyển mốc j dựa trên giá trị cực đại giữa hai bảng
        // tra cứu
        j += std::max(adaptedGs[i], qsBc[(unsigned char)y[j + m]]);
    }
}

```

CHƯƠNG 5. THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ

5.1. Thuật toán Skip Search

5.1.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Tìm kiếm nhảy cóc (Skip Search) là "sử dụng các danh sách liên kết lưu trữ toàn bộ vị trí xuất hiện của từng ký tự trong chuỗi mẫu x nhằm quyết định các điểm khởi đầu đối sánh, giúp thuật toán có thể nhảy cóc qua phần lớn văn bản mà không cần quét tuần tiến từng ô".

Pha tiền xử lý xây dựng một mảng bảng nhảy $z[ASIZE]$, trong đó mỗi phần tử $z[c]$ quản lý một danh sách liên kết (bucket) chứa tất cả các chỉ số i mà tại đó ký tự $x[i]$ bằng c . Khi tìm kiếm, thuật toán không duyệt văn bản theo từng cửa sổ tịnh tiến đơn vị mà thực hiện nhảy cóc: nó chỉ xét các ký tự tại vị trí cách nhau đúng bằng độ dài m của chuỗi mẫu (tức là duyệt các mốc $y[j]$ với $j = m - 1, 2m - 1, 3m - 1, \dots$). Tại mỗi ký tự mục tiêu $y[j]$, thuật toán tra cứu danh sách liên kết $z[y[j]]$. Với mỗi vị trí xuất hiện i của ký tự đó trong mẫu, thuật toán lập tức xác định được một vị trí xuất phát tiềm năng của mẫu trên văn bản là $(j - i)$. Sau đó, một lệnh so khớp vùng nhớ (memcmp) sẽ được kích hoạt để kiểm tra xem toàn bộ chuỗi mẫu có khớp tại $(j - i)$ hay không. Cơ chế nhảy chặng m ô này mang lại hiệu năng lướt dữ liệu cực kỳ ấn tượng. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán Skip Search được triển khai tuần tự qua 2 bước sau:

Thuật toán PreSkip: // Khởi tạo và xây dựng các danh sách liên kết vị trí ký tự cho mẫu

Input:

- Chuỗi mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Kích thước bảng chữ cái hệ thống $ASIZE$.

Output: Mảng bảng nhảy chứa danh sách liên kết $z[ASIZE]$.

Formats: PreSkip(X, m, z);

Actions:

$i = 0$;

while ($i < ASIZE$) {

$z[i] = \text{NULL}$; // Khởi tạo tất cả các bucket ban đầu bằng rỗng

$i++$;

}

$i = 0$;

while ($i < m$) {

$\text{ptr} = \text{<Cấp phát vùng nhớ cho một nút danh sách liên kết cell>}$;

$\text{ptr} \rightarrow \text{element} = i$;

$\text{ptr} \rightarrow \text{next} = z[X[i]]$;

$z[X[i]] = \text{ptr}$; // Thêm vị trí i vào đầu danh sách liên kết của ký tự $X[i]$

$i++$;

}

EndActions.

Thuật toán SkipSearch: Input:

- Chuỗi mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output: Tất cả vị trí xuất hiện X trong Y.

Formats: SkipSearch(X, m, Y, n);

Actions:

Bước 1 (Tiền xử lý):

PreSkip(X, m, z);

Bước 2 (Lặp tìm kiếm):

Plaintext

j = m - 1;

while (j < n) {

ptr = z[Y[j]]; // Tra cứu danh sách vị trí dựa trên ký tự Y[j] hiện tại

while (ptr != NULL) {

start_pos = j - ptr->element;

// Tiến hành đối sánh toàn bộ xâu mẫu tại vị trí bắt đầu start_pos

if (memcmp(X, Y + start_pos, m) == 0) {

<Tìm thấy mẫu ở vị trí start_pos>;

}

ptr = ptr->next;

}

j = j + m; // Nhảy cóc cửa sổ một khoảng đúng bằng độ dài mẫu m

}

EndActions.

5.1.2. Đặc điểm thuật toán

- Tiến trình so khớp ký tự trên văn bản được thực hiện nhảy cóc không tuần tự. Thuật toán chỉ lấy mẫu ký tự tại các điểm mốc cách nhau m đơn vị, sau đó so khớp toàn cục tại các vị trí hợp lệ dựa vào danh sách liên kết.
- Pha tiền xử lý vận hành nhanh chóng với độ phức tạp thời gian là $O(m + \sigma)$ và không gian bộ nhớ lưu trữ các danh sách liên kết phụ thuộc trực tiếp vào kích thước bảng chữ cái là $O(m + \sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$. Tuy nhiên, số lượng phép kiểm tra ký tự văn bản kỳ vọng trong thực tế (expected number of inspections) chỉ ở mức tuyến tính tối ưu là $O(n)$, giúp thuật toán chạy rất nhanh trên các chuỗi văn bản dài.

5.1.3. Kiểm nghiệm thuật toán

Kiểm nghiệm SkipSearch(x, m, y, n):

- **x xâu mẫu:** "GCAGAGAG" (m = 8),

- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" (n = 24).

Bước 1 (Tiền xử lý): Xây dựng các danh sách liên kết z[c] lưu chỉ số vị trí xuất hiện của các ký tự có trong mẫu (tra cứu trực tiếp từ bảng hình ảnh số 4 - image_e4b162.png):

- z['A'] = (6, 4, 2) (Ký tự 'A' nằm ở chỉ số 2, 4, 6 trong mẫu)

- z['C'] = (1) (Ký tự 'C' nằm ở chỉ số 1 trong mẫu)

- z['G'] = (7, 5, 3, 0) (Ký tự 'G' nằm ở chỉ số 0, 3, 5, 7 trong mẫu)

- Ký tự 'T' không nằm trong mẫu nên danh sách liên kết tương ứng bằng rỗng

(NULL).

Bảng chặng dịch chuyển tìm kiếm:

Lượt quét	Chỉ số mốc j	Ký tự kiểm tra Y[j]	Danh sách chỉ số tra cứu z[Y[j]]	Tiến trình tính mốc xuất phát và so khớp toàn cục
1	7	y[7] = 'A'	(6, 4, 2)	* Thử với i = 6: start_pos = 7 - 6 = 1. So khớp mẫu tại y[1]: Không khớp. * Thử với i = 4: start_pos = 7 - 4 = 3. So khớp mẫu tại y[3]: Không khớp. * Thử với i = 2: start_pos = 7 - 2 = 5. So khớp mẫu tại y[5]: Khớp đoạn "GCAGAGAG". Tìm thấy mẫu tại vị trí j = 5.
2	15	y[15] = 'T'	NULL (Rỗng)	Ký tự 'T' không nằm trong mẫu. Thuật toán bỏ qua luôn, không thực hiện bất kỳ phép so sánh chuỗi nào.
Dừng	23			j tăng thêm m: j = 15 + 8 = 23. Vòng lặp dừng vì chặng kế tiếp vượt quá độ dài văn bản n.

5.1.4. Cài đặt thuật toán

```
// Định nghĩa cấu trúc một nút trong danh sách liên kết (Bucket)
struct _cell {
    int element;
    struct _cell* next;
};
typedef struct _cell* List;

void SKIP(const std::string& x, int m, const std::string& y, int n) {
    int i, j;
    List ptr;
    List z[ASIZE];

    /* Bước 1: Pha tiền xử lý cấu trúc dữ liệu xâu mẫu */
    std::memset(z, 0, ASIZE * sizeof(List)); // Gán toàn bộ mảng con trỏ bằng
    NULL

    for (i = 0; i < m; ++i) {
        ptr = (List)std::malloc(sizeof(struct _cell));
        if (ptr == nullptr) {
            std::cerr << "Loi cap phat bo nho SKIP" << std::endl;
            return;
        }
        ptr->element = i;
```

```

        ptr->next = z[(unsigned char)x[i]];
        z[(unsigned char)x[i]] = ptr;
    }

    /* Bước 2: Vòng lặp tìm kiếm nhảy chặng m ô trên văn bản y */
    const char* x_ptr = x.c_str();
    const char* y_ptr = y.c_str();

    for (j = m - 1; j < n; j += m) {
        // Duyệt danh sách liên kết lưu các vị trí xuất hiện của ký tự y[j]
        for (ptr = z[(unsigned char)y_ptr[j]]; ptr != nullptr; ptr = ptr-
>next) {

            // So khớp toàn bộ vùng nhớ mẫu tại vị trí khởi đầu (j - ptr-
>element)

            if (std::memcmp(x_ptr, y_ptr + j - ptr->element, m) == 0) {
                std::cout << "Tìm thấy mẫu tại vị trí: " << (j - ptr-
>element) << std::endl;
            }
        }
    }

    // Giải phóng bộ nhớ của các danh sách liên kết sau khi hoàn thành tìm
kiếm
    for (i = 0; i < ASIZE; ++i) {
        List curr = z[i];
        while (curr != nullptr) {
            List next_node = curr->next;
            std::free(curr);
            curr = next_node;
        }
    }
}

```

5.2. Thuật toán Kmp Skip Search

5.2.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán KmpSkip Search là "tuyến tính hóa thuật toán Skip Search bằng cách tích hợp hai bảng dịch chuyển biên của Morris-Pratt (mpNext) và Knuth-Morris-Pratt (kmpNext) cùng kỹ thuật ghi nhớ đường biên văn bản (wall) để triệt tiêu toàn bộ các phép so sánh trùng lặp". Thuật toán Skip Search nguyên bản có độ phức tạp trường hợp xấu nhất là dạng bình phương $O(m \times n)$. KmpSkip khắc phục điều này bằng cách duy trì một con trỏ mang tên wall ghi lại vị trí xa nhất trên văn bản đã được kiểm tra tính đến thời điểm hiện tại.

Pha tiền xử lý thực hiện song song việc tính toán mảng mpNext[], mảng kmpNext[] của nhóm thuật toán KMP, đồng thời thiết lập mảng z[] và list[] để quản lý danh sách vị trí ký tự dưới dạng mảng một chiều tĩnh liên kết thay vì dùng cấu trúc struct động nhằm tăng tốc độ truy cập. Khi tìm kiếm, thuật toán thực hiện nhảy cóc m ô để lấy ký tự mốc y[j]. Khi xác định được mốc xuất phát tiềm năng $start = j - i$, thay vì gọi hàm memcmp để so khớp lại từ đầu, thuật toán gọi hàm attempt phụ trợ để bắt đầu đối sánh các ký tự nằm phía sau đường biên wall (tức là từ vị trí $k = wall - start$ trở đi). Nhờ sự kết hợp này, KmpSkip Search không chế thời gian tìm kiếm ở mức tuyến tính nghiêm ngặt. Để làm được điều này, thuật toán chia làm 2 bước lớn.

Thuật toán KmpSkip Search được triển khai tuần tự qua 2 bước sau:

Thuật toán attempt: // Hàm phụ trợ thực hiện đối sánh ký tự từ mốc đường biên wall trở đi
Input:

Xâu văn bản Y, mẫu X, độ dài mẫu m.

Mốc vị trí khởi đầu của số start và mốc đường biên an toàn wall.

Output: Vị trí chỉ số k trên mẫu nơi xảy ra lệch khớp hoặc đạt độ dài m.

Formats: attempt(Y, X, m, start, wall);

Actions:

$k = wall - start$;

```
while (k < m && X[k] == Y[k + start]) {  
    ++k;  
}
```

return (k);

EndActions.

Thuật toán KmpSkipSearch:

Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.

- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n.

Output: Tất cả vị trí xuất hiện X trong Y.

Formats: KmpSkipSearch(X, m, Y, n);

Actions: B

ước 1 (Tiền xử lý):

preMp(X, m, mpNext);

preKmp(X, m, kmpNext);

// Khởi tạo cấu trúc mảng liên kết tĩnh z và list thay cho con trỏ động

i = 0;


```

while (i < ASIZE) {
    z[i] = -1;
    i++;
}
for (i = 0; i < m; ++i) {
    list[i] = -1;
}

z[X[0]] = 0;
i = 1;
while (i < m) {
    list[i] = z[X[i]];
    z[X[i]] = i;
    i++;
}
Bước 2 (Lặp tìm kiếm):
wall = 0;
per = m - kmpNext[m];
i = -1;
j = -1;

do {
    j = j + m;
} while (j < n && z[Y[j]] < 0);

if (j >= n) {
    return;
}

i = z[Y[j]];
start = j - i;

while (start <= n - m) {
    if (start > wall) {
        wall = start;
    }

    // Gọi hàm attempt phụ trợ so khớp từ mốc wall trở đi
    k = attempt(Y, X, m, start, wall);
    wall = start + k;

    if (k == m) {
        <Tìm thấy mẫu ở vị trí start>;
        i = i - per;
    } else {
        kmpStart = start + k - kmpNext[k];
        k = kmpNext[k];
        start = j - i;
    }
}

```

```

while (start < kmpStart || (kmpStart < start && start < wall)) {
    if (start < kmpStart) {
        i = list[i];
        if (i < 0) {
            do {
                j = j + m;
            } while (j < n && z[Y[j]] < 0);
            if (j >= n) {
                return;
            }
            i = z[Y[j]];
        }
        start = j - i;
    } else {
        kmpStart = kmpStart + (k - mpNext[k]);
        k = mpNext[k];
    }
}
}
}
EndActions.

```

5.2.2. Đặc điểm thuật toán

- Quá trình đối sánh ký tự kết hợp nhảy cóc m ô dựa theo danh sách liên kết tĩnh của mảng z, đồng thời áp dụng cơ chế dịch chuyển thông minh của KMP để bỏ qua các ký tự nằm trước đường biên wall. Cửa sổ dịch chuyển tuần tiến từ trái sang phải trên văn bản.
- Pha tiền xử lý yêu cầu tính toán đồng thời 4 cấu trúc mảng hỗ trợ (mpNext, kmpNext, z, list). Độ phức tạp thời gian tiền xử lý là $O(m + \sigma)$ và độ phức tạp không gian bộ nhớ là $O(m + \sigma)$, với σ là kích thước bảng chữ cái.
- Pha tìm kiếm đạt hiệu năng tối ưu lý thuyết tuyệt đối với độ phức tạp thời gian trong trường hợp xấu nhất là tuyến tính $O(n)$, khắc phục hoàn toàn điểm yếu suy biến của thuật toán Skip Search gốc.

5.2.3. Kiểm nghiệm thuật toán

Kiểm nghiệm KmpSkipSearch(x, m, y, n):

- **x chuỗi mẫu:** "GCAGAGAG" (m = 8),
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" (n = 24).

Bước 1 (Tiền xử lý): Tạo bảng nhảy dựa trên các mảng tĩnh của hình ảnh 6 (image_e44180.png):

- $z['A'] = 6, z['C'] = 1, z['G'] = 7, z['T'] = -1$
- Mảng liên kết ngược: list = {-1, -1, -1, 0, 2, 3, 4, 5}
- Mảng mpNext = {-1, 0, 0, 0, 1, 0, 1, 0, 1}
- Mảng kmpNext = {-1, 0, 0, -1, 1, -1, 1, -1, 1}

Sơ đồ chặng dịch chuyển tìm kiếm:

Lượt quét	Mốc chỉ số j	Ký tự kiểm tra Y[j]	Trạng thái các biến chỉ số điều khiển	Tiến trình tính mốc start và gọi hàm attempt
1	7	y[7] = 'A'	i = z['A'] = 6 start = 7 - 6 = 1 wall = 0	Do start=1 > wall=0 \$\to\$ đặt wall = 1. Gọi attempt(y, x, 8, 1, 1): k = 1 - 1 = 0. So sánh x[0]='G' khác y[1]='C' \$\to\$ Dừng, trả về k = 0. Cập nhật biên: wall = 1 + 0 = 1.
2	7	y[7] = 'A'	i = list[6] = 5 start = 7 - 5 = 2 wall = 1	Do start=2 > wall=1 \$\to\$ đặt wall = 2. Gọi attempt(y, x, 8, 2, 2): k = 2 - 2 = 0. So sánh x[0]='G' khác y[2]='C' \$\to\$ Dừng, trả về k = 0. Cập nhật biên: wall = 2 + 0 = 2.
3	7	y[7] = 'A'	i = list[5] = 4 start = 7 - 4 = 3 wall = 2	Do start=3 > wall=2 \$\to\$ đặt wall = 3. Gọi attempt(y, x, 8, 3, 3): k = 3 - 3 = 0. So sánh x[0]='G' khác y[3]='T' \$\to\$ Dừng, trả về k = 0. Cập nhật biên: wall = 3 + 0 = 3.
4	7	y[7] = 'A'	i = list[4] = 2 start = 7 - 2 = 5 wall = 3	Do start=5 > wall=3 \$\to\$ đặt wall = 5. Gọi attempt(y, x, 8, 5, 5): Khớp liên tục toàn bộ 8 ký tự từ y[5] trở đi. Trả về k = 8. Tìm thấy mẫu tại vị trí start = 5. Cập nhật biên: wall = 5 + 8 = 13.
Dừng	15	y[15] = 'T'	z['T'] = -1 (Rỗng)	Chỉ số j tăng thêm m: j = 7 + 8 = 15. Do y[15]='T' không nằm trong mẫu nên vòng lặp dừng.

5.2.4. Cài đặt thuật toán

```
// Hàm phụ trợ thực hiện đối sánh ký tự từ mốc đường biên wall trở đi
(image_e44466.png)
int attempt(const std::string& y, const std::string& x, int m, int start,
int wall) {
    int k = wall - start;
    if (k < 0) k = 0; // Đảm bảo chỉ số k không âm

    while (k < m && x[k] == y[k + start]) {
        ++k;
    }
    return k;
}

// Hàm tiền xử lý mpNext tiêu chuẩn của Morris-Pratt
void preMp(const std::string& x, int m, std::vector<int>& mpNext) {
    int i = 0, j = -1;
    mpNext[0] = -1;
    while (i < m) {
```

```

        while (j > -1 && x[i] != x[j]) j = mpNext[j];
        i++;
        j++;
        mpNext[i] = j;
    }
}

// Hàm tiền xử lý kmpNext tiêu chuẩn của Knuth-Morris-Pratt
void preKmp(const std::string& x, int m, std::vector<int>& kmpNext) {
    int i = 0, j = -1;
    kmpNext[0] = -1;
    while (i < m) {
        while (j > -1 && x[i] != x[j]) j = kmpNext[j];
        i++;
        j++;
        if (i < m && x[i] == x[j]) kmpNext[i] = kmpNext[j];
        else kmpNext[i] = j;
    }
}

/**
 * Hàm tìm kiếm chính của giải thuật KmpSkip Search
 */
void KMPSKIP(const std::string& x, int m, const std::string& y, int n) {
    int i, j, k, kmpStart, per, start, wall;
    std::vector<int> kmpNext(XSIZE, 0);
    std::vector<int> mpNext(XSIZE, 0);
    std::vector<int> list(XSIZE, -1);
    std::vector<int> z(ASIZE, -1);

    /* Bước 1: Pha tiền xử lý đồng bộ các mảng liên kết tĩnh và bảng KMP
    */
    preMp(x, m, mpNext);
    preKmp(x, m, kmpNext);

    z[(unsigned char)x[0]] = 0;
    for (i = 1; i < m; ++i) {
        list[i] = z[(unsigned char)x[i]];
        z[(unsigned char)x[i]] = i;
    }

    /* Bước 2: Vòng lặp điều khiển tiến trình tìm kiếm */
    wall = 0;
    per = m - kmpNext[m];
    i = -1;
    j = -1;

    // Vòng lặp do-while tìm ký tự mốc hợp lệ đầu tiên trên văn bản
    do {
        j += m;
    } while (j < n && z[(unsigned char)y[j]] < 0);

    if (j >= n) return;

    i = z[(unsigned char)y[j]];
    start = j - i;

    while (start <= n - m) {
        if (start > wall) {
            wall = start;
        }
    }

    // Gọi hàm attempt so khớp vượt ranh giới kiểm soát wall

```

```

        k = attempt(y, x, m, start, wall);
        wall = start + k;

        if (k == m) {
            std::cout << "Tim thay mau tai vi tri: " << start <<
std::endl;
            i -= per;
        } else {
            kmpStart = start + k - kmpNext[k];
            k = kmpNext[k];
            start = j - i;

            // Vòng lặp điều phối dịch chuyển trạng thái lính canh KMP
            while (start < kmpStart || (kmpStart < start && start <
wall)) {
                if (start < kmpStart) {
                    i = list[i];
                    if (i < 0) {
                        do {
                            j += m;
                        } while (j < n && z[(unsigned char)y[j]] < 0);
                        if (j >= n) return;
                        i = z[(unsigned char)y[j]];
                    }
                    start = j - i;
                } else {
                    kmpStart += (k - mpNext[k]);
                    k = mpNext[k];
                }
            }
        }
    }
}
}
}
}

```

5.3. Thuật toán Alpha Skip Search

5.3.1. Trình bày thuật toán

Ý tưởng cốt lõi của thuật toán Alpha Skip Search là "nâng cấp thuật toán Skip Search bằng cách nhóm các ký tự thành các đoạn xâu con có độ dài cố định là $\log M$ (với cơ số là kích thước bảng chữ cái), sau đó lưu trữ toàn bộ các đoạn này vào một cây tiền tố (Trie) để tăng tốc tra cứu điểm xuất phát tiềm năng". Trong thuật toán Skip Search gốc, việc chỉ kiểm tra 1 ký tự mốc đơn lẻ dễ dẫn đến danh sách liên kết (bucket) bị quá dài khi gặp các ký tự lặp lại nhiều.

Alpha Skip Search giải quyết triệt để vấn đề này bằng cách: Pha tiền xử lý tính toán độ dài đoạn phân rã $\log M$ (ký hiệu là $\log M$). Sau đó, thuật toán trích xuất toàn bộ các đoạn xâu con có độ dài $\log M$ xuất hiện trong xâu mẫu x để xây dựng một cây Trie mang tên $T(x)$. Các nút lá của cây Trie này đại diện cho một đoạn xâu con duy nhất và sẽ quản lý một danh sách liên kết $z[]$ lưu trữ các vị trí chỉ số nơi đoạn đó bắt đầu trong mẫu. Khi tiến hành tìm kiếm, thuật toán lướt trên văn bản y bằng các bước nhảy cố định ($\text{shift} = m - \log M + 1$). Tại mỗi chặng, thay vì đọc 1 ký tự, thuật toán đọc một đoạn văn bản có độ dài đúng bằng $\log M$: $Y[j..j + \log M - 1]$. Đoạn văn bản này được đưa vào cây Trie $T(x)$ để tra cứu. Nếu tìm thấy nút lá tương ứng, thuật toán sẽ duyệt qua danh sách liên kết để lấy vị trí bắt đầu tiềm năng $b = j - \text{current} \rightarrow \text{element}$, rồi kích hoạt lệnh so khớp bộ nhớ memcmp để kiểm tra xâu mẫu. Nếu đoạn văn bản không tồn tại trên cây Trie, thuật toán lập tức nhảy cóc qua cả đoạn một cách an toàn. Để làm được điều này, thuật toán chia

làm 2 bước lớn.

Thuật toán Alpha Skip Search được triển khai tuần tự qua 2 bước sau:

Thuật toán setZ: // Hàm phụ trợ thực hiện cấp phát nút và lưu vị trí vào danh sách của nút lá cây Trie

Input: Chỉ số node trên cây Trie và vị trí chỉ số i trong mẫu x.

Output: Nút lá được liên kết thêm phần tử mới vào danh sách z[node].

Formats: setZ(node, i);

Actions:

cell = <Cấp phát vùng nhớ cho một cấu trúc _cell>;

cell->element = i;

cell->next = z[node];

z[node] = cell;

EndActions.

Thuật toán AlphaSkipSearch: Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m.

- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n.

- Kích thước bảng chữ cái hệ thống là a.

Output: Tất cả vị trí xuất hiện X trong Y.

Formats: AlphaSkipSearch(X, m, Y, n, a);

Actions:

Bước 1 (Tiền xử lý):

// 1. Tính toán độ dài phân rã logM

logM = 0;

temp = m;

while (temp > a) {

 ++logM;

 temp = temp / a;

}

if (logM == 0) {

 logM = 1;

} else if (logM > m / 2) {

 logM = m / 2;

}

// 2. Khởi tạo cây Trie rỗng và cấp phát mảng danh sách liên kết z

size = 2 + (2 * m - logM + 1) * logM;

trie = <Khởi tạo cấu trúc cây Trie mới với kích thước size>;

z = <Cấp phát vùng nhớ mảng con trỏ cho các nút lá của Trie>;

// 3. Quét xâu mẫu để chèn các đoạn độ dài logM vào cây Trie

root = getInitial(trie);

art = newVertex(trie);

setSuffixLink(trie, root, art);

node = newVertex(trie);

setTarget(trie, root, X[0], node);

setSuffixLink(trie, node, root);

```

for (i = 1; i < logM; ++i) {
    node = addNode(trie, art, node, X[i]);
}
pos = 0;
setZ(node, pos);
pos++;

for (i = logM; i < m - 1; ++i) {
    node = getSuffixLink(trie, node);
    childNode = getTarget(trie, node, X[i]);
    if (childNode == UNDEFINED) {
        node = addNode(trie, art, node, X[i]);
    } else {
        node = childNode;
    }
    setZ(node, pos);
    pos++;
}

node = getSuffixLink(trie, node);
childNode = getTarget(trie, node, X[m - 1]);
if (childNode == UNDEFINED) {
    lastNode = newVertex(trie);
    setTarget(trie, node, X[m - 1], lastNode);
    node = lastNode;
} else {
    node = childNode;
}
setZ(node, pos);

```

Bước 2 (Lặp tìm kiếm):

Plaintext

shift = m - logM + 1;

j = m + 1 - logM;

```

while (j < n - logM) {
    node = root;
    k = 0;

    // Đọc đoạn văn bản độ dài logM và tra cứu trên cây tiền tố Trie
    while (node != UNDEFINED && k < logM) {
        node = getTarget(trie, node, Y[j + k]);
        ++k;
    }

    // Nếu đoạn văn bản khớp hoàn toàn với một nút lá trên cây Trie
    if (node != UNDEFINED) {
        current = getZ(node);
        while (current != NULL) {

```

```

b = j - current->element;

// Tiến hành so khớp toàn cục của s2 sau khi xác định được mốc xuất phát b
if (X[0] == Y[b] && memcmp(X + 1, Y + b + 1, m - 1) == 0) {
    <Tìm thấy mẫu ở vị trí b>;
}
current = current->next;
}
}
j = j + shift; // Nhảy cóc cố định qua đoạn văn bản
}
free(z);
EndActions.

```

5.3.2. Đặc điểm thuật toán

- Tiến trình so khớp văn bản được thực hiện hoàn toàn dựa trên cơ chế nhảy chặng cố định với khoảng cách bước nhảy bằng $(m - \log M + 1)$ ô, giúp bỏ qua các phép so sánh ký tự đơn lẻ.
- Pha tiền xử lý đạt hiệu năng tuyến tính tối ưu nếu kích thước bảng chữ cái được xem là một hằng số. Độ phức tạp thời gian tiền xử lý là $O(m)$ và độ phức tạp không gian bộ nhớ lưu trữ cấu trúc cây tiền tố Trie cùng mảng danh sách liên kết là $O(m)$.
- Pha tìm kiếm có độ phức tạp thời gian trong trường hợp xấu nhất là $O(m \times n)$. Tuy nhiên, số lượng phép đối sánh ký tự trung bình thực tế trong trường hợp kỳ vọng (expected behavior) đạt mức rất thấp: $O(\log_{\sigma}(m) \times (n / (m - \log_{\sigma}(m))))$, mang lại tốc độ xử lý chuỗi cực nhanh đối với các bảng dữ liệu có dung lượng lớn.

5.3.3. Kiểm nghiệm thuật toán

Kiểm nghiệm AlphaSkipSearch(x, m, y, n, a):

- **x chuỗi mẫu:** "GCAGAGAG" ($m = 8$),
- **y văn bản:** "GCATCGCAGAGAGTATACAGTACG" ($n = 24$).
- Giả định kích thước bảng chữ cái mục tiêu $a = 4$ (gồm bộ 4 ký tự ký hiệu: A, C, G, T).

Bước 1 (Tiền xử lý):

- Độ dài phân rã chuỗi con: $\log M = \log_4(8) = 1.5 \rightarrow$ Làm tròn gán bằng 3.
- Thuật toán trích xuất các đoạn con độ dài bằng 3 trong mẫu "GCAGAGAG" để chèn vào cây tiền tố Trie):
 - o Đoạn "GCA" (bắt đầu tại chỉ số 0) $\rightarrow z["GCA"] = (0)$
 - o Đoạn "CAG" (bắt đầu tại chỉ số 1) $\rightarrow z["CAG"] = (1)$
 - o Đoạn "AGA" (bắt đầu tại chỉ số 2 và 4) $\rightarrow z["AGA"] = (4, 2)$
 - o Đoạn "GAG" (bắt đầu tại chỉ số 3 và 5) $\rightarrow z["GAG"] = (5, 3)$
- Khoảng cách bước nhảy hằng số: $\text{shift} = m - \log M + 1 = 8 - 3 + 1 = 6$.

Bảng chặng dịch chuyển tìm kiếm:

Lượt quét	Vị trí chỉ số j	Đoạn văn bản kiểm tra $Y[j..j+2]$	Kết quả tra cứu nút lá trên cây Trie	Tiến trình tính mốc xuất phát và so khớp toàn cục
1	6	"GCA"	Tìm thấy! $z["GCA"] = (0)$	Mốc xuất phát: $b = j - \text{element} = 6 - 0 = 6$. Tiến hành so khớp mẫu tại văn bản $y[6]$: Lệch ngay ký tự đầu $x[0]='G'$ khác $y[6]='A'$. → Bỏ qua. Dịch chuyển: $j = 6 + 6 = 12$.
2	12	"GAG"	Tìm thấy! $z["GAG"] = (5, 3)$	* Thử với chỉ số 5: $b = 12 - 5 = 7$. So khớp mẫu tại $y[7]$: Không khớp. * Thử với chỉ số 3: $b = 12 - 3 = 9$. So khớp mẫu tại $y[9]$: Không khớp. Dịch chuyển: $j = 12 + 6 = 18$.
3	18	"AGT"	Rỗng (NULL)	Đoạn xâu con "AGT" hoàn toàn không nằm trong cây tiền tố Trie của mẫu. Thuật toán nhảy cóc an toàn, không tồn bất kỳ phép so sánh chuỗi nào. Dịch chuyển: $j = 18 + 6 = 24$.
Dừng	24			Vòng lặp kết thúc vì $j = 24$ chạm ranh giới điều kiện biên an toàn trên văn bản.

5.3.4. Cài đặt thuật toán

```
// Cấu trúc một nút trong danh sách liên kết quản lý vị trí ở nút lá
struct _cell {
    int element;
    struct _cell* next;
};
typedef struct _cell* List;

// Giả lập cấu trúc ma trận chuyển trạng thái đồ thị của cây tiền tố Trie
struct TrieGraph {
    std::vector<std::vector<int>> target;
    std::vector<int> suffixLink;
    int vertexCount;

    TrieGraph(int size) {
        target.assign(size, std::vector<int>(ASIZE, UNDEFINED));
        suffixLink.assign(size, UNDEFINED);
        vertexCount = 0;
    }
};
typedef TrieGraph* Graph;

int newVertex(Graph trie) {
```

```

        return trie->vertexCount++;
    }

    int getInitial(Graph trie) {
        return 0;
    }

    void setSuffixLink(Graph trie, int node, int link) {
        trie->suffixLink[node] = link;
    }

    int getSuffixLink(Graph trie, int node) {
        return trie->suffixLink[node];
    }

    void setTarget(Graph trie, int node, char c, int childNode) {
        trie->target[node][(unsigned char)c] = childNode;
    }

    int getTarget(Graph trie, int node, char c) {
        return trie->target[node][(unsigned char)c];
    }

    int addNode(Graph trie, int art, int node, char c) {
        int childNode = newVertex(trie);
        setTarget(trie, node, c, childNode);
        setSuffixLink(trie, childNode, art);
        return childNode;
    }

    // Hàm phụ trợ lưu chỉ số vị trí vào nút lá cây Trie (image_e431db.png)
    void setZ(int node, int i, std::vector<List>& z) {
        List cell = (List)std::malloc(sizeof(struct _cell));
        if (cell == nullptr) {
            std::cerr << "Loi cap phat bo nho ALPHASKIP" << std::endl;
            return;
        }
        cell->element = i;
        cell->next = z[node];
        z[node] = cell;
    }

    List getZ(int node, const std::vector<List>& z) {
        return z[node];
    }

    /**
     * Hàm tìm kiếm chính của giải thuật Alpha Skip Search
     */
    void ALPHASKIP(const std::string& x, int m, const std::string& y, int n, int
a) {
        int b, i, j, k, logM, temp, shift, size, pos;
        int art, childNode, node, root, lastNode;

        // 1. Tính toán độ dài phân rã logM (image_e431c1.png)
        logM = 0;
        temp = m;
        while (temp > a) {
            ++logM;
            temp /= a;
        }
        if (logM == 0) logM = 1;
        else if (logM > m / 2) logM = m / 2;
    }

```

```

/* Bước 1: Pha tiền xử lý - Dựng cấu trúc cây tiền tố Trie */
size = 2 + (2 * m - logM + 1) * logM;
Graph trie = new TrieGraph(size);
std::vector<List> z(size, nullptr);

root = getInitial(trie);
art = newVertex(trie);
setSuffixLink(trie, root, art);

node = newVertex(trie);
setTarget(trie, root, x[0], node);
setSuffixLink(trie, node, root);

for (i = 1; i < logM; ++i) {
    node = addNode(trie, art, node, x[i]);
}
pos = 0;
setZ(node, pos, z);
pos++;

for (i = logM; i < m - 1; ++i) {
    node = getSuffixLink(trie, node);
    childNode = getTarget(trie, node, x[i]);
    if (childNode == UNDEFINED) {
        node = addNode(trie, art, node, x[i]);
    } else {
        node = childNode;
    }
    setZ(node, pos, z);
    pos++;
}

node = getSuffixLink(trie, node);
childNode = getTarget(trie, node, x[m - 1]);
if (childNode == UNDEFINED) {
    lastNode = newVertex(trie);
    setTarget(trie, node, x[m - 1], lastNode);
    node = lastNode;
} else {
    node = childNode;
}
setZ(node, pos, z);

/* Bước 2: Pha tìm kiếm chặng dịch cố định trên văn bản y
(image_e431a0.png) */
shift = m - logM + 1;
const char* x_ptr = x.c_str();
const char* y_ptr = y.c_str();

for (j = m - logM; j < n - logM; j += shift) {
    node = root;
    for (k = 0; node != UNDEFINED && k < logM; ++k) {
        node = getTarget(trie, node, y_ptr[j + k]);
    }

    if (node != UNDEFINED) {
        List current = getZ(node, z);
        while (current != nullptr) {
            b = j - current->element;
            if (b >= 0 && b <= n - m) {
                if (x_ptr[0] == y_ptr[b] && std::memcmp(x_ptr + 1, y_ptr
+ b + 1, m - 1) == 0) {

```

```

                                std::cout << "Tim thay mau tai vi tri: " << b <<
std::endl;
                                }
                                }
                                current = current->next;
                                }
                                }
                                }

// Giải phóng vùng nhớ dọn dẹp hệ thống
for (i = 0; i < size; ++i) {
    List curr = z[i];
    while (curr != nullptr) {
        List next_node = curr->next;
        std::free(curr);
        curr = next_node;
    }
}
delete trie;
}

```

KẾT LUẬN

Báo cáo chuyên đề đã mang đến một cái nhìn toàn diện và có hệ thống về bài toán đối sánh chuỗi (string matching) – một thành phần cơ sở cốt lõi đóng vai trò nền tảng trong khoa học máy tính, đặc biệt là trong lĩnh vực xử lý văn bản, hệ điều hành và thiết kế phần mềm.

Xuyên suốt báo cáo, các phương pháp tiếp cận bài toán đã được phân loại dựa trên hướng duyệt và vị trí đối sánh trên văn bản, bao gồm: duyệt xuôi, duyệt ngược, duyệt từ những vị trí mang tính chiến lược, hoặc sử dụng các kỹ thuật nhảy cóc lướt qua dữ liệu. Mặc dù mỗi nhóm giải pháp có cách tư duy khác nhau, nhưng điểm chung mang tính quy luật của hầu hết các thuật toán là việc chia quá trình xử lý thành hai giai đoạn độc lập. Giai đoạn thứ nhất là pha tiền xử lý, nhằm phân tích cấu trúc của chuỗi mẫu để xây dựng các quy tắc, bảng bước nhảy hoặc máy trạng thái. Giai đoạn thứ hai là pha tìm kiếm, tận dụng triệt để những quy tắc vừa khởi tạo để loại bỏ tối đa các phép so sánh dư thừa, giúp quá trình quét văn bản được tăng tốc mạnh mẽ.

Từ việc nghiên cứu tổng quan này, kết luận quan trọng nhất được rút ra là không tồn tại một giải pháp nào hoàn hảo và tối ưu tuyệt đối cho mọi tình huống. Hiệu năng thực tế của từng phương pháp luôn bị chi phối bởi các điều kiện đặc thù của hệ thống như kích thước tập ký tự, độ dài của chuỗi cần tìm và giới hạn về bộ nhớ. Sự phong phú của các phương pháp hiện nay chính là minh chứng cho quá trình kế thừa và cải tiến không ngừng, nơi các giải pháp sau luôn tìm cách khắc phục những điểm yếu về thời gian hoặc không gian của các giải pháp trước đó.

Tóm lại, bài toán đối sánh chuỗi tuy có định nghĩa đơn giản nhưng lại chứa đựng tư duy thuật toán vô cùng tinh tế. Việc nắm vững nguyên lý hoạt động và bản chất tối ưu chung của các nhóm phương pháp sẽ cung cấp cơ sở khoa học vững chắc, giúp các lập trình viên có thể đánh giá và lựa chọn giải pháp phù hợp nhất để giải quyết hiệu quả những bài toán xử lý dữ liệu phức tạp trong thực tế.